

Pega Platform 26.1.1 on Azure Kubernetes Service

Externalized Kafka on Confluent Cloud and search on managed OpenSearch

Design, Helm configuration, testing and operations for six environments built from a cloned and upgraded Pega Platform 8.8 database

Document version	2.1
Status	Issued for customer review and approval
Issue date	2 October 2026
Classification	Customer Confidential
Target	Pega Platform 26.1.1 on Azure AKS, Pega Helm charts 4.13.0
Source	Pega Platform 8.8 with embedded Kafka and embedded Elasticsearch
Environments	DEV, SIT, UAT, PERF, PREPROD, PROD

Document control

Item	Detail
Title	Pega Platform 26.1.1 on Azure AKS: externalized Kafka on Confluent Cloud and search on managed OpenSearch
Version	2.1
Status	Issued for customer review and approval
Issue date	2 October 2026
Evidence cut-off	Vendor pages were read between 28 September and 2 October 2026. Section 3 lists the facts used. Appendix F lists every source with its link.
Supersedes	Version 2.0 of this runbook, issued on 1 October 2026. Version 2.0 itself replaced version 1.0, which assumed a three-release path through an intermediate Pega release before the customer confirmed the clone-and-upgrade path (Section 1.2).
Owner	Customer platform engineering lead
Review cycle	Before each rehearsal, before the production cutover, and after any change to a vendor page listed in Section 3

Revision history

Version	Date	Change	Author
1.0	1 October 2026	Kafka and search runbook for a three-release upgrade path.	Implementation team
2.0	1 October 2026	Rewritten for the clone-and-upgrade path, six environments, shared non-production services, managed OpenSearch and Okta.	Implementation team
2.1	2 October 2026	Expert review: Helm values corrected after rendering with helm template (SRS authentication, SRS port 8443, SRS network policy, SRS replicas and resources); availability zone design; Confluent connection, request and partition-creation limits; Okta key rotation and rate limits; OpenSearch sizing formulas; JVM settings; new Sections 15 (performance engineering and sizing) and 18 (observability); environment build waves; Appendix H sizing calculator. Sections 15 to 18 of version 2.0 are now Sections 16, 17, 19 and 20.	Implementation team

Approvals

Approval confirms that the approver has reviewed the sections relevant to the role and accepts the design, the recommendations and the open decisions as stated.

Role	Name	Signature	Date
Customer business owner			
Customer enterprise architect			
Customer security architect			
Customer platform engineering lead			
Customer database administration lead			

Role	Name	Signature	Date
Pega Lead System Architect			
Kafka and search engineering lead			

Contents

1 Executive summary.....	13
1.1 Purpose.....	13
1.2 The clone-and-upgrade approach.....	13
1.3 Kafka and search design in brief.....	14
1.4 Sharing services to reduce cost.....	15
1.5 Answer to the migration question.....	15
1.6 Questions for Pega Support.....	15
1.7 Decisions needed.....	16
1.8 Readiness statement.....	16
2 Scope, dependencies and reading guide.....	17
2.1 Scope.....	17
2.2 Dependencies covered only to the depth Kafka and search need.....	17
2.3 Confirmed customer inputs.....	18
2.4 Where each question is answered.....	18
2.5 Reading guide by role.....	19
2.6 Conventions and naming.....	19
3 Evidence baseline.....	20
3.1 How the evidence was gathered.....	20
3.2 Facts this design depends on.....	20
4 Architecture.....	22
4.1 Pega reference architecture for '26.....	22
4.2 Target architecture on Azure.....	23
4.3 Logical flows.....	24
4.4 Network and DNS.....	25
4.5 Message path inside Pega.....	26
4.6 Availability zone resilience.....	26
5 Shared and dedicated service model.....	29
5.1 Grouping options.....	29
5.2 Recommended model.....	30
5.3 Isolation controls.....	32
5.4 Noisy neighbours and quotas.....	33
5.5 Cost comparison model.....	33
5.6 Production.....	34
6 Confluent Cloud design and configuration.....	35
6.1 What Pega uses Kafka for.....	35
6.2 Pega requirements mapped to Confluent Cloud.....	35
6.3 Cluster type per group.....	36
6.4 Private networking.....	37
6.5 Service accounts, authentication and ACLs.....	39
6.6 Topic design.....	40
6.7 Partition budget.....	40
6.8 Helm stream configuration.....	41

6.9 prpcUtils on 26.1.1.....	41
7 Managed OpenSearch and SRS design and configuration.....	43
7.1 How search works in 26.1.1.....	43
7.2 SRS and OpenSearch versions.....	43
7.3 Provider selection.....	44
7.4 Cluster settings and sizing.....	45
7.5 Isolation in the shared OpenSearch services.....	47
7.6 Pega-to-SRS tokens with Okta.....	48
7.7 SRS deployment (backingservices chart).....	51
7.8 Index build planning.....	53
7.9 Backups and restore.....	53
8 Secrets, certificates and identity.....	54
8.1 Secret inventory.....	54
8.2 External Secrets Operator manifests.....	54
8.3 Rotation.....	55
8.4 Certificates.....	56
9 Helm configuration per environment.....	57
9.1 Chart version and the three runs.....	57
9.2 Pega chart keys per environment.....	57
9.3 Backingservices chart keys per environment.....	59
9.4 Differences between the runs.....	60
9.5 Pipeline rules.....	60
10 The cloned database.....	61
10.1 Gating questions for Pega Support.....	61
10.2 How Helm settings and database settings interact.....	61
10.3 Inventory and clean-up.....	62
10.4 First-start control.....	63
10.5 Data protection and masking.....	65
11 The 8.8 content question.....	66
11.1 Answer.....	66
11.2 Treatment per content type.....	66
11.3 Application Kafka data sets.....	67
11.4 Why the Stream Migration activity is not used.....	67
11.5 Statement for approval.....	68
12 Deployment runbook per environment.....	69
12.1 Build order.....	69
12.2 Network and DNS.....	69
12.3 Secrets and identity.....	69
12.4 Confluent Cloud.....	70
12.5 OpenSearch.....	70
12.6 SRS.....	70
12.7 Database clone and upgrade.....	71
12.8 First start, index build and intake release.....	71
12.9 Per-environment notes.....	71

12.10 Building the environments one after another.....	71
13 Production cutover and rollback.....	74
13.1 Principles.....	74
13.2 Milestones.....	74
13.3 Cutover sequence.....	74
13.4 Go/No-Go gates.....	75
13.5 Application Kafka data set cutover.....	75
13.6 Rollback.....	76
14 Test strategy, rehearsals and failure scenarios.....	77
14.1 Test stages.....	77
14.2 Rehearsals and measurements.....	77
14.3 Isolation tests.....	77
14.4 Functional and integration checks.....	78
14.5 Performance tests.....	78
14.6 Failure scenario catalogue.....	79
14.7 Exact checks.....	81
14.8 Acceptance criteria.....	81
15 Performance engineering and sizing.....	82
15.1 Why performance testing is required.....	82
15.2 Workload model.....	82
15.3 Test types and measurement points.....	83
15.4 Tools and harness.....	84
15.5 Entry and exit criteria.....	84
15.6 Pitfalls seen in Pega performance tests.....	84
15.7 Sizing method.....	85
15.8 Sizing rules per layer.....	85
15.9 Worked example.....	86
16 Known issues and challenges.....	87
16.1 Cloned database.....	87
16.2 Kafka and Confluent.....	87
16.3 OpenSearch and SRS.....	88
16.4 Platform.....	89
16.5 Process.....	89
17 Troubleshooting.....	90
17.1 Method.....	90
17.2 Kafka.....	90
17.3 Search.....	91
17.4 Cloned database and upgrade.....	92
17.5 Support cases.....	92
18 Observability, monitoring and logging.....	93
18.1 Telemetry sources.....	93
18.2 Logging design.....	94
18.3 Pega Diagnostic Center.....	94
18.4 Dashboards.....	94

18.5 Alerts and thresholds.....	95
18.6 Synthetic checks.....	96
18.7 Following an incident across services.....	96
19 Operations, onboarding, refresh and retirement.....	97
19.1 Routine tasks.....	97
19.2 Onboarding a new non-production environment.....	97
19.3 Refreshing an environment from a new clone.....	97
19.4 Retiring an environment.....	98
19.5 Roles and responsibilities.....	98
20 Risks, open decisions, assumptions and source reconciliation.....	99
20.1 Risk register.....	99
20.2 Open decisions.....	99
20.3 Assumptions.....	100
20.4 Source reconciliation.....	101
Appendix A. Configuration inventory.....	102
A.1 Names per environment.....	102
A.2 Values recorded per service group.....	102
A.3 Values recorded per environment.....	103
Appendix B. Complete Helm values files.....	104
B.1 SIT: pega chart, upgrade run.....	104
B.2 SIT: pega chart, first-start run.....	106
B.3 SIT: pega chart, steady-state deploy run.....	107
B.4 SIT: backingservices chart (SRS).....	109
B.5 SIT: network policies for SRS.....	110
B.6 PROD: pega chart, upgrade run.....	111
B.7 PROD: pega chart, first-start run.....	113
B.8 PROD: pega chart, steady-state deploy run.....	114
B.9 PROD: backingservices chart (SRS).....	116
B.10 PROD: network policies for SRS.....	117
Appendix C. Command reference.....	119
C.1 Kafka and Confluent Cloud.....	119
C.2 OpenSearch and SRS.....	120
C.3 Okta tokens.....	120
C.4 Kubernetes and Helm.....	121
Appendix D. Evidence and rehearsal templates.....	122
D.1 Evidence pack contents.....	122
D.2 Clean-up record.....	122
D.3 Check record.....	123
D.4 Timed cutover record.....	123
D.5 Go/No-Go record.....	123
D.6 Retirement record.....	124
Appendix E. Glossary.....	125
Appendix F. References.....	126

Appendix G. Image sources and attribution..... 129

Appendix H. Sizing calculator..... 130

 H.1 Example outputs..... 130

List of figures

Figure 1. Clone-and-upgrade flow for each environment.....	14
Figure 2. Pega Platform Kubernetes architecture for '26.....	22
Figure 3. Earlier Pega cluster layout with embedded Kafka and in-cluster Elasticsearch.....	23
Figure 4. Externalized services in a Pega deployment.....	23
Figure 5. Target architecture for one environment on Azure AKS.....	24
Figure 6. Logical flows between Pega, Kafka, SRS, OpenSearch and Okta.....	25
Figure 7. Name resolution and traffic paths for Kafka, OpenSearch and Okta.....	25
Figure 8. Message path from producer to batch tier.....	26
Figure 9. Spreading each layer across three availability zones.....	27
Figure 10. Decision: placing a non-production environment on shared services.....	30
Figure 11. Shared non-production topology and per-environment boundaries.....	31
Figure 12. Production topology with dedicated services.....	31
Figure 13. Isolation layers between environments on shared services.....	32
Figure 14. The two Kafka use cases in Pega Platform.....	35
Figure 15. Decision: Confluent Cloud cluster type.....	37
Figure 16. Azure Private Link between a customer VNet and Confluent Cloud.....	38
Figure 17. DNS records that Confluent lists for Azure Private Link.....	38
Figure 18. Decision: Pega to Kafka authentication.....	39
Figure 19. Client-managed cloud deployment with SRS as a backing service.....	43
Figure 20. Decision: managed OpenSearch provider.....	44
Figure 21. OpenSearch cluster roles: cluster-manager and data nodes.....	45
Figure 22. Decision: one SRS per environment or one shared SRS.....	47
Figure 23. Token flow between Pega, Okta and SRS.....	48
Figure 24. Secret delivery from Key Vault to Pega, installer and SRS pods.....	54
Figure 25. Decision: treatment of an item found in the cloned database.....	62
Figure 26. Decision: Go/No-Go at the first start.....	64
Figure 27. Decision: does 8.8 content need migrating?.....	66
Figure 28. Decision: treatment of each application Kafka data set.....	67
Figure 29. Environment build waves and the gate between each.....	72
Figure 30. Production cutover sequence from the Kafka and search point of view.....	74
Figure 31. Performance test harness and measurement points in PERF.....	83
Figure 32. Sizing loop from production facts to signed-off PROD sizes.....	85
Figure 33. Connectivity chain for Kafka and search.....	90
Figure 34. Troubleshooting tree: stream service.....	90
Figure 35. Troubleshooting tree: search.....	91
Figure 36. Telemetry sources, collection, stores and use.....	93

List of tables

Table 1. Kafka and search design by environment group.....	15
Table 2. Decisions needed before the first build.....	16
Table 3. Scope of this document.....	17
Table 4. Dependencies and their gates.....	17
Table 5. Confirmed customer inputs.....	18
Table 6. Traceability from customer questions to sections.....	18
Table 7. Reading guide by role.....	19
Table 8. Environment naming.....	19
Table 9. Version and support facts used in this design.....	20
Table 10. Target architecture components.....	24
Table 11. Network flows to allow and to deny.....	26
Table 12. Availability zone design by layer.....	27
Table 13. Non-production grouping options.....	29
Table 14. Isolation controls by layer.....	32
Table 15. Cost model inputs supplied by the customer.....	33
Table 16. Cost model by option (multiply each line by the customer's unit prices).....	33
Table 17. Pega Kafka requirements mapped to Confluent Cloud.....	35
Table 18. Confluent Cloud cluster limits relevant to Pega (source [R31]; Basic and Standard rows for connections and requests are not used here).....	36
Table 19. Cluster type by group.....	37
Table 20. Private Link set-up steps per cluster.....	38
Table 21. Kafka ACLs for one environment's service account (source [R11]).....	39
Table 22. Topic design settings.....	40
Table 23. Partition budget worksheet.....	41
Table 24. SRS compatibility with OpenSearch for Pega 8.6 and later, SRS 1.44.3 or later (source [R14]).....	43
Table 25. OpenSearch provider selection worksheet.....	44
Table 26. Pega default sizing for search and SRS (source [R14]).....	46
Table 27. Starting sizes per OpenSearch service.....	46
Table 28. OpenSearch sizing formulas.....	47
Table 29. SRS per environment compared with a shared SRS.....	48
Table 30. Options for the guid claim.....	49
Table 31. Okta values used in the Helm charts.....	50
Table 32. SRS chart behaviours found by rendering.....	52
Table 33. Recovery choices for search.....	53
Table 34. Secret inventory (one set per environment, in kv-pegasvc- <code><code></code>).....	54
Table 35. Secret rotation.....	55
Table 36. Certificates.....	56
Table 37. Helm runs of the pega chart for an environment built from a clone.....	57
Table 38. Pega chart keys per environment (chart 4.13.0).....	57
Table 39. Backingservices chart keys per environment.....	59
Table 40. Values that change between runs.....	60
Table 41. Gating questions for Pega Support.....	61
Table 42. Kafka and search items in the cloned database.....	62
Table 43. First-start steps for an environment built from a clone.....	64

Table 44. Treatment of 8.8 content.....	66
Table 45. Application Kafka data set register (one row per data set, completed with the application team).....	67
Table 46. Build order.....	69
Table 47. Network and DNS steps.....	69
Table 48. Secrets and identity steps.....	69
Table 49. Confluent Cloud steps.....	70
Table 50. OpenSearch steps.....	70
Table 51. SRS steps.....	70
Table 52. Database clone and upgrade steps.....	71
Table 53. Per-environment notes.....	71
Table 54. Environment build waves.....	72
Table 55. Configuration checklist for each new environment.....	72
Table 56. Programme milestones used in this document.....	74
Table 57. Production cutover steps.....	74
Table 58. Go/No-Go gates.....	75
Table 59. Rollback steps.....	76
Table 60. Test stages.....	77
Table 61. Rehearsal measurements.....	77
Table 62. Isolation tests.....	78
Table 63. Performance tests.....	78
Table 64. Failure scenario catalogue.....	79
Table 65. Exact checks.....	81
Table 66. Acceptance criteria.....	81
Table 67. Workload model inputs.....	82
Table 68. Performance test types.....	83
Table 69. Measurement points.....	84
Table 70. Performance test entry, exit and suspension criteria.....	84
Table 71. Sizing rules per layer.....	85
Table 72. Worked sizing example (illustrative inputs).....	86
Table 73. Known issues: cloned database.....	87
Table 74. Known issues: Kafka and Confluent.....	87
Table 75. Known issues: OpenSearch and SRS.....	88
Table 76. Known issues: platform.....	89
Table 77. Kafka symptoms.....	91
Table 78. Search symptoms.....	91
Table 79. Cloned database and upgrade symptoms.....	92
Table 80. Telemetry sources.....	93
Table 81. Dashboards.....	94
Table 82. Alerts, thresholds and responses.....	95
Table 83. Synthetic checks.....	96
Table 84. Identifiers used to correlate one environment across systems.....	96
Table 85. Routine operations.....	97
Table 86. RACI (Okta objects are owned by the customer identity team: A/R).....	98
Table 87. Risk register (L likelihood, I impact: H high, M medium, L low).....	99
Table 88. Open decisions.....	99
Table 89. Assumptions.....	100

Table 90. Source reconciliation register.....101

Table 91. Names and sizes per environment.....102

Table 92. Values recorded per service group.....102

Table 93. Values recorded per environment (one copy per environment).....103

Table 94. Evidence pack items.....122

Table 95. Clean-up record (one per clone).....122

Table 96. Check record.....123

Table 97. Timed cutover record (rehearsals and production).....123

Table 98. Go/No-Go record.....123

Table 99. Retirement record.....124

Table 100. Glossary of terms.....125

Table 101. Third-party images.....129

Table 102. Sizing calculator sheets.....130

Table 103. Calculator example: Confluent capacity units (illustrative inputs).....130

Table 104. Calculator example: OpenSearch data nodes (illustrative inputs).....130

1 Executive summary

1.1 Purpose

This runbook tells the customer's IT teams how to design, configure, test and run Kafka and search for Pega Platform 26.1.1 on Azure Kubernetes Service (AKS). Kafka is provided by Confluent Cloud. Search is provided by the Pega Search and Reporting Service (SRS) on top of a managed OpenSearch service. Both are configured through the Pega Helm charts in six environments: DEV, SIT, UAT, PERF, PREPROD and PROD.

1.2 The clone-and-upgrade approach

The customer is moving from Pega Platform 8.8 without upgrading the running 8.8 servers. A new 26.1.1 platform is built on AKS with Kafka and search outside Pega from the start. The 8.8 database is cloned, and the clone is upgraded to 26.1.1 by the Pega installer, run as the Helm chart installer job. Pega 26.1.1 then starts on the upgraded clone. Each lower environment is built the same way, which gives the programme repeated rehearsals. Production uses a final clone taken after 8.8 is stopped. The 8.8 system and its database stay untouched until the rollback window closes. Figure 1 shows the flow.

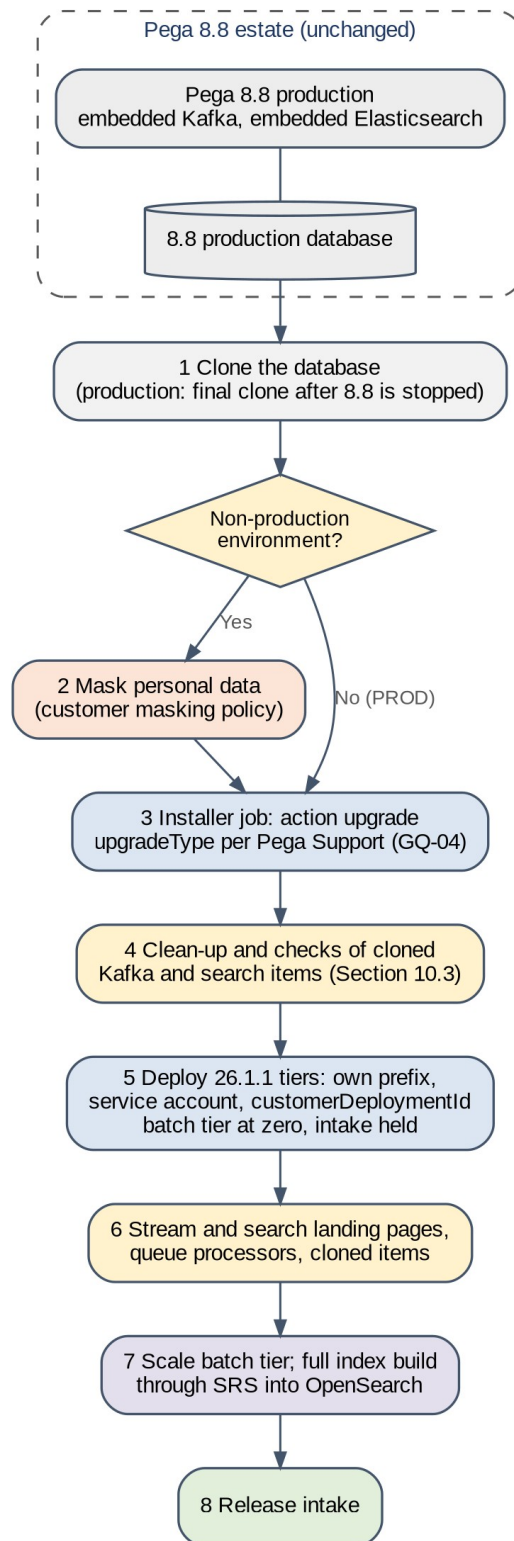


Figure 1. Clone-and-upgrade flow for each environment
 Prepared for this implementation. Not a Pega-published diagram.

1.3 Kafka and search design in brief

Table 1 summarises the design. Production has its own Kafka cluster and its own OpenSearch service. The five non-production environments share two sets of services, with isolation enforced per environment.

Table 1. Kafka and search design by environment group

Layer	PROD	PERF and PREPROD (group NP2)	DEV, SIT and UAT (group NP1)
Kafka	Confluent Cloud cluster cc-prd, Enterprise or Dedicated, Azure Private Link	Shared cluster cc-np2, same type and settings as PROD	Shared cluster cc-np1, Enterprise with Private Link
Isolation in Kafka	Dedicated cluster	Topic prefix, service account, prefixed ACLs and client quota per environment	Same as NP2
Search storage	OpenSearch service os-prd	Shared service os-np2, same provider and settings as PROD	Shared service os-np1
Isolation in search	Dedicated service	SRS per environment, customerDeploymentId per environment, OpenSearch user and index-scoped role per environment	Same as NP2
Pega-to-SRS tokens	Okta custom authorization server for PROD only	Shared non-production authorization server, one client per environment	Same as NP2
Secrets	Key Vault per environment, delivered by the External Secrets Operator	Same	Same

1.4 Sharing services to reduce cost

Six dedicated Kafka clusters and six OpenSearch services would cost the most and need the most operation. The recommended model uses three sets of services instead of six. DEV, SIT and UAT share one set. PERF and PREPROD share a second set that matches production, so load tests and production rehearsals run on production-like services without affecting functional testing. Section 5 compares this model with the alternatives and gives a cost model that the customer completes with its own prices.

1.5 Answer to the migration question

Decision. No Pega stream topic and no search index is copied from 8.8. The 8.8 system uses embedded Kafka and embedded Elasticsearch, whose data lives on the 8.8 nodes, not in the database. Pega states that existing stream data cannot be moved to a new Kafka [\[R13\]](#), and SRS indexes all searchable data after Pega connects to it [\[R15\]](#).

Before the production clone is taken, intake on 8.8 is held and queues are allowed to empty. Queue items that are held in the database, such as delayed and broken items, travel with the clone and are handled as listed in Section 10.3. Application Kafka data sets on the customer's own Kafka clusters are decided one by one (Section 11.3).

1.6 Questions for Pega Support

Pega documentation does not settle three points that the plan depends on. Section 10.1 lists eight questions in full. The three that gate the production plan are these:

- GQ-01: Pega Support must confirm that a direct upgrade of an 8.8 database to 26.1.1 is supported, and name any required 8.8 patch level.
- GQ-03: Pega states that Hazelcast must be removed before updating to '25 or later [\[R5\]](#). Pega Support must confirm how this applies to a cloned database that is upgraded offline and never started on 8.8 again.
- GQ-04: Pega Support must confirm the installer upgrade type for a cloned database ([in-place](#) or the out-of-place types the Helm chart documents [\[R23\]](#)).

1.7 Decisions needed

Table 2 lists the decisions that block the first build. Section 20.2 lists all open decisions with owners and milestones.

Table 2. Decisions needed before the first build

ID	Decision	Recommended answer	Needed by
OD-01	Pega Support answers to the gating questions	Raise the case now; plan with the conservative answer until the reply arrives	M2 DEV build
OD-02	Confluent cluster types	Enterprise with Private Link for NP1; PROD type for NP2 and PROD after partition measurement	M2 DEV build
OD-03	OpenSearch provider	Provider that passes the selection in Section 7.3	M2 DEV build
OD-05	Okta design	Dedicated custom authorization server for PROD; one shared non-production server with a per-client guid claim, if proven in DEV	M2 DEV build
OD-07	Masking approach	Mask before the upgrade, so the index build uses masked data only	M2 DEV build

1.8 Readiness statement

This document is ready for customer review as a design and runbook. Durations for draining, cloning, upgrading and index building depend on data volume, so they come from the rehearsals in Section 14.2, not from estimates. Points that no source settles are marked as tests, Pega Support questions or open decisions. Conflicts between sources are logged in Section 20.4.

2 Scope, dependencies and reading guide

2.1 Scope

Table 3. Scope of this document

In scope	Out of scope
Confluent Cloud and managed OpenSearch design for DEV, SIT, UAT, PERF, PREPROD and PROD. Configuration in the Pega Helm charts: the stream and pegasearch sections of the pega chart, and the SRS section of the backingservices chart. What the cloned 8.8 database brings that affects Kafka and search, and how it is cleaned up before and after the first 26.1.1 start. Whether Kafka topics and search indexes are migrated from 8.8, and how 8.8 in-flight work is handled at cutover. Shared non-production services with isolation between environments, and the cost model. Okta as the token issuer for Pega-to-SRS authorization. Testing, failure scenarios, known issues, troubleshooting and operations for Kafka and search.	Application functional regression content. Cassandra and Decision Data Store. Constellation. Pega Diagnostic Center beyond the network path it needs. General AKS build-out not related to Kafka or search. Commercial terms and prices from Confluent and the OpenSearch provider.

2.2 Dependencies covered only to the depth Kafka and search need

The database clone, the installer upgrade and Pega's '25 and '26 prerequisites belong to other workstreams. Table 4 names the owner of each, the gate it must pass before Kafka and search work can rely on it, and the source.

Table 4. Dependencies and their gates

Dependency	Why it matters for Kafka and search	Owner	Gate	Source
Database clone and masking	The clone carries 8.8 stream and search settings and queue items (Section 10.3). Masking decides what data the index build uses.	DBA team	Clone restored and masked; row counts recorded	Customer policy
Installer upgrade (installer.upgrade , upgradeType)	The upgrade must finish before Pega starts against Kafka and SRS.	Platform team with Pega LSA	Installer job completed; log reviewed	[R23]
Hazelcast removal	From '25, cluster messaging that Hazelcast carried uses Kafka, so a Kafka outage affects more than queue processing.	Pega LSA	GQ-03 answered	[R4, R5, R7]
Tomcat 10.1 and Jakarta EE	Custom JARs that fail to load can stop nodes before Kafka and search checks run.	Application team	Custom JARs rebuilt and tested	[R3, R9]
Primary keys on all tables	Required before an update to '25 or later.	DBA team	primaryKeyUtility dry run clean	[R3, R10]
Queue processor changes in '26	Partitioned, multi-threaded processing; DSS delayeditems/dataflowbased/threadspersnode must be removed (PEGA0179).	Pega LSA	DSS removed on the clone	[R3]

2.3 Confirmed customer inputs

Table 5. Confirmed customer inputs

Input	Value	Effect on this document
Environments	DEV, SIT, UAT, PERF, PREPROD, PROD	Six columns in every per-environment table; six prefixes, service accounts, deployment IDs, Okta clients and Key Vaults.
8.8 Kafka	Embedded Kafka (Pega stream service inside 8.8)	Nothing in it can be moved to Confluent Cloud. Queues are emptied on 8.8 before the final clone (Section 11).
8.8 search	Embedded Elasticsearch	Indexes are on the 8.8 nodes' disks and cannot be carried over. 26.1.1 builds all indexes through SRS (Section 7.8).
Token issuer for Pega to SRS	Okta	Okta design in Section 7.6.
OpenSearch provider	Not chosen yet	Selection method in Section 7.3; Helm values written so only the endpoint, port, credentials and CA depend on the provider.
Pega Support answers	Not received	Every gating question is open with an owner and a milestone (Section 10.1).

2.4 Where each question is answered

The customer asked eighteen questions. Table 6 shows where each one is answered, so a reviewer can check that none is missed.

Table 6. Traceability from customer questions to sections

#	Question	Answered in
1	Is the direct clone-and-upgrade path from 8.8 to 26.1.1 supported, and on what conditions?	Section 10.1
2	How is Confluent Cloud designed per environment?	Sections 5 and 6
3	How is the managed OpenSearch service designed per environment?	Sections 5 and 7
4	How is each one configured in the Helm charts for the upgrade run and the deploy run?	Section 9 and Appendix B
5	What in the cloned 8.8 database conflicts with the new configuration, and how is it cleaned up?	Section 10.3
6	How is a cloned non-production environment kept away from production Kafka, integrations and other environments?	Sections 5.3, 10.4 and 14.3
7	Must topics or indexes be migrated from 8.8, and how is in-flight work handled at cutover?	Sections 11 and 13
8	How do several non-production environments share one cluster and one OpenSearch service safely?	Sections 5.3 and 5.4
9	How much does sharing save, and what risk does it add?	Section 5.5
10	How is it tested?	Section 14
11	Which failures are tested, and how?	Section 14.6
12	What known issues should be avoided?	Section 16
13	How is it run day to day, including refresh and retirement?	Section 19
14	Why is performance testing needed, and how is it done?	Section 15
15	How is each layer sized, and is there a calculator?	Sections 15.7 to 15.9 and Appendix H
16	How is the platform monitored, logged and observed?	Section 18

#	Question	Answered in
17	In which order are the environments built, and what is configured in each one?	Section 12.10
18	What happens when an availability zone fails?	Section 4.6

2.5 Reading guide by role

Table 7. Reading guide by role

Reader	Read first	Then
Business owner, programme manager	Sections 1, 5.5, 13 and 20	Sections 14.2 and 15.1
Enterprise and security architects	Sections 4, 5, 7.6 and 8	Sections 18 and 20
Pega Lead System Architect	Sections 10 and 11	Sections 13 to 16
Database administrators	Sections 10.1, 10.3 to 10.5 and 12.7	Section 13
Platform, Kafka and search engineers	Sections 6 to 9 and 12	Sections 15, 17, 18 and 19, Appendices A to C and H
Test lead, performance lead	Sections 14 and 15	Section 16
Operations and monitoring teams	Section 18	Sections 17 and 19

2.6 Conventions and naming

- **Must** marks a vendor requirement or a fixed decision. **Should** marks a recommendation that the customer can change through the decision process in Section 20.2.
- Values in angle brackets, such as `<bootstrap-host>`, are environment-specific. Each one is listed in Appendix A.
- References in square brackets, such as [R11], point to the numbered sources in Appendix F.
- Figures marked "Prepared for this implementation" were drawn for this document. Figures from Pega, Pega Academy, Confluent and OpenSearch carry a source line, and Appendix G lists them.

Every environment uses the same naming pattern, shown in Table 8. Prefixes end with a hyphen, and no deployment ID is the start of another, so a prefixed ACL or an index pattern for one environment can never match another environment's topics or indexes.

Table 8. Environment naming

Environment	Code	Group	Topic prefix	customerDeploymentId	Service account	Okta client
DEV	dev	NP1	pega-dev-	pega26-dev	sa-pega-dev	pega-srs-dev
SIT	sit	NP1	pega-sit-	pega26-sit	sa-pega-sit	pega-srs-sit
UAT	uat	NP1	pega-uat-	pega26-uat	sa-pega-uat	pega-srs-uat
PERF	perf	NP2	pega-perf-	pega26-perf	sa-pega-perf	pega-srs-perf
PREPROD	ppd	NP2	pega-ppd-	pega26-ppd	sa-pega-ppd	pega-srs-ppd
PROD	prd	PRD	pega-prd-	pega26-prd	sa-pega-prd	pega-srs-prd

3 Evidence baseline

3.1 How the evidence was gathered

Every product statement in this document comes from one of these sources, in this order of authority:

1. Pega documentation for Pega Platform '26, and for '25 where the topic belongs to that release.
2. The Pega Helm charts repository on GitHub, chart version 4.13.0, for configuration keys and defaults.
3. Pega Academy for architecture illustrations.
4. Confluent Cloud documentation.
5. OpenSearch documentation, and the chosen provider's documentation once OD-03 is decided.
6. Okta developer documentation.
7. Microsoft Learn and the External Secrets Operator documentation.

Where two sources disagree, the higher one is followed and the conflict is logged in Section 20.4. Where no source settles a point, the document says so and turns it into a test, a Pega Support question or an open decision.

3.2 Facts this design depends on

Table 9 records the facts as published on the evidence cut-off date. Re-check them before each rehearsal, because vendors update these pages.

Table 9. Version and support facts used in this design

Topic	Fact	Source
Externalized services	From Pega Platform '24.2, embedded Kafka, Elasticsearch and Cassandra are not supported. Kafka and search must be external.	[R2]
Kubernetes	AKS is supported. x86-64 nodes only. Database in the same region as the worker nodes. Helm 3.0 or later.	[R20, R27]
Kafka role	"All Queue Processors and Job Schedulers in Pega Platform require Kafka." Without it, Pega Platform is not fully functional.	[R11]
Kafka client	Pega '26 ships the Kafka client library 4.0.0, backward compatible with 3.9.2.	[R3]
Kafka settings	message.max.bytes 5000000; replica.fetch.max.bytes 5000100; unclean.leader.election.enable false; auto.create.topics.enable false.	[R11]
Kafka security	Generic Kafka security only; vendor-specific features such as IAM roles are not supported.	[R11]
Kafka ACLs	TOPIC and GROUP on the stream name prefix (ALL, PREFIXED); TRANSACTIONAL_ID * (READ and WRITE, LITERAL); CLUSTER IDEMPOTENT_WRITE.	[R11]
Stream data	Existing stream data cannot be moved to a new Kafka provider.	[R13]
Helm stream keys	<code>stream.saslMechanism</code> accepts PLAIN, SCRAM-SHA-256 and SCRAM-SHA-512. External secret keys: <code>STREAM_TRUSTSTORE_PASSWORD</code> , <code>STREAM_KEYSTORE_PASSWORD</code> , <code>STREAM_JAAS_CONFIG</code> .	[R23]
SRS role	SRS replaces embedded search and the legacy plug-in, which are not available from '24.2.	[R14]
SRS matrix	SRS 1.44.3 or later. Image <code>search-n-reporting-service-os</code> supports OpenSearch 1.3, 2.15 and 2.19; best practice OpenSearch 2.15.	[R14]
SRS sharing	SRS can support more than one Pega deployment; it isolates data with a unique <code>CUSTOMER_DEPLOYMENT_ID</code> . Do not share the search service with non-Pega software.	[R14]
Deployment ID	The <code>customerDeploymentId</code> is the prefix of all indexes and must equal the <code>guid</code> claim when OAuth is used. It defaults to the namespace name.	[R23]
Index build	After Pega connects to SRS, SRS indexes all searchable data, which requires a downtime period.	[R15]

Topic	Fact	Source
Search settings	Index auto-creation must be off (SRS sets it if its user has the manage cluster privilege). destructive_requires_name false.	[R14]
SRS tokens	OAuth client credentials with private_key_jwt or client_secret_basic; scope pega.search:full; private key in SRS_OAUTH_PRIVATE_KEY.	[R23]
Hazelcast	Removed in '25. "Before updating to Pega Platform '25, remove Hazelcast from your deployment." Affects embedded Hazelcast and the Clustering Service.	[R5]
Queue processing in '26	Partitioned, multi-threaded queue processing. Remove DSS delayeditems/dataflowbased/threadspernode (PEGA0179). SystemPulse topic uses 6 partitions.	[R3]
Installer actions	install, deploy, install-deploy, upgrade, upgrade-deploy. Upgrade types include in-place, out-of-place-rules, out-of-place-data, zero-downtime, custom.	[R23]
Confluent quotas	Client quotas are supported on Enterprise, Freight and Dedicated clusters, applied per service account or identity pool, not per API key.	[R36]
Okta	Custom scopes and custom claims need a custom authorization server. The org authorization server cannot be customized.	[R42, R43, R44]
Okta keys and limits	Authorization server signing keys rotate about four times a year; clients look keys up by kid. Rate limits apply per org; excess requests receive HTTP 429.	[R52, R53]
Confluent limits	Per Enterprise eCKU: 18,000 connections, 500 connection attempts per second, 7,500 requests per second; partition creation and deletion 500 per five minutes per cluster; limits enforced from 2026. 99.99 % SLA from 2 eCKU.	[R31]
Pega partitions	New stream topics get 6 partitions by default (DSS pyTopicPartitionsCount); queue processor threads beyond the partition count do no work.	[R49, R50]
SRS chart	srsStorage.tls is certificate authentication and cannot be combined with basic authentication; minimum 2 SRS replicas, 3 as best practice.	[R24]

4 Architecture

4.1 Pega reference architecture for '26

Pega's Kubernetes architecture for '26 has three layers: the Pega application namespace (web and batch deployments and an installer job), backing services that Pega provides (SRS and Constellation), and services the client provides (search, Kafka, Cassandra for Customer Decision Hub only, and the database) [R21]. Hazelcast does not appear, because '25 removed it.

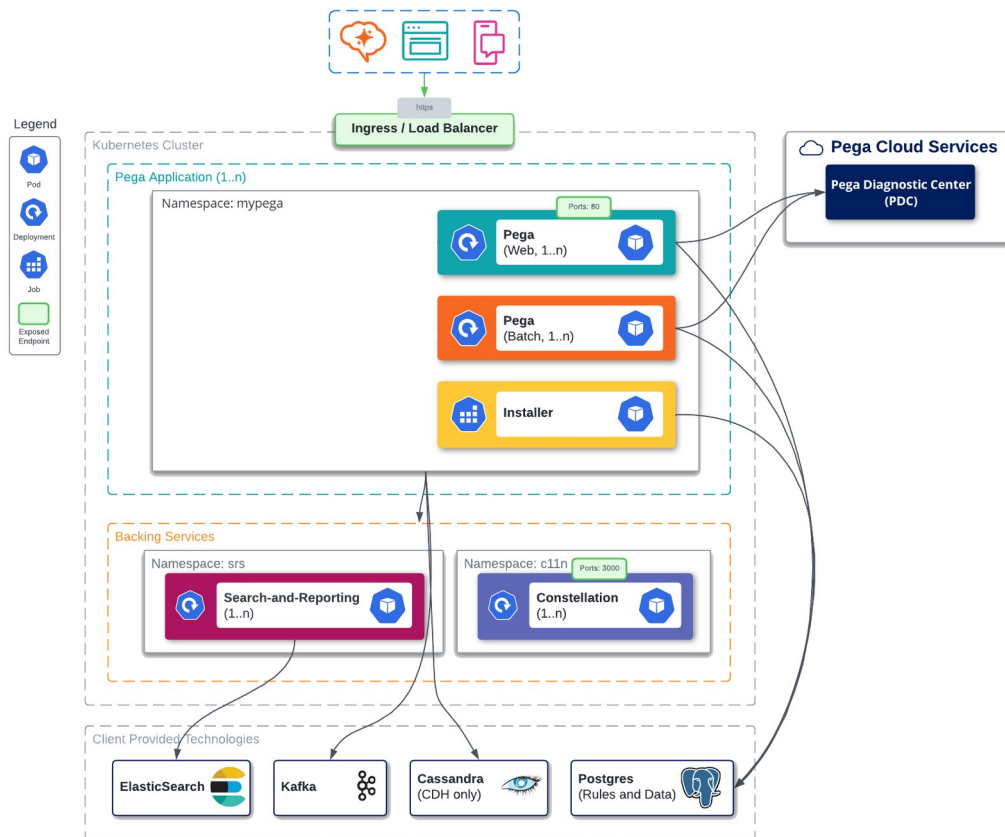


Figure 2. Pega Platform Kubernetes architecture for '26

Source: Pega Documentation, "Understanding the Pega Platform deployment architecture" [R21]. © Pegasystems Inc. Reproduced with attribution. The database shown is an example.

The 8.8 source system runs Kafka and Elasticsearch inside the Pega cluster. Pega Academy shows that older layout and the externalized layout side by side [R29, R30]. The comparison explains why nothing in the 8.8 Kafka or search storage can be moved: it lives on the 8.8 nodes.

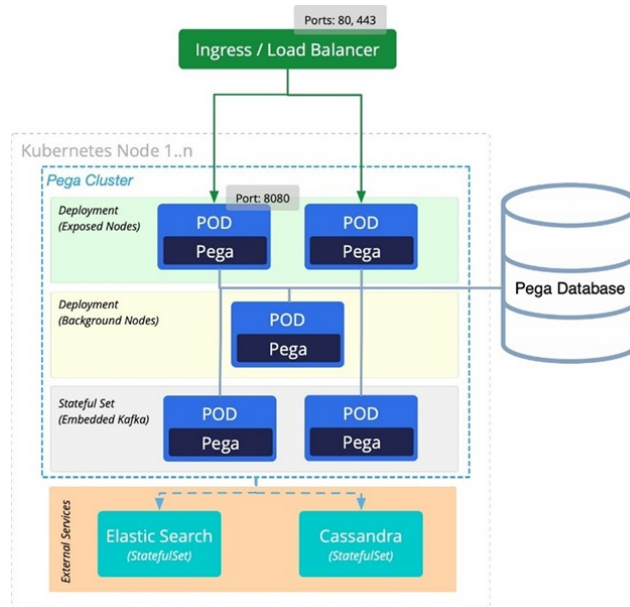


Figure 3. Earlier Pega cluster layout with embedded Kafka and in-cluster Elasticsearch

Source: Pega Academy, "Cloud deployment architecture" [R30]. © Pegasystems Inc. Reproduced with attribution. Shown to explain the 8.8 source; this layout is not supported from '24.2.

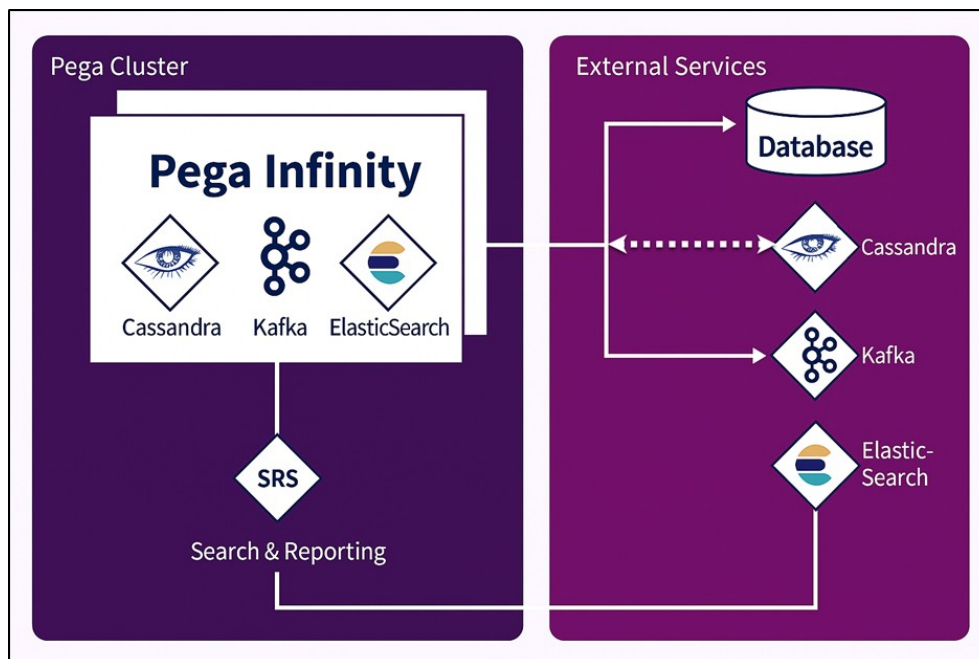


Figure 4. Externalized services in a Pega deployment

Source: Pega Academy, "Deployment architecture with external services" [R29]. © Pegasystems Inc. Reproduced with attribution.

4.2 Target architecture on Azure

Figure 5 shows one environment. The Pega web and batch tiers, the installer job and SRS run in a private AKS cluster. Confluent Cloud, the OpenSearch service, the upgraded database, Key Vault and the container registry are reached through private endpoints. Users reach Pega through Application Gateway with Web Application Firewall. Okta is a public SaaS service, reached through the hub firewall on an allow-list.

Table 10. Target architecture components

Component	Role	Design choice	Source
Web tier	User and API traffic (nodeType WebUser)	HPA on CPU; pod disruption budget	[R23]
Batch tier	Queue processors, job schedulers, search indexing, data flows	No ingress; held at zero replicas for the first start of a clone (Section 10.4)	[R21, R23]
Installer job	Upgrades the cloned database	Runs with <code>global.actions.execute: upgrade</code> (Section 9.4)	[R23]
SRS	Search and Reporting Service between Pega and OpenSearch	One deployment per environment; network policy; OAuth enabled	[R14, R24]
External Secrets Operator	Copies Key Vault secrets into Kubernetes secrets	Workload identity to Key Vault	[R23, R48]
Confluent Cloud	Kafka for the Pega stream service	Private Link; replication factor 3; prefixed ACLs	[R11, R31, R32]
Managed OpenSearch	Index storage for SRS	Version on the SRS matrix; private endpoint	[R14]
Okta	Issues tokens Pega presents to SRS	Custom authorization server; one client per environment	[R42, R43]
Pega database	Upgraded clone of the 8.8 database	Same region as AKS; private endpoint; TLS	[R20]

Figure 6 shows the five flows that matter for Kafka and search. Flow 1 is the user request. Flow 2 is queue processing: producers on any tier write to Kafka, and the batch tier consumes. Flow 3 is search: the batch tier

indexes changed records through SRS, and the web tier sends queries through SRS. Flow 4 is the token path to Okta. Flow 5 is telemetry.

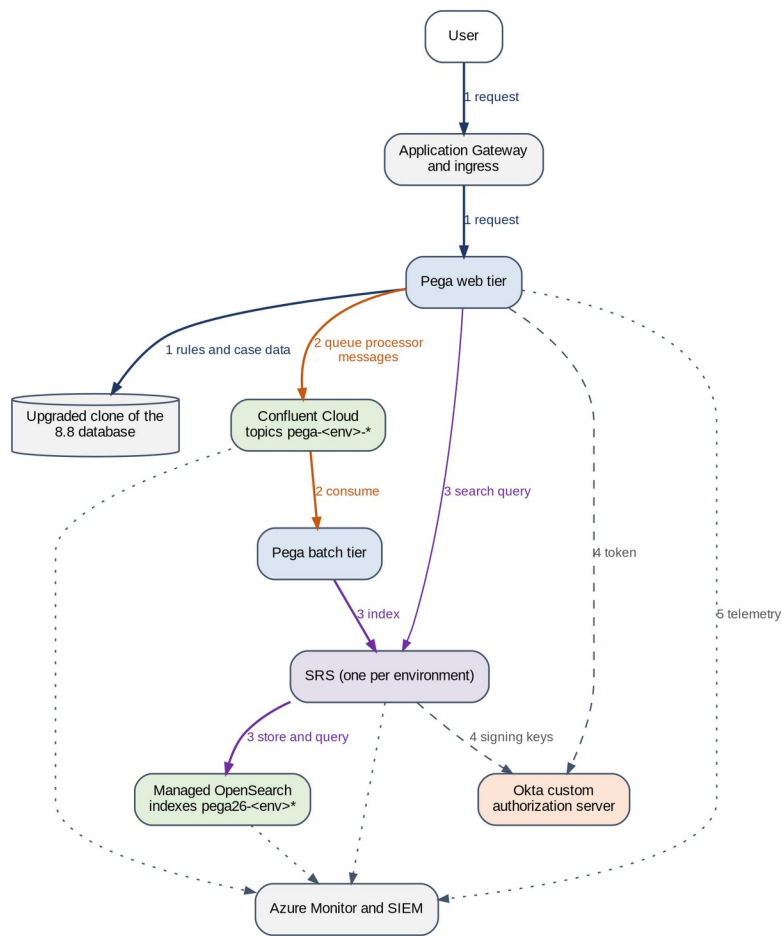


Figure 6. Logical flows between Pega, Kafka, SRS, OpenSearch and Okta

Prepared for this implementation. Not a Pega-published diagram.

4.4 Network and DNS

Every private endpoint needs a DNS record that resolves to its private IP from inside AKS [R47]. Confluent Private Link needs the wildcard and zonal records that Confluent lists for the cluster's network [R32]. A missing zonal record is the most common reason a Kafka client reaches the bootstrap server and then fails on broker connections. Figure 7 shows the name resolution and traffic paths, and Table 11 lists the flows the firewall and network policies must allow.

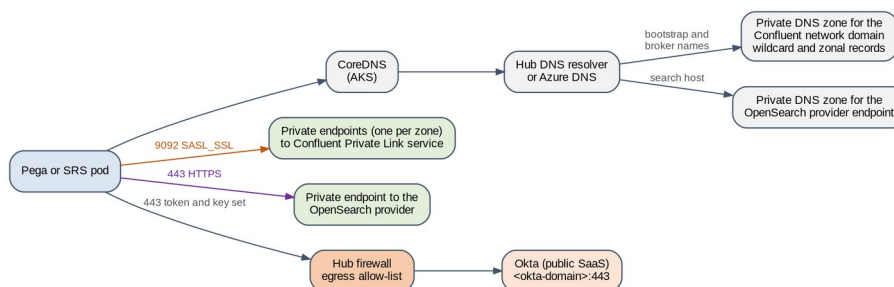


Figure 7. Name resolution and traffic paths for Kafka, OpenSearch and Okta

Prepared for this implementation. Not a Pega-published diagram.

Table 11. Network flows to allow and to deny

From	To	Port	Protocol	Purpose
Pega pods (web, batch)	Confluent private endpoints	9092	SASL_SSL	Stream service [R32]
Pega pods	SRS service in srs-<env>	8443	HTTPS with bearer token	Indexing and search (Section 7.7)
SRS pods	OpenSearch private endpoint	443 or the provider port	HTTPS	Index storage and queries
Pega pods and installer job	Database private endpoint	Engine port	TLS	Rules and data
Pega pods	Okta <okta-domain>	443	HTTPS through the firewall	Token request
SRS pods	Okta <okta-domain>	443	HTTPS through the firewall	Key set download
External Secrets Operator	Key Vault private endpoint	443	HTTPS	Secret synchronization
Any pod in a non-production namespace	Production Kafka, production OpenSearch, production integrations	Any	Any	Denied by firewall and network policy (Section 5.3)

4.5 Message path inside Pega

Each queue processor writes to a Kafka topic named from the stream name pattern, for example `pega-sit-<stream name>`. The batch tier reads each topic through a consumer group under the same prefix, and one partition is read by one consumer at a time. Items that are not yet due wait in the database as delayed items. Items that fail after their retries become broken items, also held in the database. This split matters for the cloned database: the topics are empty in 26.1.1, but the database-held items are not (Section 10.3). Figure 8 shows the path.

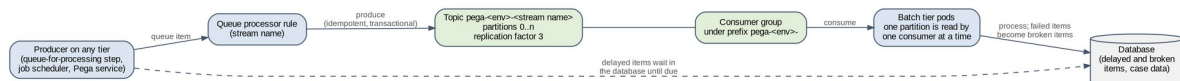


Figure 8. Message path from producer to batch tier
Prepared for this implementation. Not a Pega-published diagram.

Note. Pega '26 moves queue processing to a partitioned, multi-threaded model [R3]. The number of partitions per topic therefore limits how many batch threads can work on one queue processor at once. Measure partitions after the first start (Section 6.7) rather than estimating them.

4.6 Availability zone resilience

PERF, PREPROD and PROD are built so that the loss of one Azure availability zone does not stop Pega, Kafka or search. Figure 9 shows the layout. DEV, SIT and UAT may run in fewer zones to save cost, but their shared Kafka cluster and OpenSearch service follow the same rules because they carry three environments.

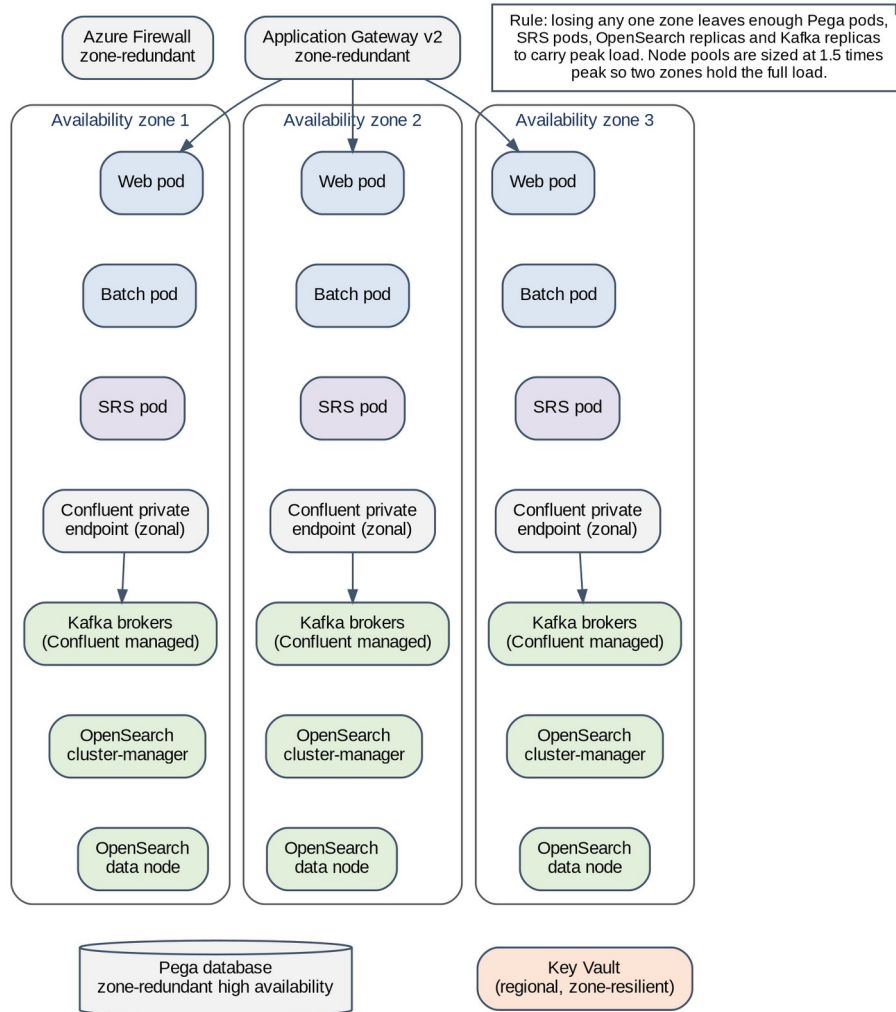


Figure 9. Spreading each layer across three availability zones

Prepared for this implementation. Not a Pega-published diagram.

Table 12. Availability zone design by layer

Layer	Zone design	Setting	Source
AKS node pools	Each user node pool spans zones 1, 2 and 3. The zones of a node pool cannot be changed after it is created, so this is decided before the first build	--zones 1 2 3 at node pool creation	[R59]
Pega web and batch pods	Spread evenly across zones; the scheduler still places pods if a zone is down	tier[*].topologySpreadConstraints on topology.kubernetes.io/zone, whenUnsatisfiable: ScheduleAnyway (Appendix B)	[R59, R61]
SRS pods	Preferred anti-affinity across zones	srsRuntime.affinity (Section 7.7)	[R24]
Node pool capacity	Two zones must hold the full peak load, so each pool is sized at 1.5 times the peak node count	Sizing calculator (Appendix H)	This design
Pod disruption budgets	Keep at least one pod per tier during drains and upgrades	tier[*].pdb.minAvailable: 1	[R23]
Confluent Cloud	Enterprise: the 99.99 % SLA needs at least 2 eCKU. Dedicated: the cluster must be created multi-zone, which needs at least 2 CKU and cannot be changed later	Cluster creation (CC-1); one private endpoint per zone	[R31, R32]

Layer	Zone design	Setting	Source
OpenSearch	Three cluster-manager nodes and data nodes in three zones, one replica, zone awareness on	Provider setting (Section 7.4)	[R37]
Database, Key Vault, gateway, firewall	Zone-redundant service tiers	Customer landing zone standards	Customer

Caution. Topology spread with [ScheduleAnyway](#) is a preference. After a zone returns, Kubernetes does not move running pods back, so the spread stays uneven until the next rollout. Check the spread after FS-21 and after every zone incident, and restart the tier if one zone holds most of the pods.

5 Shared and dedicated service model

Pega allows one Kafka cluster and one search service to serve more than one Pega deployment. Kafka isolation comes from the stream name prefix and prefixed ACLs [R11]. Search isolation comes from the customerDeploymentId, which SRS uses as the index prefix [R14, R23]. This section uses those features to cut the number of service sets from six to three, and states the controls that keep each environment's data apart.

5.1 Grouping options

Four groupings were assessed. Table 13 compares them. Option B is recommended.

Table 13. Non-production grouping options

Option	Service sets	Strengths	Weaknesses
A. Dedicated per environment	6 Kafka clusters, 6 OpenSearch services	No shared blast radius; simplest isolation	Highest cost; six sets to patch, monitor and rotate
B. Two shared groups plus PROD (recommended)	NP1: DEV, SIT, UAT. NP2: PERF, PREPROD. PROD dedicated. 3 sets in total	Load tests and rehearsals run on production-like services without touching functional testing; PREPROD matches PROD	PERF and PREPROD runs must be scheduled so they do not overlap
C. One shared non-production set plus PROD	2 sets	Lowest cost	A PERF load test or PREPROD rehearsal affects DEV, SIT and UAT; non-production services cannot match PROD type without paying PROD price for all
D. PREPROD alone, others shared	DEV, SIT, UAT, PERF shared; PREPROD; PROD. 3 sets	PREPROD fully separate	PERF load tests affect functional testing; same number of sets as B with less benefit

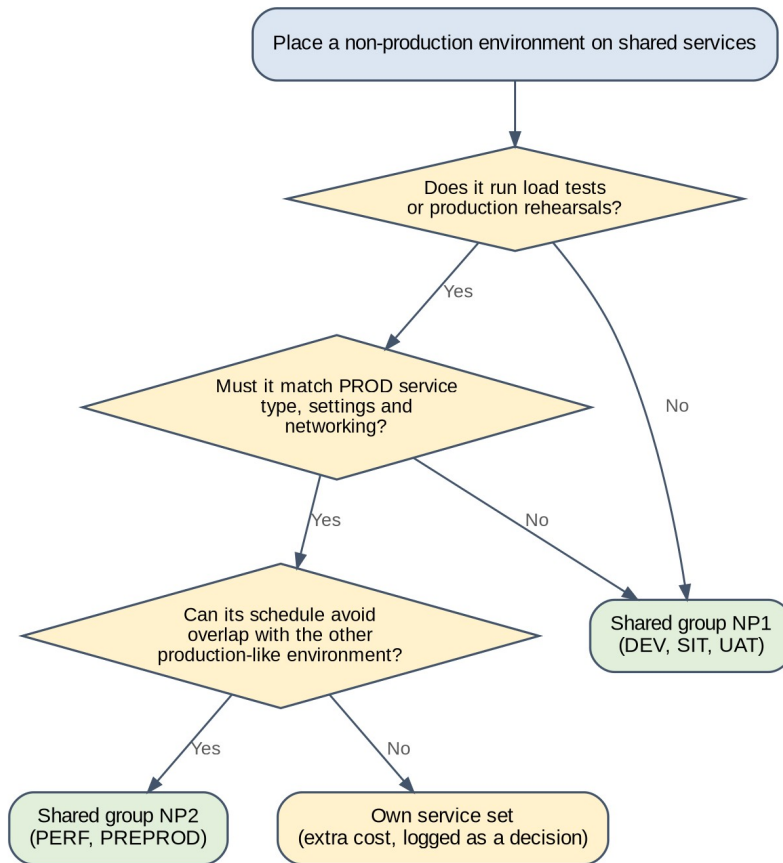


Figure 10. Decision: placing a non-production environment on shared services

Prepared for this implementation. Not a Pega-published diagram.

5.2 Recommended model

Figure 11 shows the two non-production groups and Figure 12 shows production. Each environment has its own namespace, topic prefix, service account, customerDeploymentId, SRS deployment, OpenSearch user, Okta client and Key Vault. Only the Kafka cluster and the OpenSearch service are shared within a group.

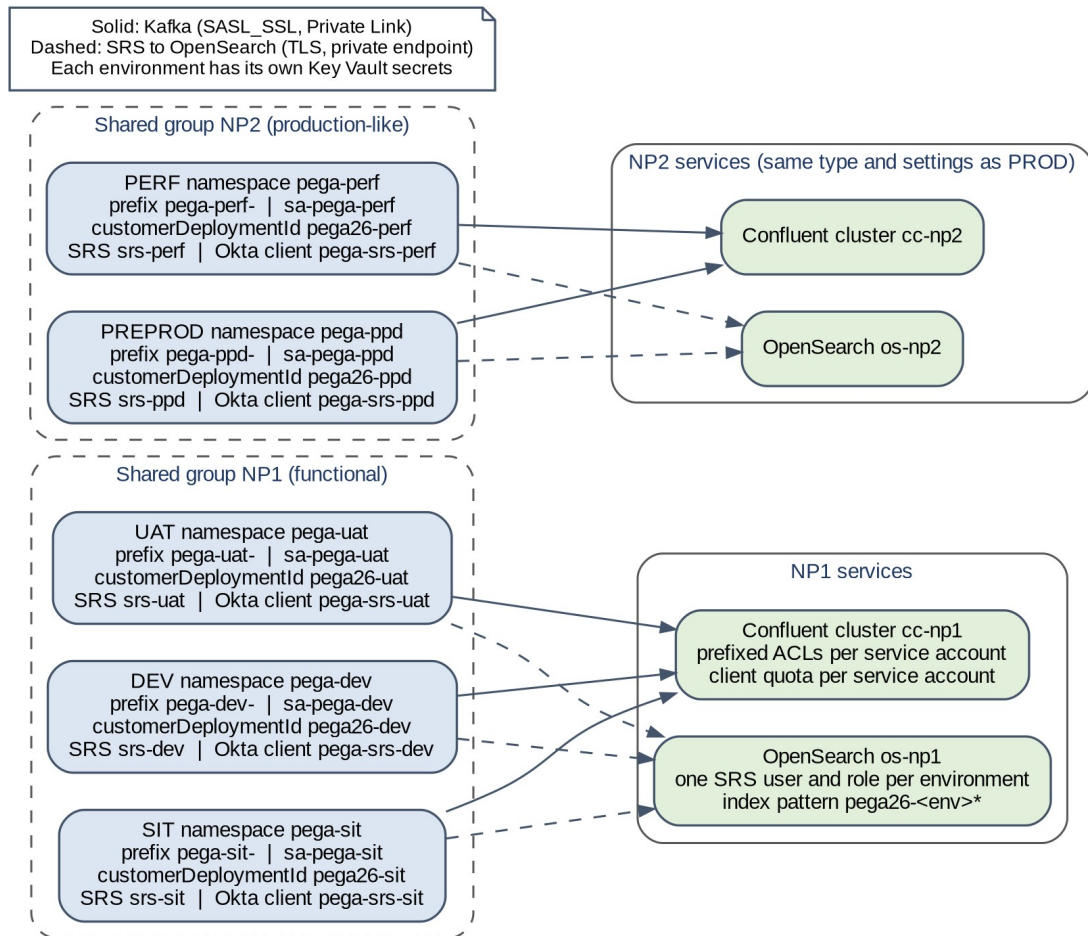


Figure 11. Shared non-production topology and per-environment boundaries

Prepared for this implementation. Not a Pega-published diagram.

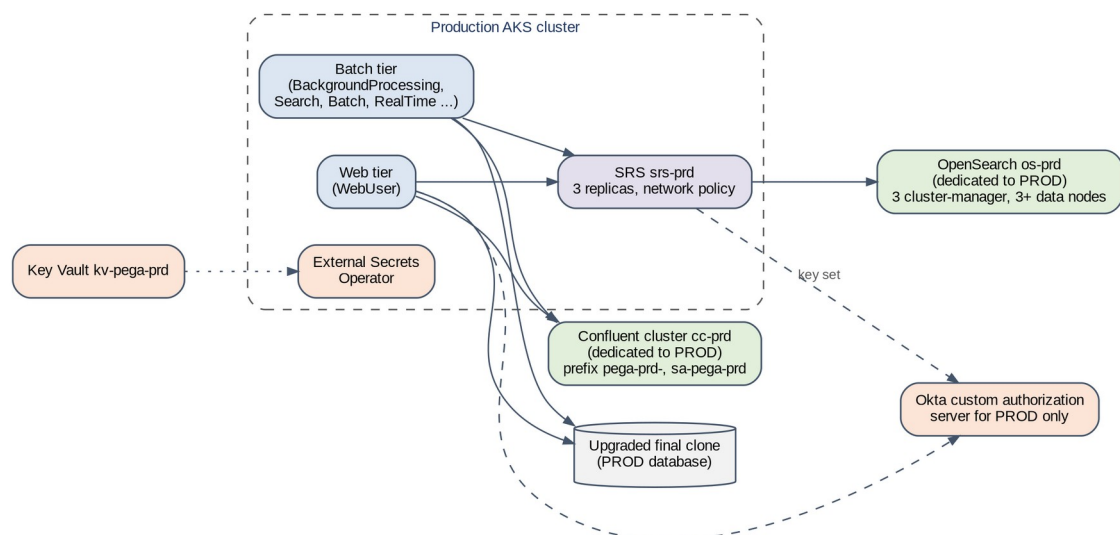


Figure 12. Production topology with dedicated services

Prepared for this implementation. Not a Pega-published diagram.

5.3 Isolation controls

Isolation is enforced at every layer, so a single mistake does not expose another environment. Table 14 lists the control at each layer and the test that proves it.

Table 14. Isolation controls by layer

Layer	Control	What it prevents	Proof (Section 14.3)
Kafka topics	Prefix <code>pega-</code> per environment through <code>stream.streamNamePattern</code> ; TOPIC ACL ALL PREFIXED on that prefix only [R11, R23]	Reading, writing or deleting another environment's topics	IT-01, IT-02
Kafka consumer groups	GROUP ACL ALL PREFIXED on the same prefix [R11]	Joining or resetting another environment's consumer groups	IT-03
Kafka credentials	One service account per environment; API keys in that environment's Key Vault only [R36]	Credential reuse across environments	IT-04
Kafka capacity	Client quota per service account on Enterprise or Dedicated clusters [R36]	One environment's load starving the others	PT-03
Search indexes	customerDeploymentId <code>pega26-</code> , used as the index prefix [R23]	Index name collisions	IT-05
Search credentials	One SRS deployment and one OpenSearch user per environment, with a role limited to index pattern <code>pega26-*</code> if SRS accepts it (Section 7.5) [R40, R41]	One environment's SRS reading or deleting another's indexes	IT-06
Search tokens	Okta client per environment; SRS checks that <code>guid</code> equals its own customerDeploymentId [R23]	A token for one environment being accepted by another environment's SRS	IT-07
Secrets	Key Vault per environment; External Secrets Operator identity per namespace [R48]	Pods reading another environment's secrets	IT-08
Network	Firewall allow-list per environment; Kubernetes network policies deny by default	A cloned environment reaching production Kafka, production OpenSearch or production integrations	IT-09, IT-10
Cloned data	Application Kafka data sets and other endpoints reported or disabled before the batch tier starts (Section 10.4)	Test environments consuming production topics or calling production systems	IT-10, FS-27

Figure 13 shows the same controls as layers. A request from one environment has to pass every layer to reach another environment's data, so a single misconfiguration does not expose it.

Layer	Kafka (Confluent Cloud)	Search (SRS and OpenSearch)
Network	Firewall allow-list per environment; network policies deny by default	Same
Identity	Service account <code>sa-pega-env</code> ; own API key	OpenSearch user <code>srs-env</code> ; Okta client <code>pega-srs-env</code>
Authorization	TOPIC and GROUP ACLs on prefix <code>pega-env-</code>	Role on <code>pega26-env*</code> ; <code>guid</code> claim checked by SRS
Naming	<code>streamNamePattern</code> <code>pega-env-{stream.name}</code>	<code>customerDeploymentId</code> <code>pega26-env</code>
Capacity	Client quota per service account	Build schedule; data nodes sized for one build
Secrets	Key Vault <code>kv-pega-env</code> ; workload identity per namespace	Same
Proof	IT-01 to IT-04, IT-08 to IT-10	IT-05 to IT-07

Figure 13. Isolation layers between environments on shared services

Prepared for this implementation. Not a Pega-published diagram.

Caution. Wildcard ACLs are forbidden on a shared cluster. Pega's ACL list includes `TRANSACTIONAL_ID *` [R11], which every environment needs. That ACL does not give access to topics or groups, but it means transactional IDs are not separated by environment. Record this as an accepted residual risk (RK-07).

Never grant TOPIC or GROUP with a literal `*` or with the bare prefix `pega-`, which would match every environment.

5.4 Noisy neighbours and quotas

Confluent supports client quotas on Enterprise, Freight and Dedicated clusters. Quotas apply to a service account or an identity pool, not to an API key, and the Metrics API labels throughput by principal [R36]. One service account per environment therefore gives both a quota boundary and per-environment metrics.

- Set a produce and a consume quota per service account in NP1 and NP2, sized from the measured peak of that environment plus headroom (OD-09).
- Leave PROD without quotas unless another workload is added to its cluster.
- In NP2, PERF load tests and PREPROD rehearsals are booked in a shared calendar. A run that overlaps the other environment's run needs both test leads' approval.
- If a load test in one environment affects another, stop the test, check the Metrics API by principal, lower that service account's quota and rerun (FS-22).

OpenSearch has no equivalent quota per user. In the shared search services, limit the effect of one environment on the others by running full index builds outside the other environments' test hours, and by sizing the data nodes for the largest concurrent build (Section 7.4).

5.5 Cost comparison model

Prices change and depend on the customer's agreements, so this document gives no figures. Table 15 lists the inputs the customer supplies, and Table 16 shows how each option is costed from them.

Table 15. Cost model inputs supplied by the customer

Input	Where it comes from	Notes
Confluent cluster base price per type	Confluent price list or contract	Enterprise and Dedicated differ in pricing unit (eCKU or CKU) [R31]
Capacity units per cluster	Measured partitions and throughput (Section 6.7)	Shared clusters add the needs of each environment
Private Link and data transfer	Confluent and Azure price lists	One set of private endpoints per cluster
OpenSearch nodes per service	Sizing in Section 7.4	Cluster-manager and data nodes
OpenSearch storage and snapshots	Provider price list	Measured searchable data volume
Operations effort per service set	Customer operating model	Patching, monitoring, on-call, key rotation
Support plans	Confluent and provider contracts	Production support level for PROD and NP2

Table 16. Cost model by option (multiply each line by the customer's unit prices)

Cost line	Option A	Option B (recommended)	Option C
Kafka clusters	6 x (base + units)	NP1 + NP2 + PROD: 3 x (base + units)	2 x (base + units)
Private Link sets	6	3	2
OpenSearch services	6 x (nodes + storage)	3 x (nodes + storage)	2 x (nodes + storage)
SRS deployments	6 (inside AKS)	6 (inside AKS)	6 (inside AKS)

Cost line	Option A	Option B (recommended)	Option C
Okta authorization servers	Up to 6	2 (PROD and shared non-production)	2
Operations effort	6 service sets	3 service sets	2 service sets
Risk cost	Lowest	Shared blast radius within NP1 and NP2; scheduling in NP2	Load tests affect all non-production; no production-like test services

Option B removes three Kafka clusters and three OpenSearch services compared with Option A. The saving is the base price of those services plus their private endpoints and operations effort, less any extra capacity units the shared clusters need. Option C saves one more set but removes the production-like test services, which raises the chance of a production problem that rehearsals did not show.

5.6 Production

Production always has its own Confluent cluster, its own OpenSearch service, its own Okta authorization server and its own Key Vault. Production credentials, endpoints and DNS zones are not linked to any non-production network.

6 Confluent Cloud design and configuration

6.1 What Pega uses Kafka for

Pega uses Kafka in two ways. The stream service carries every queue processor and job scheduler, so Pega is not fully functional without it [R11]. This is what the Helm `stream` section configures. Separately, a Kafka data set is an application feature that reads from or writes to the customer's own Kafka topics. Pega requires that Kafka data sets use a different Kafka service from the stream service [R11]. Section 11.3 covers data sets.

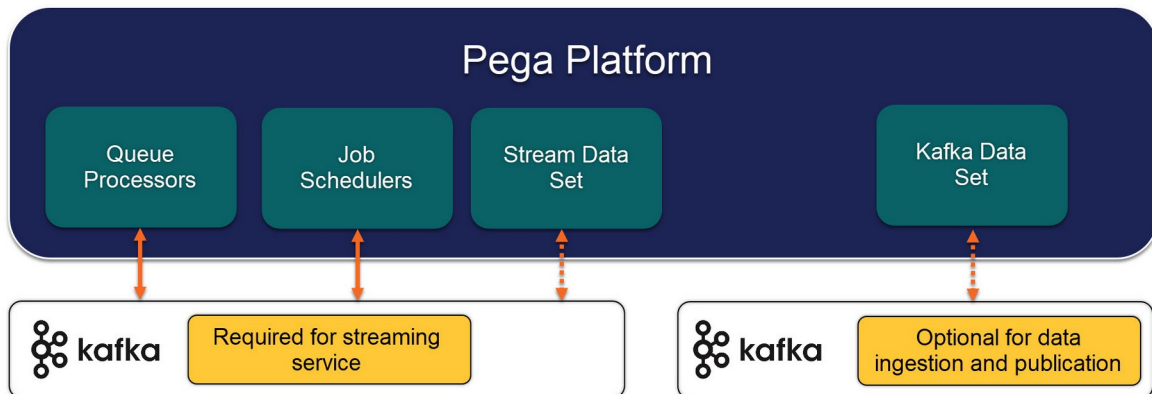


Figure 14. The two Kafka use cases in Pega Platform

Source: Pega Documentation, "External Kafka in your deployment" [R11]. © Pegasystems Inc. Reproduced with attribution.

Important. From '25, Pega uses its streaming function and the external Kafka service for the cluster messaging that Hazelcast used to carry, and Pega asks clients to review the Kafka configuration before the change [R7]. A Kafka outage on 26.1.1 therefore affects more than queue processing. Treat the Kafka service as a tier-1 dependency and test its loss explicitly (FS-01, FS-03).

6.2 Pega requirements mapped to Confluent Cloud

Confluent Cloud is a managed service, so some broker settings Pega lists cannot be changed by the customer. Table 17 shows each requirement, what Confluent Cloud allows, and what the runbook does.

Table 17. Pega Kafka requirements mapped to Confluent Cloud

Pega requirement	Confluent Cloud position	Action
Kafka client 4.0.0 in '26, compatible with 3.9.2 [R3]	Fully managed Kafka	Produce, consume and transactional checks from a Pega pod in every environment (Section 14.4)
Confluent 5.4.x or later [R11]	Current release line	None beyond the checks above
message.max.bytes 5000000 [R11]	Topic max.message.bytes default 2,097,164; editable up to 20,971,520 on Enterprise and Dedicated [R33]	After each start, set max.message.bytes to 5,000,000 on every Pega topic (command K-6)
replica.fetch.max.bytes and replica.fetch.response.max.bytes 5000100 [R11]	Not in the list of editable cluster settings [R34]	Ask Confluent support to confirm in writing (RC-03)
unclean.leader.election.enable false [R11]	Not customer-editable [R34]	Ask Confluent support to confirm in writing (RC-03)
auto.create.topics.enable false [R11]	Editable on Dedicated; default false [R34]	Keep false. Pega creates topics through its admin client, which needs CREATE in the topic ACL
Generic Kafka security only [R11]	SASL_SSL with API keys (PLAIN) is generic Kafka	Use SASL_SSL with PLAIN (Section 6.5)

Pega requirement	Confluent Cloud position	Action
ACLs for TOPIC, GROUP, TRANSACTIONAL_ID, CLUSTER IDEMPOTENT_WRITE [R11]	Supported on service accounts	Create before the first start (Section 6.5)
Production sizing supports about 1000 partitions [R11]	Partition limits per capacity unit are published [R31]	Measure after the first start (Section 6.7)

6.3 Cluster type per group

Confluent offers Basic, Standard, Enterprise, Dedicated and Freight clusters [R31]. Basic and Standard do not offer private networking, so they cannot hold cloned or masked production data. Freight does not support idempotent producers or transactions [R31], and Pega's ACL profile needs both [R11]. That leaves Enterprise and Dedicated.

Table 18. Confluent Cloud cluster limits relevant to Pega (source [R31]; Basic and Standard rows for connections and requests are not used here)

Dimension (per unit)	Basic eCKU	Standard eCKU	Enterprise eCKU	Dedicated CKU
Ingress MBps	5	25	60	60
Egress MBps	15	75	180	180
Partitions (before replication)	30	250	3,000	4,500
Client connections			18,000	18,000
Connection attempts per second			500	500
Requests per second			7,500	15,000
Partition creations and deletions per 5 minutes (cluster)			500	5,000
Maximum message size			20 MB	20 MB
Private networking	No	No	Yes (Private Link)	Yes (Private Link or VNet peering)
Client quotas [R36]	No	No	Yes	Yes

Important. Confluent now enforces the connection and request limits on Enterprise clusters rather than treating them as guidance: request limits from March 2026 and connection limits from June 2026 [R31]. A client over a limit is throttled, and the Metrics API reports the throttling per principal with the limit that was hit [R56]. Pega pods open many Kafka connections: one set per producer, per queue-processor consumer thread and for cluster messaging. A rolling restart of a large tier, or every environment in a group starting at once after a refresh, can reach the connection-attempt limit. Measure connections per pod in DEV (Section 15.8), stagger restarts in shared groups, and alert on throttling (Section 18.5).

Caution. Partition creation and deletion are paced at 500 per five minutes per cluster on Enterprise [R31]. The first start of an environment creates every Pega topic, and a refresh deletes them all. With several hundred partitions per environment, both can take tens of minutes and slow every other environment on the cluster that creates topics at the same time. Include this time in the rehearsal measurements and never refresh two environments in a group at once.

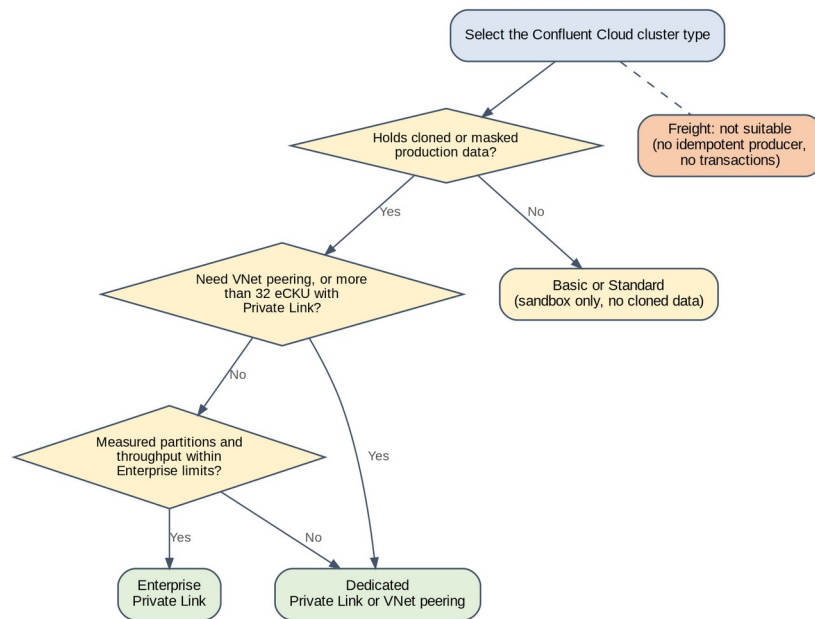


Figure 15. Decision: Confluent Cloud cluster type

Prepared for this implementation. Not a Pega-published diagram.

Table 19. Cluster type by group

Group	Recommended type	Reason
NP1 (DEV, SIT, UAT)	Enterprise with Private Link	Holds masked production data; quotas available; elastic capacity suits uneven test load
NP2 (PERF, PREPROD)	Same type as PROD	Load tests and rehearsals must show production behaviour
PROD	Enterprise with Private Link, or Dedicated if measured partitions or throughput exceed Enterprise limits, or VNet peering is required	Decided after the first PERF measurement (OD-02)

Caution. Enterprise clusters that use Private Link on Azure are limited to 32 eCKU [R31]. Add up the measured partitions of every environment in the group before confirming Enterprise.

The 99.99 % uptime SLA applies to Enterprise clusters of at least 2 eCKU, and to Dedicated clusters only when they are created multi-zone, which needs at least 2 CKU and cannot be changed later [R31]. Set the minimum capacity of cc-np2 and cc-prd to 2 units. Enterprise scales quickly up to 10 eCKU and more slowly above that, so do not rely on elastic scaling to absorb a cutover peak above 10 eCKU; raise the minimum before the event.

6.4 Private networking

Azure Private Link gives one-way private access from the customer VNet to Confluent Cloud. Confluent's Azure steps are: create the Confluent network, add a Private Link access for the customer subscription, create one private endpoint per zone, create the DNS records, and test [R32]. Each group's cluster needs its own set. The NP1 and NP2 private DNS zones are linked only to non-production networks, and the PROD zone only to the production network.

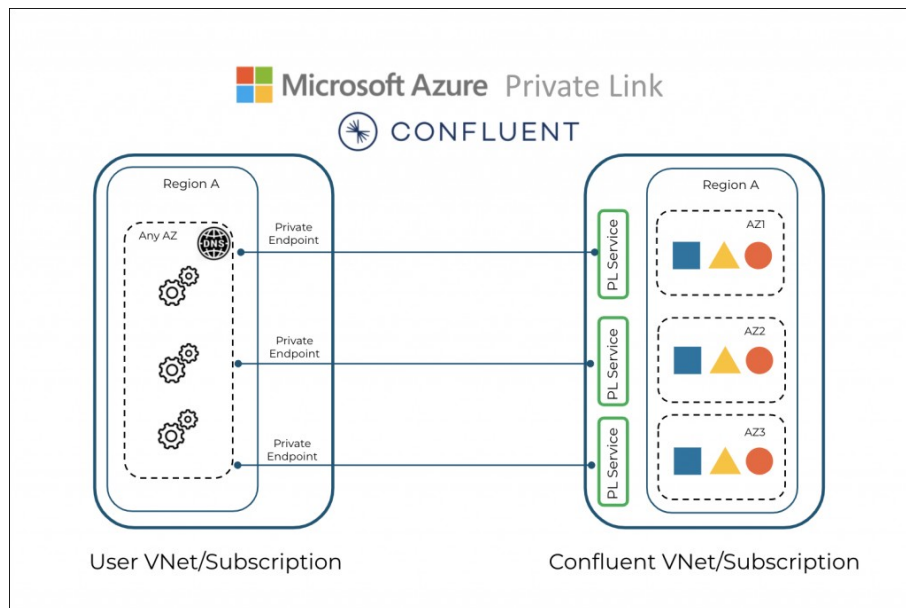


Figure 16. Azure Private Link between a customer VNet and Confluent Cloud

Source: Confluent Documentation, "Use Azure Private Link for Dedicated clusters on Confluent Cloud" [R32]. © Confluent, Inc. Reproduced with attribution.

+ Record set → Move ▾ Delete zone Refresh			
^ Essentials			
Resource group (move) : akumar-privatelink-test			
Subscription (move) : Cloud Private Development			
Subscription ID : d2b206bc-847a-437e-8d8d-aedd84a69ed0			
Tags (edit) : Add tags			
📘 You can search for record sets that have been loaded on this page. If you don't see what you're looking for, you can try scrolling to allow more record sets to load.			
🔍 Search record sets			
Name	Type	TTL	Value
*	A	60	172.16.128.4 172.16.128.5 172.16.128.6
@	SOA	3600	Email: azureprivatedns-host.microsoft.com Host: azureprivatedns.net Refresh: 3600 Retry: 300 Expire: 2419200 Minimum TTL: 10 Serial number: 1
*.az1	A	3600	172.16.128.4
*.az2	A	3600	172.16.128.5
*.az3	A	60	172.16.128.6

Figure 17. DNS records that Confluent lists for Azure Private Link

Source: Confluent Documentation [R32]. © Confluent, Inc. Reproduced with attribution.

Table 20. Private Link set-up steps per cluster

Step	Action	Check
N-1	Create the Confluent network in the AKS region and register the customer subscription for Private Link.	Network ID and subscription recorded in Appendix A
N-2	Create one private endpoint per zone in the private endpoint subnet, using the service aliases Confluent shows.	Connection state Approved for each
N-3	Create the private DNS zone for the Confluent network domain with the wildcard and zonal records Confluent shows; link it to the right VNet or resolver.	Zone export kept as evidence
N-4	From a pod in <code>pega-<code><code></code></code> , resolve the bootstrap name and a broker name.	Both return private IPs (K-1)
N-5	From the same pod, open TLS to the bootstrap name on port 9092.	Certificate chain shown (K-2)

Step	Action	Check
N-6	Run a metadata request.	Every broker name resolves privately (K-3)

Note. Confluent states that the Confluent CLI might need internet access to the Confluent control plane even when the cluster uses Private Link [R32]. Allow that only from the administration jump host, never from Pega pods.

6.5 Service accounts, authentication and ACLs

Pega lists several SASL options [R11], but the Helm `stream.saslMechanism` key accepts PLAIN, SCRAM-SHA-256 and SCRAM-SHA-512 [R23]. Confluent Cloud API keys use SASL/PLAIN. Figure 18 shows the choice.

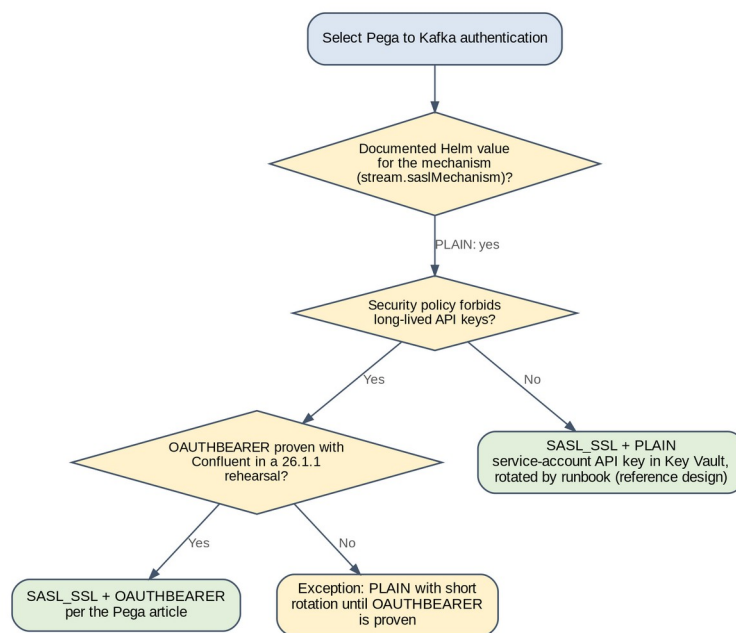


Figure 18. Decision: Pega to Kafka authentication

Prepared for this implementation. Not a Pega-published diagram.

Create one service account per environment and grant it only the ACLs Pega lists, scoped to that environment's prefix. Table 21 shows the ACLs for one environment; repeat them for each environment on its own cluster.

Table 21. Kafka ACLs for one environment's service account (source [R11])

Resource type	Resource name	Operation	Pattern	Why
TOPIC	pega - <code>-	ALL	PREFIXED	Create, write, read and delete Pega topics
GROUP	pega - <code>-	ALL	PREFIXED	Consumer groups for queue processors and data flows
TRANSACTIONAL_ID	*	READ, WRITE	LITERAL	Transactional producers
CLUSTER	the cluster	IDEMPOTENT_WRITE	LITERAL	Idempotent producers

Creating and listing one environment's service account and ACLs

```
# Confluent CLI. Check flags with: confluent kafka acl create --help
confluent iam service-account create sa-peg-a-<code> --description "Pega 26.1.1 <ENV>"
confluent kafka acl create --allow --service-account <sa-id> --operations all \
  --topic "peg-a-<code>-" --prefix --cluster <lkc-id>
confluent kafka acl create --allow --service-account <sa-id> --operations all \
  --consumer-group "peg-a-<code>-" --prefix --cluster <lkc-id>
confluent kafka acl create --allow --service-account <sa-id> --operations read,write \
  --transactional-id "*" --cluster <lkc-id>
confluent kafka acl create --allow --service-account <sa-id> \
  --operations idempotent-write --cluster-scope --cluster <lkc-id>
confluent api-key create --resource <lkc-id> --service-account <sa-id>
confluent kafka acl list --service-account <sa-id> --cluster <lkc-id>
```

To remove an environment's access, delete its ACLs with `confluent kafka acl delete` using the same flags, delete its API keys, then delete the service account (Section 19.4).

6.6 Topic design

Table 22. Topic design settings

Setting	Value	Reason
streamNamePattern	peg-a- <code><code></code> - {stream.name}	Separates environments on a shared cluster. Chart default is <code>peg-a-{stream.name}</code> [R23].
replicationFactor	3	Pega's recommended value; cannot exceed the number of brokers [R23].
Topic creation	By Pega through its admin client (CREATE in the topic ACL)	Pega calls broker auto-creation a legacy method [R11]. If policy forbids CREATE, pre-create topics from the rehearsal list (OD-11).
max.message.bytes	5,000,000 on every Pega topic	Matches Pega's maximum message size [R11]; Confluent default is lower [R33].
SystemPulse_SystemPulseTopicName	6 partitions	'26 prerequisite; automatic if dynamic topic creation is allowed [R3].
Cluster messaging topics after Hazelcast removal	Covered by the prefixed ACL if Pega applies the pattern to them	Pega names five topics in the Hazelcast removal prerequisites [R6] but does not say whether the pattern applies. List the topics after the first DEV start and add literal ACLs for any outside the prefix (FS-04).

When an environment is refreshed from a new clone, keep its prefix and delete its old topics and consumer groups before the first start (Section 19.3). Keeping the prefix keeps ACLs, quotas and dashboards unchanged. A new prefix is only needed if old topics cannot be deleted, for example during an investigation; in that case create a new prefix such as `peg-a-sit2-` with its own ACLs.

6.7 Partition budget

Since Pega 8.7, each new stream topic is created with 6 partitions by default. The default is set by the DSS `prconfig/dsm/services/stream/pyTopicPartitionsCount/default` in ruleset Pega-Engine, and applies only to topics created after the change [R49]. On a queue processor rule, threads beyond the topic's partition count do no work, because one partition is read by one consumer at a time [R50]. Two rules follow:

- **Expected partitions per environment** = (number of Pega topics x partitions per topic), counted at replication factor 1. For example, with the default of 6, 150 topics give 900 partitions, close to the 1,000 that Pega's production sizing names [R11].
- **Effective consumers per queue processor** = min(partitions, batch pods x threads per node). Adding batch pods beyond the partition count does not speed up a queue processor; raise its partition count first, and only for the queue processors that the load test shows as the bottleneck.

Caution. A database cloned from 8.8 may carry a changed `pyTopicPartitionsCount` DSS or per-topic overrides from tuning on 8.8 (CD-15). Record the value at time point A. If it differs from 6, decide with the performance lead whether to keep it, because it multiplies the partition count of every topic created at first start.

Partitions drive both Confluent cost and limits. Measure them instead of estimating:

1. After the first full start of each environment, count partitions under its prefix (command K-5).
2. Record the count in Table 23 and add the headroom agreed for new queue processors (30 % is a sound starting point).
3. Add up the environments in each group and compare with the cluster limit for the chosen type (Table 18).
4. Repeat after each application release that adds queue processors.

Table 23. Partition budget worksheet

Environment	Group	Measured partitions	With 30 % headroom	Group total	Cluster limit	Within limit
DEV	NP1	<measured>	<x 1.3>	<sum for group>	<from Table>	<yes or no>
SIT	NP1	<measured>	<x 1.3>	<sum for group>	<from Table>	<yes or no>
UAT	NP1	<measured>	<x 1.3>	<sum for group>	<from Table>	<yes or no>
PERF	NP2	<measured>	<x 1.3>	<sum for group>	<from Table>	<yes or no>
PREPROD	NP2	<measured>	<x 1.3>	<sum for group>	<from Table>	<yes or no>
PROD	PRD	<measured>	<x 1.3>	<sum for group>	<from Table>	<yes or no>

6.8 Helm stream configuration

The `stream` section connects every tier to Kafka [R12]. Passwords and the JAAS string are left empty in the values file and supplied by an external secret with the keys `STREAM_TRUSTSTORE_PASSWORD`, `STREAM_KEYSTORE_PASSWORD` and `STREAM_JAAS_CONFIG` [R23].

values-<code>.yaml: stream section

```
stream:
  enabled: true
  bootstrapServer: "<bootstrap-host>:9092"
  securityProtocol: SASL_SSL
  saslMechanism: PLAIN
  trustStore: ""
  keyStore: ""
  jaasConfig: ""
  streamNamePattern: "pega-<code>-{stream.name}"
  replicationFactor: "3"
  external_secret_name: "pega-stream-secret"
```

Value of `STREAM_JAAS_CONFIG` in Key Vault (stored as one line)

```
org.apache.kafka.common.security.plain.PlainLoginModule required
  username="<api-key>" password="<api-secret>";
```

6.9 prpcUtils on 26.1.1

From '25, `prpcUtils` needs its own connection to Kafka for clustering. Pega documents a `-DNodeSettings` argument in `custom.jvm.args` of `prpcUtils.properties` for '25 and '26 [R8]. Pipelines that run `prpcUtils` against an upgraded environment must pass that environment's bootstrap server, prefix and JAAS value, generated at run time from Key Vault and deleted after the run.

Important. The installer job that runs the upgrade (run U) is built from the same Pega command-line tooling, but chart 4.13.0 passes it no stream settings: a rendered run U contains no `STREAM_*` values. Whether the 26.1.1 upgrade of an 8.8 database needs Kafka is therefore an open question for Pega Support (GQ-08). Until it is answered, allow the installer pod to reach the environment's Confluent endpoint and watch the installer log for Kafka connection attempts in the first DEV upgrade.

7 Managed OpenSearch and SRS design and configuration

7.1 How search works in 26.1.1

SRS is a Pega backing service between Pega Platform and the search cluster. Pega sends indexing and query requests to SRS, and SRS manages the indexes in OpenSearch [\[R14\]](#). SRS isolates each Pega environment's data with a customerDeploymentId, which becomes the index prefix [\[R14, R23\]](#). The search service can serve several Pega deployments, but Pega advises against sharing it with non-Pega software [\[R14\]](#).

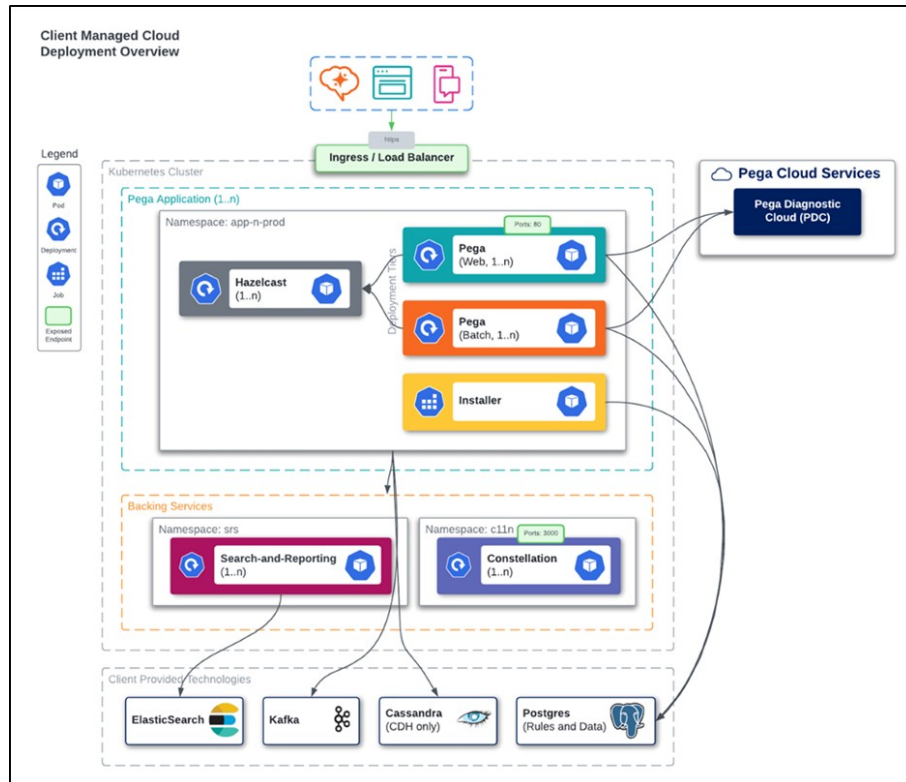


Figure 19. Client-managed cloud deployment with SRS as a backing service

Source: Pega Academy, "Search and Reporting Service" [\[R28\]](#). © Pegasystems Inc. Reproduced with attribution. The Hazelcast pod in this illustration does not apply to '25 and later.

7.2 SRS and OpenSearch versions

Table 24. SRS compatibility with OpenSearch for Pega 8.6 and later, SRS 1.44.3 or later (source [\[R14\]](#))

SRS image	Authentication	Certified OpenSearch versions	Pega best practice
search-n-reporting-service-os	Enabled	AWS OpenSearch service with Elasticsearch 7.10; OpenSearch 1.3, 2.15, 2.19	OpenSearch 2.15

- Choose OpenSearch 2.15 or 2.19. Version 1.3 is on the matrix but is an older line, and the provider may not offer it.
- Use the newest `search-n-reporting-service-os` tag that the SRS chart README lists for chart 4.13.0 [\[R24\]](#). The README lists a newer SRS than the Pega page (RC-01).
- Use only official OpenSearch images if the customer runs OpenSearch itself; custom images are not supported [\[R14\]](#).
- The search cluster that the backingservices chart can provision is for development and test only, and cannot be OpenSearch [\[R14, R24\]](#).

7.3 Provider selection

Azure has no first-party managed OpenSearch service, so the choice is between third-party providers that run OpenSearch in Azure regions and a self-managed cluster on AKS. Pega accepts a cloud subscription, a licensed product or open source [R14]. This document names no provider: a provider becomes a candidate only when its own documentation shows an Azure region and OpenSearch 2.15 or 2.19, and that check is part of the selection.

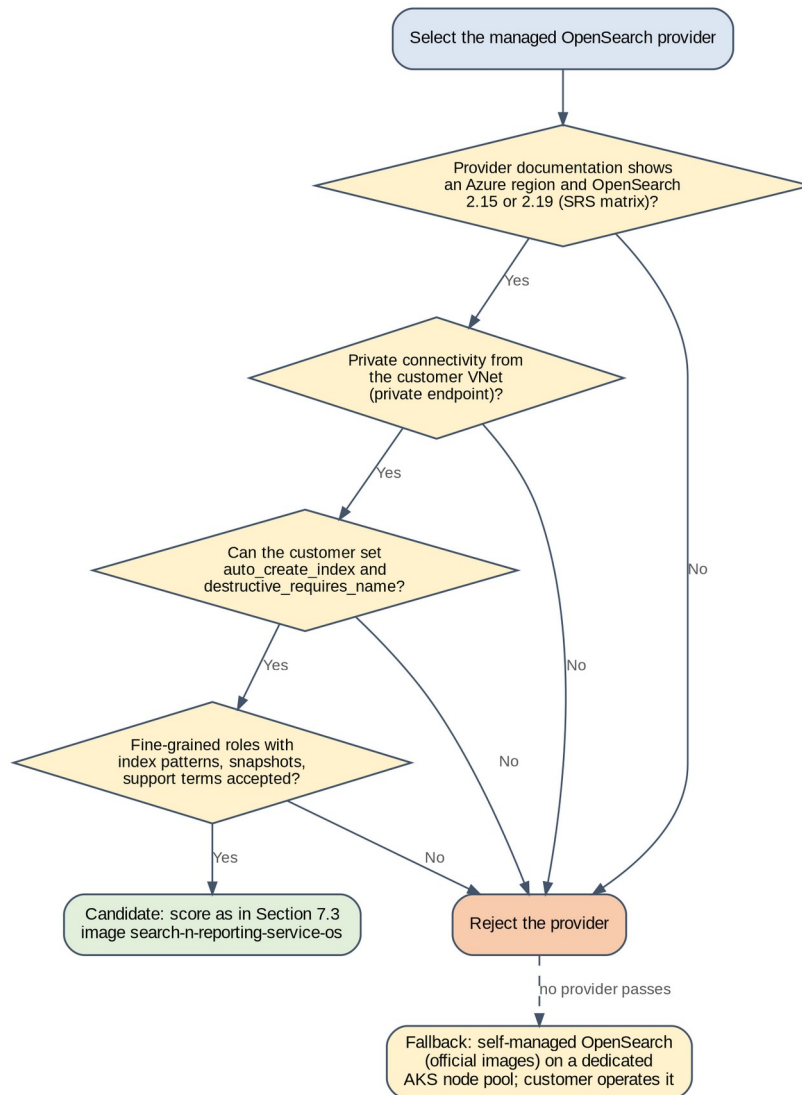


Figure 20. Decision: managed OpenSearch provider

Prepared for this implementation. Not a Pega-published diagram.

Score each candidate in Table 25. A provider that fails a mandatory criterion is rejected whatever its score.

Table 25. OpenSearch provider selection worksheet

Criterion	Type	Weight	Provider 1	Provider 2	Self-managed on AKS
Azure region same as AKS, shown in provider documentation	Mandatory				Yes
OpenSearch 2.15 or 2.19 available and pinnable	Mandatory				Yes
Private endpoint into the customer VNet	Mandatory				Yes

Criterion	Type	Weight	Provider 1	Provider 2	Self-managed on AKS
Customer can set <code>action.auto_create_index</code> and <code>action.destructive_requires_name</code> [R14]	Mandatory				Yes
Fine-grained access control with index patterns [R40, R41]	Mandatory				Yes
Snapshots with customer-chosen retention	Scored	15			
Support terms and response times for PROD	Scored	20			
Metrics export to Azure Monitor	Scored	15			
Version upgrade process and notice	Scored	15			
Operations effort for the customer	Scored	20			
Data residency and certifications	Scored	15			

Only four values depend on the provider: the endpoint host, the port, the SRS user credentials and the CA certificate. The Helm values in Appendix B keep those in angle brackets and in Key Vault, so the rest of the design does not change when OD-03 is decided.

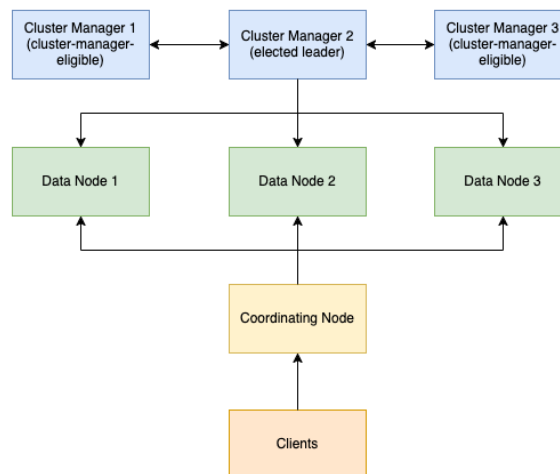


Figure 21. OpenSearch cluster roles: cluster-manager and data nodes

Source: OpenSearch Documentation, "Creating a cluster" [R37]. © OpenSearch contributors. Reproduced with attribution.

7.4 Cluster settings and sizing

SRS has its own index naming, so index auto-creation must be off. If the SRS user has the manage cluster privilege, SRS sets this itself; otherwise apply it manually [R14]. SRS also deletes indexes by pattern, which needs `destructive_requires_name` set to false [R14]. With an index-scoped SRS user (Section 7.5), apply both settings as an administrator before SRS starts.

Cluster settings required by SRS (run once per OpenSearch service)

```

PUT _cluster/settings
{
  "persistent": {
    "action": {
      "auto_create_index": "false",
      "destructive_requires_name": "false"
    }
  }
}

```

Caution. `destructive_requires_name: false` lets any user with delete rights remove indexes by wildcard. On a shared service, give delete rights only through the per-environment index-scoped roles, and keep administrator access to a break-glass group.

Pega publishes default sizing for search and SRS. Start from it and adjust after measuring, as Pega advises [R14].

Table 26. Pega default sizing for search and SRS (source [R14])

Service	Landscape	Instances	CPU each	RAM GB each	Storage GB each
Cluster-manager nodes	Production, stage	3	2	8	N/A
Data nodes	Production, stage	3	4	16	100
Data nodes	Testing, development	1	2	8	100
SRS	Production, stage	3 (autoscaled)	2	2	N/A
SRS (chart 4.13.0 defaults, for comparison)	All	2 minimum, 3 best practice [R24]	0.65 request, 1.3 limit	4	N/A
SRS	Testing, development	1 (autoscaled)	2	2	N/A

Table 27. Starting sizes per OpenSearch service

Service	Starting size	Basis	Adjust when
os-np1 (DEV, SIT, UAT)	3 cluster-manager, 3 data nodes at the Pega production size	Three environments' indexes plus one full build at a time; three nodes give zone spread	Disk above 60 % after all three builds
os-np2 (PERF, PREPROD)	Same as os-prd	Production-like behaviour for load tests and rehearsals	Never smaller than os-prd
os-prd	3 cluster-manager, 3 or more data nodes; storage from measured volume	Pega production sizing, scaled by the searchable data measured in the upgraded clone	Index build or query latency misses target in PERF

- **Searchable data volume.** Measure it from the first upgraded clone: run the full index build in DEV and record index sizes with `_cat/indices` (command S-4). Scale PROD storage from that, plus growth and one rebuild's working space.
- **Shards and replicas.** SRS creates and names the indexes itself [R14]. After the first build, read the shard and replica count of each index with `_cat/indices` and record it in Appendix A. Do not change index settings directly unless Pega Support advises it.
- **Shard budget.** Add up the shards of every environment on the service and compare with the provider's per-node shard limit.
- **Disk watermarks.** Record the provider's low, high and flood-stage watermark values in Appendix A, and alert before the low watermark is reached (Section 18.5). At the flood stage, indexes become read-only and indexing stops (FS-17).

7.4.1 Sizing formulas

Pega does not publish a storage or shard formula for SRS. The formulas below are the general OpenSearch sizing guidance published for Amazon OpenSearch Service [R54, R55], which applies to any OpenSearch cluster because it rests on OpenSearch's own overheads. Use them with the measured index sizes from the DEV build. The sizing calculator in Appendix H applies them.

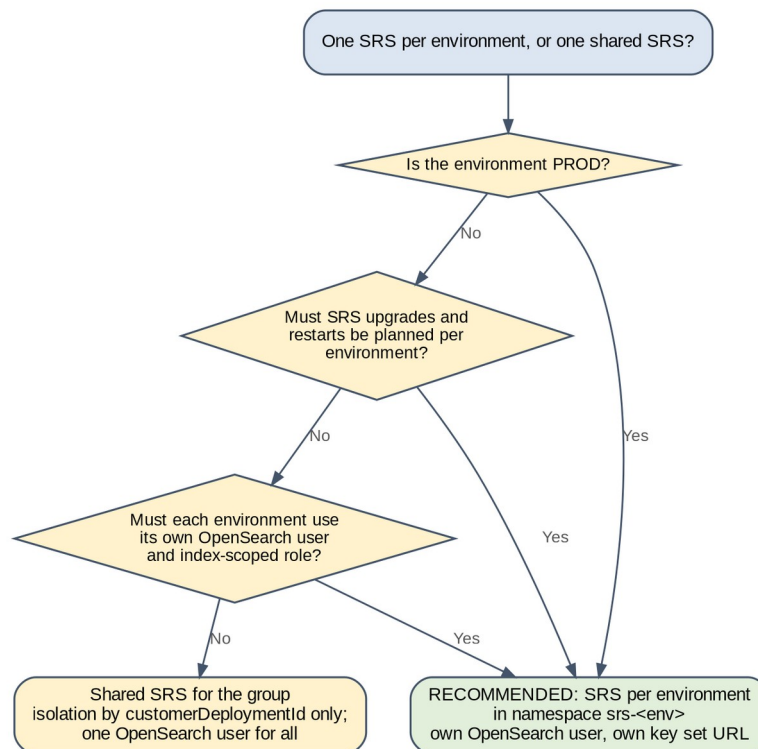
Table 28. OpenSearch sizing formulas

Quantity	Formula	Notes
Primary data (S)	Sum of <code>pri.store.size</code> for <code>pega26-<code><code></code>*</code> after a full build (S-4)	Measured, never estimated from database size. Scale to PROD by the ratio of indexed row counts
Minimum storage	$S \times (1 + \text{replicas}) \times 1.45$ [R54]	1.45 covers 10 % indexing overhead, 5 % reserved by the operating system and 20 % free space kept below the watermarks
Storage with growth	Minimum storage $\times (1 + \text{yearly growth})^{\text{years planned}}$	Add the peak seen during a full rebuild in DEV if it is above the steady state
Shard size	10 to 30 GiB per primary shard for search workloads [R55]	SRS sets shard counts. Record them; raise a Pega Support case before changing index settings
Shards per node	At most 25 shards per GiB of JVM heap [R55]	Heap is normally half the node memory. Add every environment on a shared service
Data nodes	Largest of: storage / usable disk per node; total shards / shards per node; 3 for zone spread	Use a multiple of the zone count, so each zone holds a full copy of the data

Use at least one replica, three cluster-manager nodes and data nodes in three availability zones, with the provider's zone awareness setting turned on. With one replica and zone awareness, the loss of one zone leaves a full copy of every shard, so search stays available while the provider replaces the nodes (FS-15).

7.5 Isolation in the shared OpenSearch services

Each environment has its own `customerDeploymentId` (Section 2.6). Set it explicitly in every values file, because the chart defaults it to the namespace name [R23], and treat it as immutable, as Pega asks when several environments share an SRS [R15]. A cloned environment must never reuse another environment's ID: its index build would overwrite or mix with the other environment's indexes, and its tokens would carry the other environment's `guid`.

**Figure 22. Decision: one SRS per environment or one shared SRS**

Prepared for this implementation. Not a Pega-published diagram.

Table 29. SRS per environment compared with a shared SRS

Aspect	SRS per environment (recommended)	One SRS per group
OpenSearch credentials	One user per environment, index-scoped role	One user for the whole group
Blast radius	An SRS fault or bad upgrade affects one environment	Affects every environment in the group
Upgrades	Upgrade SRS per environment, in step with that environment's tests	One upgrade for all; all environments retest
Token key set	SRS points at that environment's Okta key set	One key set; every environment's tokens come from the same server
Cost	Small: 2 pods in DEV, SIT, UAT (chart minimum [R24]); 3 in PERF, PREPROD, PROD [R14, R24]	Slightly smaller

Give each SRS its own OpenSearch user, mapped to a role limited to its index pattern [R40, R41]. Pega says the manage cluster privilege is optional when the cluster settings are applied manually [R14]. Pega does not say whether SRS needs any other cluster-level permission, so test the role in DEV (IT-06) and widen it only by the permissions the SRS log shows as missing.

Index-scoped role for one environment's SRS user (OpenSearch roles.yml format [R40, R41])

```
pega26-<code>-srs:
  cluster_permissions:
    - cluster_composite_ops
  index_permissions:
    - index_patterns:
      - "pega26-<code>*"
      allowed_actions:
        - "<index action group confirmed in DEV, for example indices_all>"
```

When an environment is refreshed or retired, delete its indexes with `DELETE /pega26-<code>*` as the environment's own SRS user, which cannot touch any other environment's indexes (Sections 19.3 and 19.4).

7.6 Pega-to-SRS tokens with Okta

Pega obtains a token with the OAuth client credentials grant, authenticating with `private_key_jwt` or `client_secret_basic`, and asks for the scope `pega.search:full` [R16, R23]. SRS checks the token signature with the key set at `OAuthPublicKeyURL`, and checks that the `guid` claim equals the `customerDeploymentId` [R23]. Figure 23 shows the flow.

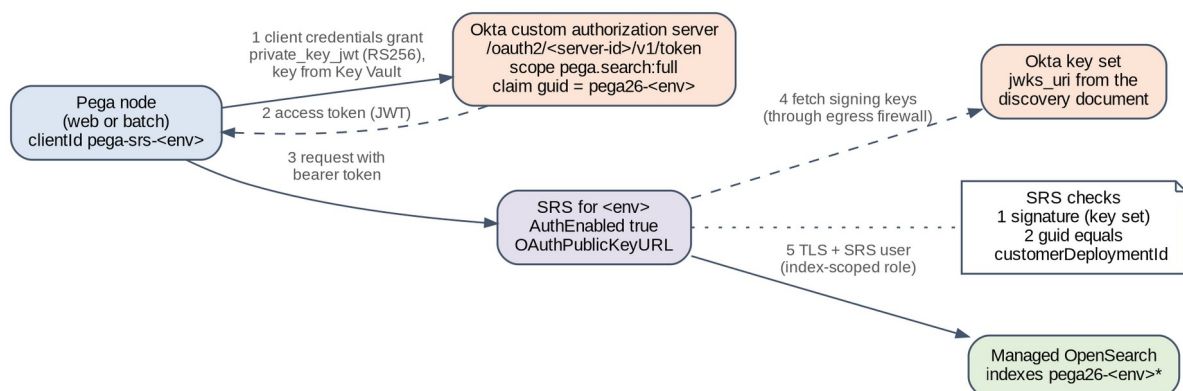


Figure 23. Token flow between Pega, Okta and SRS

Prepared for this implementation. Not a Pega-published diagram.

7.6.1 Why a custom authorization server is required

Okta's org authorization server cannot be customized: its audience, claims, policies and scopes are fixed, and its tokens are meant for Okta's own APIs [R42]. The client credentials flow has no user, so it cannot use OpenID scopes and needs a custom scope [R43]. Custom claims can only be added to a custom authorization server [R44]. Pega's scope and the `guid` claim are both custom, so Okta tokens for SRS must come from a custom authorization server. Custom authorization servers are part of Okta API Access Management [R42]. Confirm that the customer's Okta licence includes it, and how many servers it allows, before M2 (OD-05).

7.6.2 Issuing a different guid per environment

Table 30. Options for the guid claim

Option	How it works	Strengths	Weaknesses
1. One authorization server per environment	Each server has a claim <code>guid</code> with a fixed value, for example " <code>pega26-sit</code> ", included in access tokens	Simplest claim; full separation of keys and policies	Six servers; licence may limit the number
2. One shared server, claim based on the client	One claim <code>guid</code> whose value is an Okta expression on the requesting app, for example a conditional on <code>app.clientId</code> [R46]	One server for all non-production; fewer objects	Every non-production client shares a signing key set; must prove that the expression is evaluated for client credentials tokens

Okta's expression language exposes `app.id`, `app.clientId` and `app.profile` for custom claims [R46], and supports conditional expressions of the form `[Condition] ? [Value if TRUE] : [Value if FALSE]` [R46]. An example claim value for option 2 is shown below. Okta's pages do not state whether app attributes are available when the token is issued to a client with no user, so option 2 is accepted only after the DEV test.

Claim value expression for option 2 (written as one line in Okta)

```
app.clientId == "<dev-client-id>" ? "pega26-dev" :
app.clientId == "<sit-client-id>" ? "pega26-sit" :
app.clientId == "<uat-client-id>" ? "pega26-uat" :
app.clientId == "<perf-client-id>" ? "pega26-perf" :
app.clientId == "<ppd-client-id>" ? "pega26-ppd" : "none"
```

Decision. Recommendation (OD-05): PROD uses its own custom authorization server with a fixed `guid` claim of `pega26-prd`. The five non-production environments use one shared custom authorization server with the client-based claim (option 2), if the DEV test proves it. If the test fails, fall back to option 1 for non-production. Proof in DEV: request a token for the DEV client, decode it (command O-2), and confirm `scp` contains `pega.search:full` and `guid` equals `pega26-dev`. Repeat with the SIT client and confirm `pega26-sit`.

7.6.3 Okta set-up per environment

1. In the authorization server, create the scope `pega.search:full` and an access policy with a rule that allows the client credentials grant for the environment's client.
2. Create the claim `guid` for access tokens (fixed value for option 1, expression for option 2) [R44].
3. Create one API services application per environment, named `pega-srs-<code>`, with client authentication by public key / private key (`private_key_jwt`) [R45].
4. Generate an RSA key pair in a controlled workstation or pipeline. Register the public key in the application's key set (JWKS) [R45]. Store the private key, in PKCS8 form encoded with base64 as the chart requires, in Key Vault as `SRS_OAUTH_PRIVATE_KEY` [R23]. Delete local copies.
5. Record the client ID, the token endpoint and the key set URL in Appendix A.

Table 31. Okta values used in the Helm charts

Setting	Value	Source
Issuer	<code>https://<okta-domain>/oauth2/<auth-server-id></code>	[R42]
Token endpoint (<code>pegasearch.srsAuth.url</code>)	<code>https://<okta-domain>/oauth2/<auth-server-id>/v1/token</code>	[R43]
Key set URL (<code>srsRuntime.env.0AuthPublicKeyURL</code>)	The <code>jwtks_uri</code> value from <code>https://<okta-domain>/oauth2/<auth-server-id>/well-known/openid-configuration</code>	[R42]
Client ID (<code>pegasearch.srsAuth.clientId</code>)	Client ID of <code>pega-srs-<code></code>	[R23]
Authentication (<code>pegasearch.srsAuth.authType</code>)	<code>private_key_jwt</code>	[R23, R45]
Key algorithm (<code>pegasearch.srsAuth.privateKeyAlgorithm</code>)	RS256 (chart default)	[R23]
Scope (<code>pegasearch.srsAuth.scopes</code>)	<code>pega.search:full</code>	[R23]

7.6.4 Network path, token lifetime and rotation

- Pega pods call the Okta token endpoint, and SRS pods fetch the Okta key set. Both go to `<okta-domain>` on port 443 through the hub firewall. Add an FQDN allow rule for `<okta-domain>` from the Pega and SRS subnets only.
- Token lifetime is set in the access policy rule of the authorization server [R42]. Use the same lifetime in all environments and record it in Appendix A. Pega requests a new token when needed; measure how often in DEV by counting token requests in the Okta system log.
- Pega does not state whether SRS checks the issuer or audience. Test it in DEV with a token from another authorization server (FS-12) and record the result.
- Rotate the client key by adding a second public key to the application, updating `SRS_OAUTH_PRIVATE_KEY` in Key Vault, restarting the Pega tiers, checking search, and then removing the old public key. Pega does not state whether its client assertion carries a key ID (`kid`) header; test the rotation in DEV with both public keys registered (FS-34).

7.6.5 Okta signing key rotation and rate limits

- **Signing keys.** Okta rotates the signing keys of an authorization server about four times a year, and clients must look the key up from the key set URL by its `kid` rather than keep a fixed copy [R53]. SRS must therefore read `0AuthPublicKeyURL` dynamically. Prove it in DEV by rotating the authorization server's keys manually, as Okta allows, and checking that search continues without an SRS restart (FS-33). If the test fails, rotation becomes a planned event with an SRS restart, and the authorization server's key rotation mode is set to manual.
- **Rate limits.** Okta applies rate limits per org and per endpoint. A request over the limit receives HTTP 429, and Okta writes warning and violation events to the System Log [R52]. All six environments share the customer's Okta org, so every non-production token request draws on the same limit as production. The key set and discovery endpoints are not the concern; the token endpoint is.
- Measure token requests per minute per environment in DEV and under load in PERF (from the System Log), then compare the sum across all environments with the org's published limit for the token endpoint. Keep the token lifetime long enough that Pega does not request a token on each call.

- Forward Okta System Log rate-limit warnings and violations to the monitoring service (Section 18.5). A warning means another application in the org, or a misbehaving environment, is close to blocking search for every environment.

7.7 SRS deployment (backingservices chart)

Each environment has its own SRS release in namespace `srs-<code>`. The keys come from the SRS chart README [\[R24\]](#).

backingservices values for one environment (complete files in Appendix B)

```
global:
  k8sProvider: "aks"
  imageCredentials:
    registry: "<acr-name>.azurecr.io"
srs:
  enabled: true
  deploymentName: "srs-<code>"
  srsRuntime:
    replicaCount: <2 in DEV, SIT, UAT; 3 in PERF, PREPROD, PROD>
    srsImage: "<acr-name>.azurecr.io/platform-services/search-n-reporting-service-os:<tag>"
    resources:
      requests: { cpu: 1, memory: "4Gi" }
      limits: { cpu: 2, memory: "4Gi" }
    affinity:
      podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              topologyKey: topology.kubernetes.io/zone
              labelSelector:
                matchLabels:
                  app.kubernetes.io/name: srs-service
  env:
    AuthEnabled: true
    OAuthPublicKeyURL: "<okta-jwks-url>"
  ssl:
    enabled: true
    clientAuthentication: "want"
    keystore:
      file: "srs-keystore.p12"
      type: "PKCS12"
    truststore:
      file: "srs-truststore.jks"
      type: "JKS"
    certsSecret: "srs-runtime-certs"
  srsStorage:
    provisionInternalESCluster: false
    domain: "<search-host>"
    port: <search-port>
    protocol: https
    tls:
      enabled: false
    basicAuthentication:
      enabled: true
    authSecret: "srs-search-credentials"
    requireInternetAccess: false
    networkPolicy:
      enabled: false
```

7.7.1 Three chart behaviours that break this design if left at the defaults

These were found by rendering the values files with `helm template` against chart 4.13.0, not from the README alone.

Table 32. SRS chart behaviours found by rendering

Key	What the chart does	Effect	Setting in this design
<code>srsStorage.tls.enabled</code>	Means TLS certificate authentication to the search service, not encryption [R24]. The chart refuses to render when it is combined with <code>basicAuthentication.enabled</code> ("Only one authentication can be enabled")	Helm install fails	<code>false</code> . Encryption comes from <code>protocol: https</code> ; the SRS user authenticates with basic authentication
<code>pegasearch.externalURL</code>	With <code>srsRuntime.ssl.enabled: true</code> , the SRS container serves HTTPS on 8443 only, and its probes call <code>https://localhost:8443/health</code> . The Service also exposes 8080 and 80, which point at a port that is not listening [R62]	An HTTPS URL without a port goes to 443, which the Service does not expose; search fails at first start	<code>https://srs- <code>.srs- <code>.svc.cluster.local:8443</code>
<code>srsStorage.networkPolicy.enabled</code>	Allows ingress to SRS from <code>0.0.0.0/0</code> on 8080 and 8443, and egress only to an in-cluster <code>elasticsearch-master</code> on 9200 and DNS [R62]. <code>requireInternetAccess</code> only adds a pod label	SRS cannot reach the external OpenSearch service or the Okta key set; any pod can call SRS	<code>false</code> , replaced by the policy below

Network policies for `srs-<code>`, replacing the chart policy

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: srs-<code>-default-deny
  namespace: srs-<code>
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: srs-<code>-allow
  namespace: srs-<code>
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: srs-service
  policyTypes: ["Ingress", "Egress"]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: pega-<code>
      ports:
        - protocol: TCP
          port: 8443
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    - to:
        - ipBlock:
            cidr: "<opensearch-private-endpoint-cidr>"
      ports:
        - protocol: TCP
          port: 443
    - to:
        - ipBlock:
            cidr: "0.0.0.0/0"
            except: ["10.0.0.0/8", "172.16.0.0/12", "192.168.0.0/16"]
      ports:

```

```
- protocol: TCP
  port: 443
```

The policy allows SRS to be called only from `pega-<code>` on 8443, and lets SRS reach DNS, the OpenSearch private endpoint and public HTTPS. Okta has no fixed address range, so the last rule is limited to public addresses on 443, and the hub firewall restricts it to `<okta-domain>` by FQDN. Kubernetes network policies are enforced only if the AKS cluster was created with a network policy engine; check this before relying on them.

7.8 Index build planning

After Pega connects to SRS, SRS indexes all searchable data, and Pega states that this needs a downtime period. The length depends on the data model, the resources given to Pega and SRS, the number of queue processors and the amount of searchable data [R15]. For a cloned environment, the build runs during the first start with intake held (Section 10.4), so its length sets part of the production outage.

1. In every rehearsal, record the start and end of the build, the number of batch pods, SRS replicas and OpenSearch data nodes, and the row counts of the main indexed classes.
2. Plot build time against searchable data volume across DEV, PERF and PREPROD. PREPROD, built from a recent full-size clone on production-like services, is the best predictor.
3. Estimate production time as the PREPROD time scaled by the ratio of production to PREPROD data volume, plus 25 % contingency. Use the larger of this and the measured PREPROD time if the volumes are close.
4. If the estimate does not fit the outage window, add batch pods and OpenSearch data nodes for the cutover and measure again.

The build is complete when the search landing page shows every class indexed with no errors [R19], the document count of each main index matches the row count of its class in the database within the tolerance agreed with the application team, and no class shows CONFLICTS FOUND [R19]. Pega documents the rebuild steps and screens [R17].

7.9 Backups and restore

Every index can be rebuilt from the Pega database, so OpenSearch snapshots are optional. They shorten recovery but add storage cost.

Table 33. Recovery choices for search

Choice	Recovery after loss of the OpenSearch service	Recovery time
No snapshots	Recreate the service, apply the cluster settings, restart SRS, run a full index build	Measured full build time plus provisioning time
Provider snapshots	Restore the latest snapshot, then reindex classes changed since the snapshot	Restore time plus a partial build; measure in PERF

Elasticsearch 8.x snapshots cannot be restored into OpenSearch [R38, R39], and the 8.8 embedded indexes are not in a snapshot-ready form, so snapshots play no part in moving from 8.8.

8 Secrets, certificates and identity

No password, key or JAAS string is written in a Helm values file. The Pega Helm charts support external secrets for the database, stream, SRS OAuth and certificates [R23]. Key Vault is the system of record, one vault per environment, and the External Secrets Operator copies secrets into Kubernetes [R48].

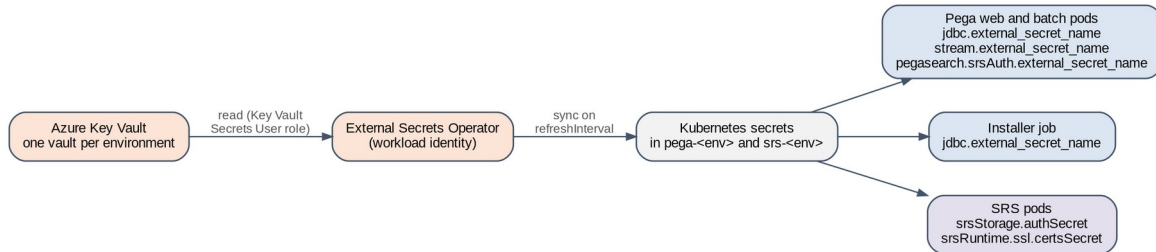


Figure 24. Secret delivery from Key Vault to Pega, installer and SRS pods

Prepared for this implementation. Not a Pega-published diagram.

8.1 Secret inventory

Table 34 lists every secret. Key names marked as fixed are required by the Pega charts; the secret names are this design's convention.

Table 34. Secret inventory (one set per environment, in kv-pegas-<code>)

Kubernetes secret	Keys	Used by	Key Vault secrets	Key names
pega-db-secret	DB_USERNAME, DB_PASSWORD	Pega tiers and installer job (global.jdbc.external_secret_name)	pega-db-username, pega-db-password	Fixed [R23]
pega-stream-secret	STREAM_TRUSTSTORE_PASSWORD, STREAM_KEYSTORE_PASSWORD, STREAM_JAAS_CONFIG	Pega tiers (stream.external_secret_name)	stream-truststore-password, stream-keystore-password, confluent-jaas	Fixed [R23]
pega-srs-oauth	SRS_OAUTH_PRIVATE_KEY	Pega tiers (pegasearch.srsAuth.external_secret_name)	okta-srs-private-key	Fixed [R23]
srs-search-credentials	username, password	SRS (srs.srsStorage.authSecret)	opensearch-srs-username, opensearch-srs-password	Per SRS README [R24]
srs-runtime-certs	keystore and truststore files, keystorePassword, truststorePassword	SRS (srs.srsRuntime.ssl.certsSecret)	srs-tls-keystore, srs-tls-truststore, passwords	Per SRS README [R24]
pega-tier-tls	TOMCAT_KEYSTORE_CONTENT, TOMCAT_KEYSTORE_PASSWORD, ca.crt	Web tier TLS (tier.service.tls.external_secret_names)	pega-tls-keystore, pega-tls-password, pega-tls-ca	Fixed [R23]

8.2 External Secrets Operator manifests

SecretStore and ExternalSecrets for the Pega namespace (Azure Key Vault provider [R48])

```

apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: keyvault
  namespace: pega-<code>
spec:
  provider:

```

```

    azurekv:
      authType: WorkloadIdentity
      vaultUrl: "https://kv-peg-<code>.vault.azure.net"
      serviceAccountRef:
        name: eso-keyvault-reader
    ---
  apiVersion: external-secrets.io/v1beta1
  kind: ExternalSecret
  metadata:
    name: pega-stream-secret
    namespace: pega-<code>
  spec:
    refreshInterval: 1h
    secretStoreRef:
      name: keyvault
      kind: SecretStore
    target:
      name: pega-stream-secret
    data:
      - secretKey: STREAM_TRUSTSTORE_PASSWORD
        remoteRef: { key: stream-truststore-password }
      - secretKey: STREAM_KEYSTORE_PASSWORD
        remoteRef: { key: stream-keystore-password }
      - secretKey: STREAM_JAAS_CONFIG
        remoteRef: { key: confluent-jaas }
    ---
  apiVersion: external-secrets.io/v1beta1
  kind: ExternalSecret
  metadata:
    name: pega-srs-oauth
    namespace: pega-<code>
  spec:
    refreshInterval: 1h
    secretStoreRef:
      name: keyvault
      kind: SecretStore
    target:
      name: pega-srs-oauth
    data:
      - secretKey: SRS_OAUTH_PRIVATE_KEY
        remoteRef: { key: okta-srs-private-key }

```

Create the same pattern for `pega-db-secret` and `pega-tier-tls` in `pega-<code>`, and for `srs-search-credentials` and `srs-runtime-certs` in `srs-<code>`. Each namespace has its own SecretStore and workload identity, and that identity can read only its own environment's vault.

Note. Check which External Secrets Operator API version is installed (`v1beta1` or `v1`) and use it. The stream truststore and keystore password keys can hold empty values with SASL/PLAIN and a public CA, but keep the keys in the order the chart expects [\[R23\]](#).

8.3 Rotation

Table 35. Secret rotation

Secret	Steps	Restart needed
Confluent API key	Create a second key for the same service account; write the new JAAS value to Key Vault; wait for or force the External Secrets refresh; rolling restart of the Pega tiers; check the Stream landing page; delete the old key	Yes, Pega tiers
OpenSearch SRS user password	Change the password on the provider; update Key Vault; refresh; restart SRS; check search (S-5)	Yes, SRS
Okta client key	Add the new public key to the Okta app; update <code>okta-srs-private-key</code> ; refresh; restart Pega tiers; check search; remove the old public key	Yes, Pega tiers
Database password	Per the DBA process; update Key Vault; refresh; restart Pega tiers	Yes

Caution. Pods read these values at start. A new Key Vault version reaches the Kubernetes secret on the next refresh, but running pods keep the old value until they restart. A rotation without a restart works until the old credential is deleted, then fails (FS-20).

8.4 Certificates

Table 36. Certificates

Certificate	Issued by	Used for	Renewal owner
Public TLS certificate for the Pega host name	Customer public CA	Application Gateway listener	Platform team
Web tier backend certificate	Customer private CA	Application Gateway to web pods	Platform team
SRS server certificate	Customer private CA	Pega to SRS TLS	Platform team
Confluent broker certificates	Confluent (public CA)	SASL_SSL from Pega	Confluent
OpenSearch endpoint certificate	Provider or customer CA	SRS to OpenSearch TLS	OpenSearch provider
Okta endpoint certificate	Okta (public CA)	Token and key set calls	Okta

Caution. Do not route Pega-to-Confluent, SRS-to-OpenSearch or Okta traffic through a TLS-inspecting proxy. Re-signed certificates break the trust chain that the clients check.

9 Helm configuration per environment

9.1 Chart version and the three runs

This design uses Pega Helm charts 4.13.0: the `pega` chart for Pega Platform and the installer job, and the `backingservices` chart for SRS [R23, R24]. Every key in this section and in Appendix B was checked against the README and default values file of that chart version. Each environment built from a clone goes through three Helm runs of the `pega` chart, shown in Table 37.

Table 37. Helm runs of the `pega` chart for an environment built from a clone

Run	<code>global.actions.execute</code>	What runs	Why
U: upgrade	<code>upgrade</code>	Installer job only; no Pega pods	Upgrades the cloned database. Pega must not start until the clean-up in Section 10.3 is done.
F: first start	<code>deploy</code>	Web tier only; batch tier <code>replicas: 0</code> with HPA off	No queue processor, job scheduler or real-time data flow runs while cloned items are checked (Section 10.4).
D: steady state	<code>deploy</code>	Web and batch tiers at normal size	Index build, then normal operation.

The chart's actions are `install`, `deploy`, `install-deploy`, `upgrade` and `upgrade-deploy` [R23]. `upgrade-deploy` is not used, because it would start Pega straight after the upgrade, before the cloned items are checked.

9.1.1 Upgrade type

The chart supports the upgrade types `in-place` (rules and data in one run, with downtime), `out-of-place-rules` and `out-of-place-data` (a two-step out-of-place upgrade), `zero-downtime` and `custom` [R23]. The chart default is `in-place`. The zero-downtime type needs REST credentials for the running pre-upgrade system to perform pre- and post-actions [R23], and a cloned database has no running 8.8 system attached. The design therefore uses `in-place` on the clone, which is disposable until the cutover, and asks Pega Support to confirm it (GQ-04).

9.2 Pega chart keys per environment

Table 38 lists every `pega` chart key this design sets, with its value per environment. Keys not listed keep the chart default. Values in angle brackets are recorded in Appendix A.

Table 38. Pega chart keys per environment (chart 4.13.0)

Key	Meaning	DEV	SIT	UAT	PERF	PREPROD	PROD	Source
<code>global.provider</code>	Kubernetes provider	aks	aks	aks	aks	aks	aks	[R23]
<code>global.actions.execute</code>	Run type (Table 37)	U/F/D	U/F/D	U/F/D	U/F/D	U/F/D	U/F/D	[R23]
<code>global.customerDeploymentId</code>	Index prefix and guid value	pega26-dev	pega26-sit	pega26-uat	pega26-perf	pega26-ppd	pega26-prd	[R23]
<code>global.jdbc.url</code>	Upgraded clone for this environment	<jdbc-url>	<jdbc-url>	<jdbc-url>	<jdbc-url>	<jdbc-url>	<jdbc-url>	[R23]
<code>global.jdbc.external_secret_name</code>	DB_USERNAME, DB_PASSWORD	pega-db-secret	pega-db-secret	pega-db-secret	pega-db-secret	pega-db-secret	pega-db-secret	[R23]
<code>global.jdbc.rulesSchema, dataSchema</code>	Schemas of the clone	<rules-schema>	<rules-schema>	<rules-schema>	<rules-schema>	<rules-schema>	<rules-schema>	[R23]
<code>global.docker.pega.image</code>	Pega image	platform / pega:26.1.1	platform / pega:26.1.1	platform / pega:26.1.1	platform / pega:26.1.1	platform / pega:26.1.1	platform / pega:26.1.1	[R23]

Key	Meaning	DEV	SIT	UAT	PERF	PREPROD	PROD	Source
tier[web].replicas and HPA min / max	Web pods	1 / 2	1 / 3	2 / 3	3 / 6	3 / 6	3 / 6	[R23]
tier[batch].replicas and HPA min / max	Batch pods (run F: 0, HPA off)	1 / 2	1 / 3	1 / 3	3 / 6	3 / 6	3 / 6	[R23]
tier[*].pdb.enabled	Pod disruption budget	true	true	true	true	true	true	[R23]
tier[*].resources	Memory, CPU request / limit	12Gi, 3 / 4	12Gi, 3 / 4	12Gi, 3 / 4	12Gi, 3 / 4	12Gi, 3 / 4	12Gi, 3 / 4	[R23]
tier[*].topologySpreadConstraints	Spread over zones (Section 4.6)	zone, skew 1	zone, skew 1	zone, skew 1	zone, skew 1	zone, skew 1	zone, skew 1	[R61]
tier[web].service.tls.external_secret_names	Backend TLS keystore	pega-tier-tls	pega-tier-tls	pega-tier-tls	pega-tier-tls	pega-tier-tls	pega-tier-tls	[R23]
tier[web].ingress.domain	Host name	<pega-host>	<pega-host>	<pega-host>	<pega-host>	<pega-host>	<pega-host>	[R23]
cassandra.enabled	No Decision Data Store	false	false	false	false	false	false	[R23]
hazelcast.enabled, clusteringServiceEnabled	Hazelcast removed in '25	false	false	false	false	false	false	[R5, R23]
stream.enabled	Stream service on	true	true	true	true	true	true	[R23]
stream.bootstrapServer	Confluent bootstrap	cc-np1	cc-np1	cc-np1	cc-np2	cc-np2	cc-prd	[R23]
stream.securityProtocol	TLS and SASL	SASL_SSL	SASL_SSL	SASL_SSL	SASL_SSL	SASL_SSL	SASL_SSL	[R23]
stream.saslMechanism	API key	PLAIN	PLAIN	PLAIN	PLAIN	PLAIN	PLAIN	[R23]
stream.streamNamePattern	Topic prefix	pega-dev-{stream.name}	pega-sit-{stream.name}	pega-uat-{stream.name}	pega-perf-{stream.name}	pega-ppd-{stream.name}	pega-prd-{stream.name}	[R23]
stream.replicationFactor	Replicas per partition	3	3	3	3	3	3	[R23]
stream.external_secret_name	STREAM_* keys	pega-stream-secret	pega-stream-secret	pega-stream-secret	pega-stream-secret	pega-stream-secret	pega-stream-secret	[R23]
pegasearch.externalSearchService	Use SRS	true	true	true	true	true	true	[R23]
pegasearch.externalURL	SRS service, HTTPS port 8443	rs-dev:8443	rs-sit:8443	rs-uat:8443	rs-perf:8443	rs-ppd:8443	rs-prd:8443	[R23, R62]
pegasearch.srsAuth.enabled	OAuth to SRS	true	true	true	true	true	true	[R23]
pegasearch.srsAuth.url	Okta token endpoint	non-prod server	non-prod server	non-prod server	non-prod server	non-prod server	PROD server	[R23, R43]
pegasearch.srsAuth.clientId	Okta client	pega-srs-dev	pega-srs-sit	pega-srs-uat	pega-srs-perf	pega-srs-ppd	pega-srs-prd	[R23]
pegasearch.srsAuth.scopes	Scope	pega.search:full	pega.search:full	pega.search:full	pega.search:full	pega.search:full	pega.search:full	[R23]
pegasearch.srsAuth.authType	Client authentication	private_key_jwt	private_key_jwt	private_key_jwt	private_key_jwt	private_key_jwt	private_key_jwt	[R23]
pegasearch.srsAuth.privateKeyAlgorithm	Key algorithm	RS256	RS256	RS256	RS256	RS256	RS256	[R23]
pegasearch.srsAuth.external_secret_name	SRS_OAUTH_PRIVATE_KEY	pega-srs-oauth	pega-srs-oauth	pega-srs-oauth	pega-srs-oauth	pega-srs-oauth	pega-srs-oauth	[R23]
pegasearch.srsMTLS.enabled	Mutual TLS to SRS	false	false	false	false	false	false	[R23]
installer.image	Installer image	installer:26.1.1	installer:26.1.1	installer:26.1.1	installer:26.1.1	installer:26.1.1	installer:26.1.1	[R23]
installer.upgrade.upgradeType	Upgrade type (GQ-04)	in-place	in-place	in-place	in-place	in-place	in-place	[R23]

Note. Starting replica counts are sized for function, not load. Set the PERF, PREPROD and PROD values from the PERF load test (Section 15), and keep PREPROD equal to PROD.

9.2.1 Topology spread, heap and garbage collection logging

- `topologySpreadConstraints` is read by the tier template of chart 4.13.0 [R61] but is not listed in the README, so check it in the `helm template` output after every chart upgrade. The label selector must match the pod label the chart sets, `app: <deployment name>-<tier name>`, for example `app: pega-web`.
- The chart defaults the Pega container to 12Gi of memory with a commented heap of 8192m [R23]. Keep the heap at about two thirds of the container memory: the JVM also needs native memory for threads, metaspace, Kafka client buffers and the code cache, and a container that exceeds its limit is killed without a Java error.
- Set the JVM arguments Pega recommends in `tier[*].javaOpts` [R63]: garbage collection logging to `/usr/local/tomcat/logs/gc.log`, `-XX:MaxMetaspaceSize=768m` so metaspace cannot exhaust node memory, `-XX:+UseStringDeduplication` and `-XX:+HeapDumpOnOutOfMemoryError`. The values files in Appendix B carry them. GC logs and heap dumps are on ephemeral pod storage, so the log collector must tail the GC file (Section 18.2).
- Pega images run in Etc/UTC. If the cloned 8.8 database does not use UTC, set `-Duser.timezone` to the database time zone in `javaOpts` and in the installer's `customJVMArgs` as well [R63]. A mismatch between the two shifts delayed queue items and job scheduler times.
- Long GC pauses show up as Kafka consumer group rebalances and as SRS request timeouts, so the GC log is the first thing to read when those appear (Section 17).
- Set `pegaDiagnosticUser` and its password through the diagnostic secret, so support staff can download Tomcat logs without a redeploy [R23].

9.3 Backingservices chart keys per environment

Table 39. Backingservices chart keys per environment

Key	Meaning	DEV	SIT	UAT	PERF	PREPROD	PROD	Source
<code>global.k8sProvider</code>	Provider	aks	aks	aks	aks	aks	aks	[R24]
<code>srs.deploymentName</code>	SRS release	srs-dev	srs-sit	srs-uat	srs-perf	srs-ppd	srs-prd	[R24]
<code>srs.srsRuntime.replicaCount</code>	SRS pods	2	2	2	3	3	3	[R14, R24]
<code>srs.srsRuntime.srsImage</code>	SRS image	<code>-os:<tag></code>	<code>-os:<tag></code>	<code>-os:<tag></code>	<code>-os:<tag></code>	<code>-os:<tag></code>	<code>-os:<tag></code>	[R14, R24]
<code>srs.srsRuntime.env.AuthEnabled</code>	Token checks	true	true	true	true	true	true	[R24]
<code>srs.srsRuntime.env.OAuthPublicKeyURL</code>	Okta key set	non-prod server	non-prod server	non-prod server	non-prod server	non-prod server	PROD server	[R24, R42]
<code>srs.srsRuntime.ssl.*</code>	TLS on SRS	enabled	enabled	enabled	enabled	enabled	enabled	[R24]
<code>srs.srsStorage.provisionInternalEScluster</code>	External search	false	false	false	false	false	false	[R24]
<code>srs.srsStorage.domain</code>	OpenSearch host	os-np1	os-np1	os-np1	os-np2	os-np2	os-prd	[R24]
<code>srs.srsStorage.port, protocol</code>	Endpoint	443, https	443, https	443, https	443, https	443, https	443, https	[R24]
<code>srs.srsRuntime.resources</code>	CPU request / limit, memory	1 / 2, 4Gi	1 / 2, 4Gi	1 / 2, 4Gi	1 / 2, 4Gi	1 / 2, 4Gi	1 / 2, 4Gi	[R14, R24]
<code>srs.srsRuntime.affinity</code>	Zone anti-affinity	preferred	preferred	preferred	preferred	preferred	preferred	[R24]
<code>srs.srsStorage.tls.enabled</code>	Certificate authentication, not encryption (Table 32)	false	false	false	false	false	false	[R24]
<code>srs.srsStorage.basicAuthentication.enabled</code>	SRS user	true	true	true	true	true	true	[R24]
<code>srs.srsStorage.authSecret</code>	username, password	srs-search-credentials	srs-search-credentials	srs-search-credentials	srs-search-credentials	srs-search-credentials	srs-search-credentials	[R24]

Key	Meaning	DEV	SIT	UAT	PERF	PREPROD	PROD	Source
<code>srs.srsStorage.requireInternetAccess</code>	No internet egress	false	false	false	false	false	false	[R24]
<code>srs.srsStorage.networkPolicy.enabled</code>	Chart policy off; own policy (Section 7.7)	false	false	false	false	false	false	[R62]

9.4 Differences between the runs

Only two places change between runs: the action, and the batch tier size. Table 40 shows them for SIT; PROD is the same with its own sizes. Appendix B has the complete files for SIT and PROD.

Table 40. Values that change between runs

Key	Run U (upgrade)	Run F (first start)	Run D (steady state)
<code>global.actions.execute</code>	upgrade	deploy	deploy
<code>tier[batch].replicas</code>	not used	0	1 in SIT, 3 in PROD
<code>tier[batch].hpa.enabled</code>	not used	false	true
<code>installer.*</code>	used	ignored	ignored

9.5 Pipeline rules

- Keep one values file per environment and run in Git. Each change goes through review by the platform team and the Pega LSA.
- Pin the chart version (`helm upgrade --install ... --version 4.13.0`) and record image digests in Appendix A.
- Run `helm template` and `helm lint` with the values file in the pipeline before every apply.
- Fail the pipeline if a values file contains a literal password, API secret, JAAS string or private key. Secrets come only through `external_secret_name` keys.
- Fail the pipeline if `customerDeploymentId` or `streamNamePattern` does not match the environment's row in Table 8.

10 The cloned database

A database cloned from 8.8 production carries 8.8 settings, records and work. Some of it refers to the embedded Kafka and embedded Elasticsearch that no longer exist, and some of it points at production systems. This section lists what must be confirmed with Pega Support, what must be cleaned up, and the controlled first start that every cloned environment follows.

10.1 Gating questions for Pega Support

Pega's pages say that Hazelcast must be removed before updating to '25 or later, and that this applies to embedded Hazelcast and the Clustering Service [R5]. They do not say how this applies to a database that is cloned, upgraded offline and only ever started on 26.1.1. Table 41 lists the questions to raise in one Pega Support case before M2. Any condition Pega sets becomes a mandatory step in Section 12.7.

Table 41. Gating questions for Pega Support

ID	Question	Why it matters	Evidence found	Answer	Date
GQ-01	Is a direct upgrade of an 8.8 database to 26.1.1 with the 26.1.1 installer supported?	The whole path depends on it	Not settled by the pages read	Open	
GQ-02	Which 8.8 patch level is required before the clone?	May need an 8.8 patch in production before the final clone	Not settled	Open	
GQ-03	Does the Hazelcast removal requirement apply to a clone upgraded offline? If so, which settings must be in the database before the upgrade?	Pega states removal is required before updating to '25 [R5]; the removal steps assume a running system [R6]	Not settled for this case	Open	
GQ-04	Which installer upgrade type suits a cloned database: <i>in-place</i> , or <i>out-of-place-rules</i> then <i>out-of-place-data</i> ?	Sets run U and the outage length	Chart types listed in [R23]	Open	
GQ-05	How does 26.1.1 treat 8.8 stream and search DSS and node records in an upgraded database, and which takes precedence: DSS in the database or Helm settings?	A stale DSS could override the Helm configuration	Not settled; tested as FS-24	Open	
GQ-06	How are 8.8 delayed and broken queue items processed after the upgrade, given the '26 queue processing changes?	Items may fail or run unexpectedly	'26 changes queue processing [R3]	Open	
GQ-07	Is any SRS or customerDeploymentId state stored in the database that a clone would carry?	Could link a clone to another environment's indexes	Not settled; tested as IT-05	Open	
GQ-08	Does the 26.1.1 installer need a Kafka connection while it upgrades an 8.8 database, and if so how is it supplied in the Helm installer job?	Run U could fail or stall; chart 4.13.0 passes no stream settings to the installer job	From '25, Pega command-line tooling needs Kafka [R8]; rendered run U has no <code>STREAM_*</code> values	Open	

Important. Until the answers arrive, the plan assumes the conservative answer for each question: rehearse the full path twice, apply the clean-up in Section 10.3, and treat any difference from expected behaviour as a stop condition at the first-start Go/No-Go.

10.2 How Helm settings and database settings interact

Pega 26.1.1 gets its stream and search configuration from the Helm chart. A cloned 8.8 database may also hold Dynamic System Settings (DSS) for the stream service or for search. Pega documents the stream setting names, for example `services/stream/provider`, `services/stream/broker/url` and

`services/stream/name/pattern` [R8]. The pages read do not settle whether a DSS with such a name in the database would override the value the Helm chart supplies. The design therefore treats it as a risk: search the cloned DSS for any setting that contains `services/stream` or refers to search hosts, record them, and prove in the first DEV start that the landing pages show the Helm values (FS-24). GQ-05 asks Pega Support for the rule.

10.3 Inventory and clean-up

Table 42 lists the items that affect Kafka and search. For each one it gives where it lives, what 26.1.1 does with it, the risk, the action, when the action happens and the source. Time points are: B (before the upgrade, on the clone), A (after the upgrade, before the first start), F (during the first start, with the batch tier at zero), and S (after the first start). No table names are given: Pega has not published them for these items, and each change uses the Pega screen named.

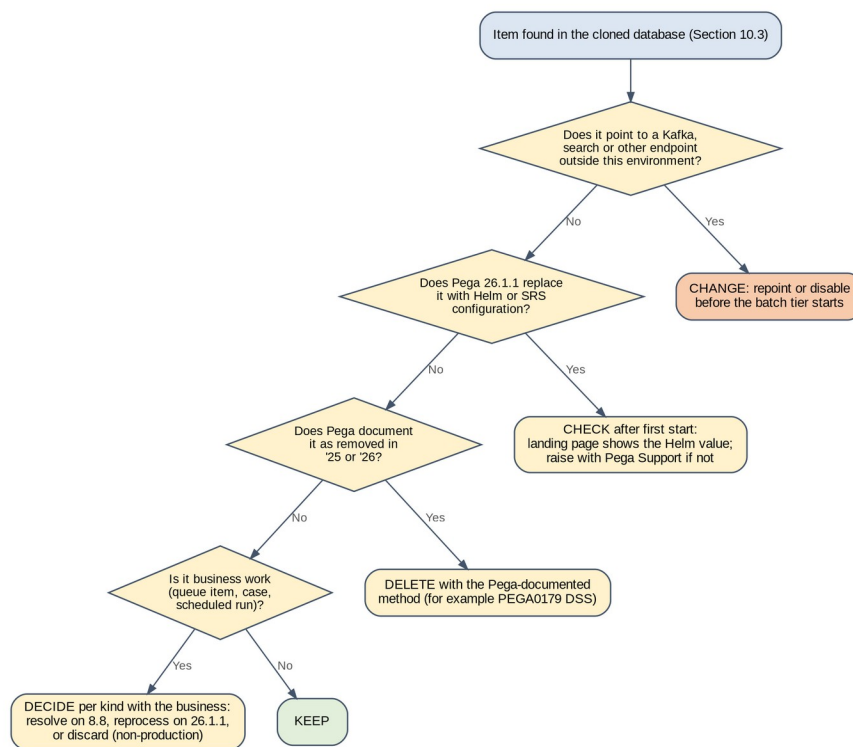


Figure 25. Decision: treatment of an item found in the cloned database

Prepared for this implementation. Not a Pega-published diagram.

Table 42. Kafka and search items in the cloned database

ID	Item	Where it lives	26.1.1 behaviour	Risk	Action	When	Source
CD-01	Stream service DSS from 8.8 (provider, broker URL, name pattern, replication)	DSS (Records > SysAdmin > Dynamic System Settings)	Helm supplies the stream settings; precedence not documented	Stale value overrides Helm	Record; check landing page shows Helm values; remove only as Pega Support advises (GQ-05)	F	[R8, R12]
CD-02	Embedded stream node records from 8.8	Stream service landing page (Configure > Decisioning > Infrastructure > Services > Stream)	External Kafka has no stream nodes	Landing page shows unavailable 8.8 nodes; confusion during checks	Check; remove stale entries from the landing page if shown	F	[R13]
CD-03	Search settings and index host node records from 8.8 embedded Elasticsearch	DSS and search landing page	SRS configured by Helm replaces embedded search	Stale setting points search at 8.8 nodes	Check search landing page shows SRS; record and remove stale settings as Pega Support advises	F	[R14, R16]
CD-04	Search index status from 8.8	Search landing page	Indexes are rebuilt through SRS	Status shows old indexes as present	Full index build; check status [R19]	S	[R15, R19]

ID	Item	Where it lives	26.1.1 behaviour	Risk	Action	When	Source
CD-05	Delayed queue items	Database (queue processor delayed items)	Become due and are queued to Kafka when their time arrives	Items run on 26.1.1 against changed rules, or call production systems from a clone	Count on 8.8 before the clone; in non-production, decide per queue processor to discard or keep; in PROD, keep and test in PREPROD (GQ-06)	B, S	[R3]
CD-06	Broken queue items	Database (queue processor broken items, shown in Admin Studio)	Stay broken until requeued or deleted	Requeue on 26.1.1 runs old work with new rules	Resolve on 8.8 before the final clone where possible; record counts; decide per queue processor	B, S	[R3]
CD-07	Job scheduler state	Job scheduler rules and their run state	Job schedulers run on batch nodes after start	Clone runs production jobs (email, extracts) on first start	Batch tier at zero in run F; disable jobs that call external systems in non-production before scaling the batch tier	F	[R11]
CD-08	Data flow runs, including real-time runs on Kafka data sets	Data flow landing pages	Runs restart against 26.1.1 topics or the customer's Kafka	Runs resume against old offsets or production topics	Stop or delete runs that refer to 8.8 stream partitions; restart from the agreed point	F, S	[R11]
CD-09	Kafka configuration instances and Kafka data sets for application integrations	Records > SysAdmin > Kafka; Records > Data Model > Data Set	Connect to the customer's own Kafka clusters	A non-production clone reads production topics and moves production consumer offsets	Repoint to test clusters or disable in every non-production environment; decide per data set in PROD (Section 11.3)	F	[R11]
CD-10	Other production endpoints: connectors, listeners, email accounts, file listeners	Integration rules and data instances	Run on start if enabled	A clone calls or polls production systems	Short checklist owned by the application team; firewall denies production endpoints from non-production	F	Customer
CD-11	DSS delayeditems/dataflowbased/threadspersnode	DSS	No longer used in '26	PEGA0179 alert	Delete	A	[R3]
CD-12	Hazelcast-related settings from 8.8	DSS and prconfig values	Hazelcast removed in '25	Unknown until GQ-03 is answered	Apply Pega Support's instructions	B or A	[R5]
CD-13	Tables without primary keys	Database	Required for '25 and later	Upgrade blocked	Run primaryKeyUtility as the DBA team plans	B	[R10]
CD-14	Custom search properties and reports that rely on search	Rules	Indexed by SRS after the build	Feature behaves differently on SRS	List them; include in the functional checks (Section 14.4)	S	[R14]
CD-15	Partition count DSS prconfig/dsm/services/stream/pyTopicPartitionsCount/default and per-topic partition changes made on 8.8	DSS (ruleset Pega-Engine)	Applies to every topic created at first start [R49]	Partition count, and so Confluent capacity and cost, differs from the plan	Record the value; keep or reset to 6 with the performance lead; recount with K-5	A	[R49]
CD-16	Pega Diagnostic Center endpoint and settings from production	Configure > System > Settings	Clone sends diagnostics to the production PDC endpoint [R51]	Non-production alerts mixed with production in PDC	Point each environment at its own PDC endpoint, or clear it	F	[R51]

Caution. Do not change these items with direct SQL against Pega tables unless Pega Support provides the statement for this release. Use the Pega screens named in the table during run F, when no background processing is running.

10.4 First-start control

Every environment built from a clone follows the same scripted first start. Table 43 gives each step an owner, a check and the evidence to keep, and Figure 26 shows the Go/No-Go logic.

Table 43. First-start steps for an environment built from a clone

Step	Action	Owner	Check	Evidence
ST-1	Run U: upgrade the clone with the installer job	Platform team, DBA	Job completed; no errors in the log (P-5)	Installer log
ST-2	Apply time-point A actions from Table 42	Pega LSA, DBA	Each item signed off	Clean-up record (Appendix D)
ST-3	Run F: deploy the web tier with this environment's prefix, service account and customerDeploymentId; batch tier at zero	Platform team	Web pods Ready (P-1)	kubect! output
ST-4	Hold intake: no public DNS record, inbound listeners off, batch tier at zero	Platform team, application team	No user or inbound traffic in logs	Ingress and listener status
ST-5	Check the Stream landing page and the Search landing page; apply time-point F actions	Pega LSA	Provider ExternalKafka, status NORMAL, prefix <code>pega-</code> ; search uses SRS with <code>pega26-</code>	Screenshots
ST-6	Check the firewall log for denied connections to production or other environments	Platform team	No attempts, or each attempt traced to an item and fixed	Firewall log extract
ST-7	Run D: scale the batch tier; start the full index build	Platform team, Pega LSA	Index build running; batch pods Ready	Start time
ST-8	Check index completeness and counts (Section 7.8)	Pega LSA	All classes indexed; counts match	Count sheet
ST-9	Release intake: public DNS, listeners and job schedulers as agreed	Application team	Smoke tests pass	Test report

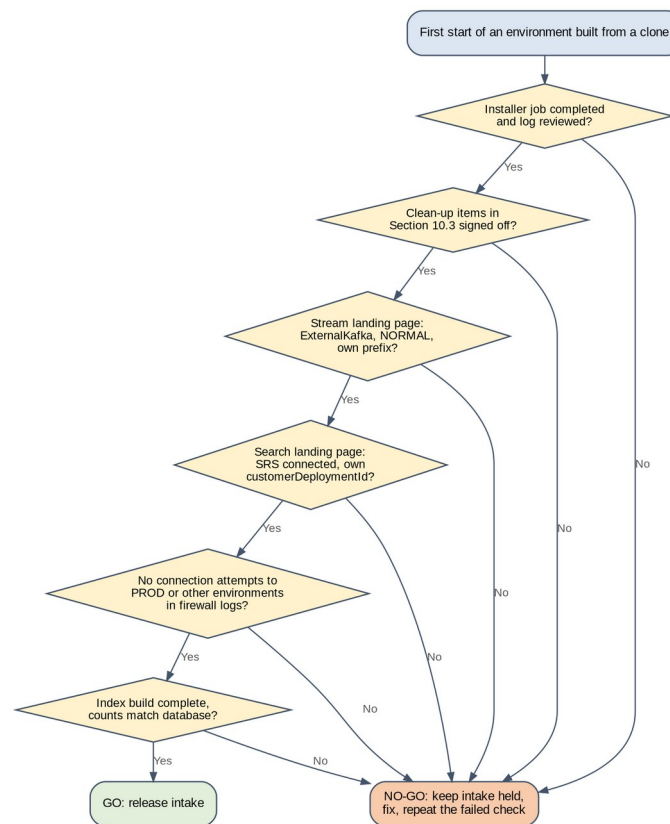


Figure 26. Decision: Go/No-Go at the first start
 Prepared for this implementation. Not a Pega-published diagram.

10.5 Data protection and masking

Non-production clones of production data must follow the customer's masking policy. Mask the clone before the upgrade (time point B). The index build then reads only masked data, and no unmasked value reaches OpenSearch, Kafka topics or logs. If masking runs after the upgrade, it must still finish before run D, because the index build copies searchable data into OpenSearch. Masking after the index build leaves unmasked data in the indexes until a full rebuild. The masking approach is OD-07.

Note. Masking rules must keep values that integrations and searches depend on in a usable form, for example case IDs and keys used in search. Agree the rules with the application team, and include search checks on masked data in the functional tests.

11 The 8.8 content question

11.1 Answer

Nothing in the 8.8 Kafka or search storage is migrated. Figure 27 shows the decision for each kind of content, and Table 44 gives the treatment and the evidence. Each statement was checked against the current Pega pages on the evidence cut-off date.

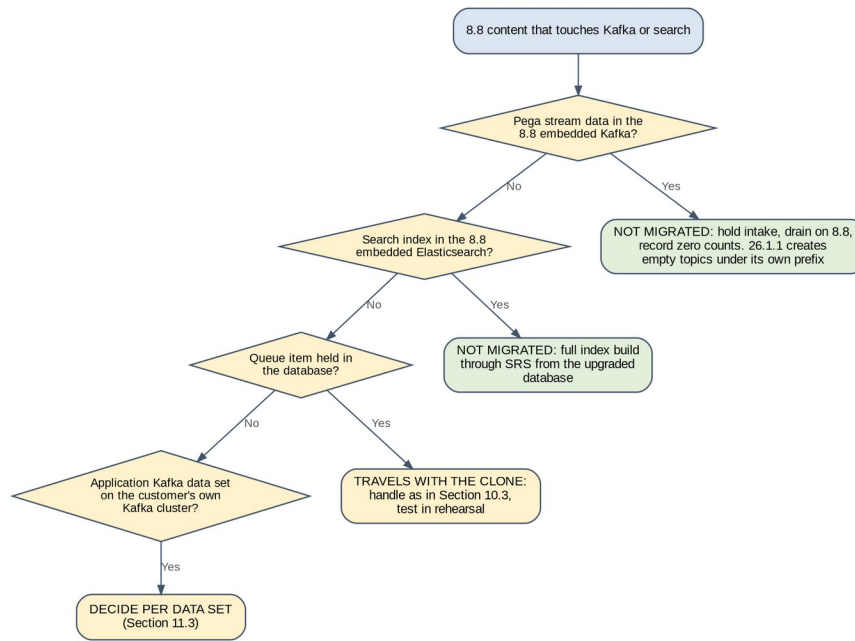


Figure 27. Decision: does 8.8 content need migrating?

Prepared for this implementation. Not a Pega-published diagram.

11.2 Treatment per content type

Table 44. Treatment of 8.8 content

Content	Answer	How it is handled	Evidence
Pega stream data in Kafka (queue processor messages, data flow partitions)	Not migrated	Pega states existing stream data cannot be moved to a new Kafka. 8.8 uses embedded Kafka, so there is no external cluster to drain to. Before the production clone, hold intake on 8.8, wait until queue processors show nothing ready to process, record the counts and any broken items, then stop 8.8. 26.1.1 starts with empty topics that it creates under <code>pega-prd-</code> .	[R13, R25]
Queue items held in the database (delayed, broken)	Travel with the clone	Database content, not Kafka content. Handled as items CD-05 and CD-06 (Section 10.3) and tested in rehearsal.	[R3]
Topic names and configuration	Not migrated	26.1.1 creates its own topics under its own prefix. Only the topic settings policy, such as <code>max.message.bytes</code> , carries over as configuration.	[R11, R23]
Search indexes (embedded Elasticsearch on 8.8)	Not migrated	The indexes live on the 8.8 nodes, not in the database. They are built from the upgraded database after Pega connects to SRS, which needs downtime; the time is measured in rehearsal (Section 7.8). Elasticsearch 8.x snapshots cannot be restored into OpenSearch.	[R15, R17, R38, R39]
Application Kafka data sets on the customer's own topics	Decided per data set	Keep the existing cluster and topic, point at a new cluster, or mirror with Confluent Cluster Linking, which keeps offsets. Agree the start offset so messages are neither skipped nor processed twice.	[R11, R35]

Content	Answer	How it is handled	Evidence
Custom search data (custom search properties, reports that depend on search)	Rebuilt and verified	Rebuilt by the full index build; verified with count comparisons and functional checks.	[R18, R19]

11.3 Application Kafka data sets

Application Kafka data sets read from or write to the customer's own Kafka clusters, which are separate from the Pega stream service [R11]. Each one is decided on its own, with the application owner (Figure 28). In every non-production environment, every data set is repointed to a test topic or disabled before the batch tier starts, so a clone never consumes from a production topic or moves a production consumer group's offsets.

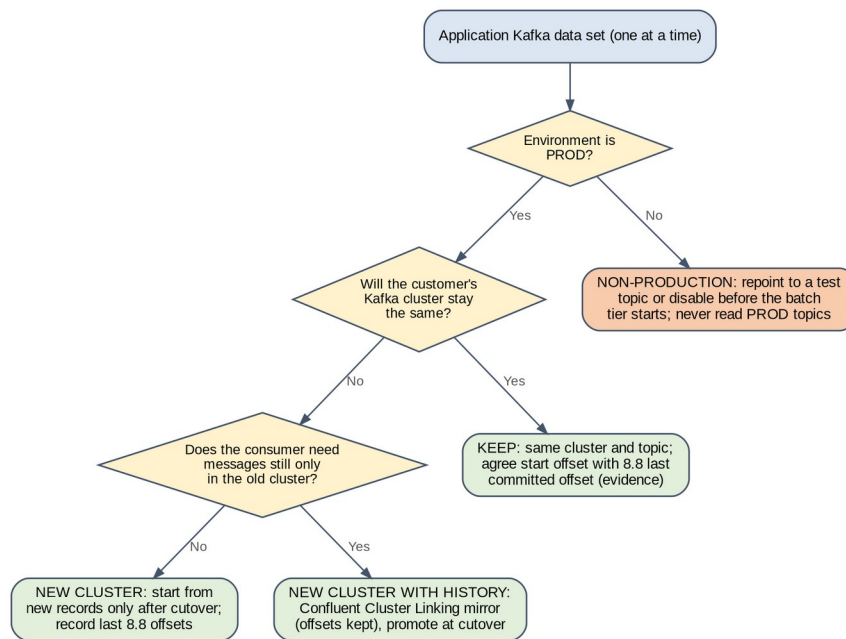


Figure 28. Decision: treatment of each application Kafka data set

Prepared for this implementation. Not a Pega-published diagram.

Table 45. Application Kafka data set register (one row per data set, completed with the application team)

Data set	Direction	Cluster and topic	PROD treatment	Start offset at cutover	Non-production treatment	Owner
<data set name>	<consume or produce>	<cluster / topic>	<keep, new cluster, Cluster Linking>	<offset rule and evidence>	<test topic or disabled>	<application owner>

11.4 Why the Stream Migration activity is not used

Pega provides a Stream Migration activity and a Helm chart guide for switching a running deployment from embedded Stream to an external Kafka, or between Kafka providers [R13, R25]. It drains the queues of a running system before the switch. This programme never switches the running 8.8 system to an external Kafka: 8.8 keeps its embedded Kafka until it stops, and 26.1.1 starts on a different system with its own Confluent cluster. Holding intake and letting the 8.8 queues empty gives the same result, an empty queue at the moment of the final clone, without changing 8.8.

11.5 Statement for approval

Decision. No Pega stream topic and no search index is copied from 8.8. Database-held queue items travel with the clone and are handled as listed in Table 42. Application Kafka topics are handled one by one as listed in Table 45.

12 Deployment runbook per environment

Every environment is built in the same order, with a check after each step. Shared services are created once per group (NP1, NP2, PROD); per-environment objects are created for each environment. Values in angle brackets come from Appendix A, and commands from Appendix C.

12.1 Build order

Table 46. Build order

Phase	Component	Scope	Depends on	Owner
1	Network and DNS: private endpoint subnet, firewall rules, private DNS zones	Per group and per environment	Landing zone	Cloud architect
2	Key Vault, External Secrets Operator identity, Okta authorization server and client	Per environment	Phase 1	Platform team, identity team
3	Confluent cluster and Private Link; service account, ACLs, quota, API key	Cluster per group; rest per environment	Phases 1, 2	Kafka engineer
4	OpenSearch service and private endpoint; cluster settings; SRS user and role	Service per group; user per environment	Phases 1, 2	Search engineer
5	SRS deployment (backingservices chart)	Per environment	Phases 2, 4	Search engineer
6	Database clone, masking, upgrade (run U)	Per environment	GQ answers or conservative plan	DBA team, platform team
7	Clean-up and first start (run F), index build (run D), intake release	Per environment	Phases 3, 5, 6	Pega LSA, platform team

12.2 Network and DNS

Table 47. Network and DNS steps

ID	Action	Owner	Check	Evidence
NW-1	Create the private endpoint subnet with network policies for private endpoints enabled.	Cloud architect	Subnet exists; NSG applies	Azure export
NW-2	For a new group, complete N-1 to N-6 (Section 6.4) for its Confluent cluster.	Kafka engineer	K-1 to K-3 pass from a pod	Command output
NW-3	For a new group, create the OpenSearch private endpoint and DNS record as the provider documents.	Search engineer	S-1 resolves to a private IP and returns the version	Command output
NW-4	Add firewall rules for this environment: its own Confluent and OpenSearch endpoints, <okta-domain>:443, registry and Pega Diagnostic Center if used. Deny production endpoints from non-production.	Cloud architect	Allowed tests pass; denied tests fail (IT-09)	Rule export, test output
NW-5	Confirm the AKS cluster has a network policy engine. Apply Kubernetes network policies: deny by default; allow pega to srs on 8443, srs to the OpenSearch endpoint and Okta, pods to DNS (Section 7.7).	Platform team	Test pod cannot reach a denied target	Test output

12.3 Secrets and identity

Table 48. Secrets and identity steps

ID	Action	Owner	Check	Evidence
SI-1	Create kv - pega - <code> with RBAC, private endpoint and public access disabled.	Platform team	Reachable from the spoke only	Test output

ID	Action	Owner	Check	Evidence
SI-2	Create the workload identity for the External Secrets Operator in <code>pega-<code><code></code></code> and <code>srs-<code><code></code></code> ; grant Key Vault Secrets User on this vault only.	Platform team	SecretStore Ready	kubecttl output (P-4)
SI-3	Create the Okta client <code>pega-srs-<code><code></code></code> and register its public key (Section 7.6); store the private key in Key Vault.	Identity team	Token request succeeds (O-1)	Decoded claims (O-2)
SI-4	Create the ExternalSecrets in Section 8.2.	Platform team	All secrets SecretSynced	kubecttl output

12.4 Confluent Cloud

Table 49. Confluent Cloud steps

ID	Action	Owner	Check	Evidence
CC-1	For a new group, create the cluster (type per OD-02) in the AKS region with Private Link.	Kafka engineer	Cluster status Up	Cluster ID
CC-2	Create service account <code>sa-pega-<code><code></code></code> and the four ACLs (Section 6.5).	Kafka engineer	ACL list matches Table 21 (K-7)	ACL export
CC-3	Create an API key for the service account; write the JAAS value to <code>kv-pega-<code><code></code></code> as <code>confluent-jaas</code> .	Kafka engineer	Secret version exists; key not in any file or ticket	Secret metadata
CC-4	In NP1 and NP2, create the client quota for the service account (OD-09).	Kafka engineer	Quota listed	Quota export
CC-5	Run IT-01 to IT-04 against the other environments in the group.	Kafka engineer	All denied	Test output

12.5 OpenSearch

Table 50. OpenSearch steps

ID	Action	Owner	Check	Evidence
OS-1	For a new group, provision the OpenSearch service (OD-03) at the size in Section 7.4, version 2.15 or 2.19.	Search engineer	S-1 shows the version; S-2 green	Output
OS-2	For a new group, apply the cluster settings in Section 7.4 as administrator.	Search engineer	S-3 shows both settings	Output
OS-3	Create the SRS user for this environment and map it to the index-scoped role <code>pega26-<code><code></code>-srs</code> (Section 7.5).	Search engineer	User can create and delete <code>pega26-<code><code></code>-test</code> ; cannot touch another prefix	Test output
OS-4	Store the SRS user credentials in <code>kv-pega-<code><code></code></code> .	Search engineer	Secret synced to <code>srs-<code><code></code></code>	kubecttl output

12.6 SRS

Table 51. SRS steps

ID	Action	Owner	Check	Evidence
SR-1	Create the SRS TLS keystore and truststore and store them in Key Vault.	Platform team	Secret <code>srs-runtime-certs</code> synced	kubecttl output
SR-2	Deploy SRS: <code>helm upgrade --install srs-<code><code></code> pega/backingservices -n srs-<code><code></code> -f backingservices-<code><code></code>.yaml</code> , then <code>kubecttl apply -f netpol-srs-<code><code></code>.yaml</code> (Section 7.7).	Search engineer	SRS pods Ready (P-1); SRS reaches OpenSearch and the Okta key set	Output
SR-3	From a pod in <code>pega-<code><code></code></code> , call the SRS service with and without a token.	Search engineer	Without token refused; with token accepted (S-5)	Output

12.7 Database clone and upgrade

Table 52. Database clone and upgrade steps

ID	Action	Owner	Check	Evidence
DB-1	For non-production, take the clone from the agreed production copy; for PROD, take the final clone at cutover step C-3.	DBA team	Clone restored; row counts recorded	Clone record
DB-2	Mask the clone (non-production) per OD-07.	DBA team	Masking report	Report
DB-3	Apply any pre-upgrade condition from Pega Support (GQ-02, GQ-03) and the time-point B items in Table 42.	DBA team, Pega LSA	Each item signed off	Clean-up record
DB-4	Run U: <code>helm upgrade --install pega pega/pega -n pega-<code><code></code> -f pega-<code><code></code>-upgrade.yaml --version 4.13.0.</code>	Platform team	Installer job completed (P-5)	Installer log, duration
DB-5	If the installer job fails, read the log, fix the cause, and follow Pega's guidance for rerunning the upgrade on this database. If in doubt, restore the clone and start again from DB-1 (FS-26).	Platform team, DBA	Rerun completes	Incident record

12.8 First start, index build and intake release

Follow the first-start control in Section 10.4 (steps ST-1 to ST-9). The Helm commands are:

Helm commands for runs F and D

```
# Run F: web tier only, batch tier held at zero
helm upgrade --install pega pega/pega -n pega-<code> -f pega-<code>-first.yaml \
--version 4.13.0
# Run D: steady state, batch tier at normal size (starts queue processing)
helm upgrade --install pega pega/pega -n pega-<code> -f pega-<code>-deploy.yaml \
--version 4.13.0
```

12.9 Per-environment notes

Table 53. Per-environment notes

Environment	Built from	Masked	Shared group	Notes
DEV	Production copy	Yes	NP1	First build. Proves the Okta claim design, the index-scoped role, DSS precedence and the full path; finds the cluster messaging topics.
SIT	Production copy (can be the DEV clone)	Yes	NP1	First environment with integrations to test systems; data set register applied.
UAT	Recent production copy	Yes	NP1	Business tests on search over masked data.
PERF	Full-size production copy	Yes	NP2	Load tests, partition and index build measurements; sets PROD sizes.
PREPROD	Most recent full-size production copy	Per policy	NP2	Rehearsal 1 and Rehearsal 2 of the production cutover, run by the cutover team.
PROD	Final clone at cutover	No	PRD	Section 13.

12.10 Building the environments one after another

The environments are built in waves, shown in Figure 29. Each wave reuses what the earlier waves proved and adds only what is new. A wave starts only when the exit check of the previous wave has passed, so a design fault is found once, in DEV, rather than six times.

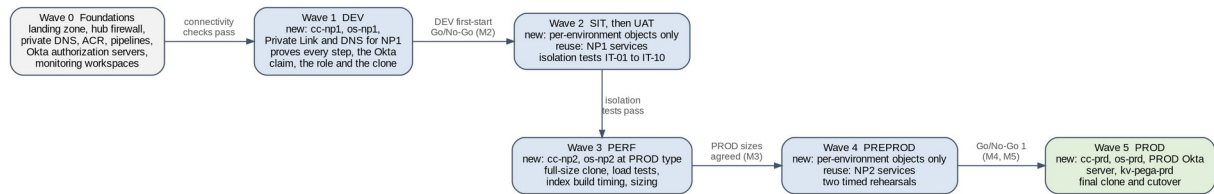


Figure 29. Environment build waves and the gate between each
Prepared for this implementation. Not a Pega-published diagram.

Table 54. Environment build waves

Wave	Environment	New shared objects	New per-environment objects	Exit check
0	Foundations	Landing zone, hub firewall rules, private DNS zones, registry, pipelines, both Okta authorization servers, monitoring workspaces	None	Connectivity checks from a test pod; pipeline renders all values files
1	DEV	cc-np1 with Private Link and DNS; os-np1 with cluster settings	All objects in phases 2 to 7 of the build order	First-start Go/No-Go; Okta claim, index-scoped role, DSS precedence (FS-24), GQ-08 behaviour and SRS key rotation (FS-33) recorded (M2)
2	SIT, then UAT	None	Namespace, vault, Okta client, service account, ACLs, quota, SRS user, SRS, clone	IT-01 to IT-10 between DEV, SIT and UAT
3	PERF	cc-np2 and os-np2 at the PROD type and size	Same as wave 2, plus load test harness	Load tests and sizing (Section 15); PROD sizes and cluster type agreed (M3)
4	PREPROD	None	Same as wave 2	Two timed rehearsals; isolation from PERF (M4, M5)
5	PROD	cc-prd, os-prd, PROD Okta authorization server, kv-pegaprd	Same as wave 2, from the final clone	Go/No-Go gates in Section 13.4

Table 55 lists what must be configured for each new environment, whichever wave it is in. Keep one completed copy per environment with the evidence.

Table 55. Configuration checklist for each new environment

Item	Value for this environment	Where configured	Check
Code, group, namespaces	<code>, NP1 / NP2 / PRD, pega - <code>, srs - <code>	Table 8	Code is not the start of another code
Key Vault and workload identity	kv-pegaprd - <code>	SI-1, SI-2	SecretStore Ready
Okta client and key	pega-srs-<code>; guid = pega26-<code>	SI-3, Section 7.6	O-2 shows the right guid
Confluent service account, ACLs, quota, API key	sa-pegaprd - <code> on the group cluster	CC-2 to CC-4	K-7 ACL list; quota listed
Stream values	streamNamePattern: pega - <code> - {stream.name}, group bootstrap	Values file	Pipeline naming check
OpenSearch user and role	pega26-<code>-srs on the group service	OS-3, OS-4	IT-06
SRS release and network policy	srs-<code>, URL with : 8443	SR-1 to SR-3	S-5
customerDeploymentId	pega26-<code>	Values file	Pipeline naming check; IT-05
Firewall and DNS	Own Confluent and OpenSearch endpoints, Okta, deny production	NW-4, NW-5	IT-09
Clone, masking, upgrade	Clone date and masking report	DB-1 to DB-5	Installer log

Item	Value for this environment	Where configured	Check
Clean-up CD-01 to CD-16	Clean-up record	Section 10.3	Signed record
Monitoring	Dashboards and alerts with the environment label	Section 18	Test alert received
Measurements	Partitions, index size, build time, token rate, connections	Appendix A	Values recorded

13 Production cutover and rollback

13.1 Principles

- The 8.8 system and its database are not changed by the cutover, apart from stopping 8.8. They stay available for rollback until the rollback window closes.
- Every step was run at least twice in PREPROD with production-sized data, by the people who will run it in production.
- The outage window comes from the rehearsal measurements plus 25 % contingency, not from estimates.
- Rollback is free of data loss until intake is released on 26.1.1. After that, rollback loses work done on 26.1.1, so the business decides.

13.2 Milestones

Table 56. Programme milestones used in this document

Milestone	Content	Exit criteria
M1 Design approval	This document approved; open decisions owned	Signatures on the approvals page
M2 DEV build	First environment from a clone	First-start Go/No-Go passed; Okta, role and DSS tests recorded
M3 PERF measurements	Load tests, partitions, index build time	PROD sizes and cluster type agreed (OD-02)
M4 Rehearsal 1 (PREPROD)	Full cutover sequence with timings	All steps completed; defects logged
M5 Rehearsal 2 (PREPROD)	Full cutover sequence by the cutover team	Timing fits the window; no open severity 1 or 2 defects
M6 Go/No-Go 1 and production cutover	Section 13.3	Go/No-Go 3 passed

13.3 Cutover sequence

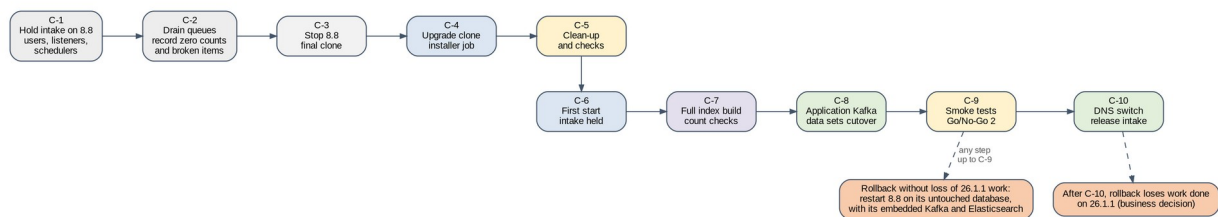


Figure 30. Production cutover sequence from the Kafka and search point of view

Prepared for this implementation. Not a Pega-published diagram.

Table 57. Production cutover steps

Step	Action	Owner	Duration from	Evidence
C-1	Hold intake on 8.8: block user access, stop inbound listeners and services, disable job schedulers that create work.	Application team	Rehearsal	Change record
C-2	Let queue processors finish. Record the count of items ready to process (must be zero) and the count of broken items per queue processor from Admin Studio. Record the last committed offset of each application Kafka data set consumer.	Pega LSA	Rehearsal	Screenshots, offset list
C-3	Stop 8.8. Take the final clone (DB-1).	DBA team	Rehearsal	Clone record

Step	Action	Owner	Duration from	Evidence
C-4	Run U on the clone (DB-3, DB-4).	Platform team	Rehearsal	Installer log
C-5	Apply time-point A clean-up; run F; time-point F checks (ST-2 to ST-6).	Pega LSA	Rehearsal	Clean-up record, screenshots
C-6	Go/No-Go at first start (Figure 26).	Cutover manager	Fixed	Gate record
C-7	Run D; full index build; completeness and count checks (ST-7, ST-8).	Pega LSA	Rehearsal	Count sheet
C-8	Application Kafka data sets cutover (Section 13.5).	Application team	Rehearsal	Offset evidence
C-9	Smoke tests; Go/No-Go 2.	Test lead	Rehearsal	Test report, gate record
C-10	Switch public DNS to Application Gateway; release intake (ST-9); Go/No-Go 3 after 30 minutes.	Platform team	DNS TTL	DNS change record

Important. Lower the TTL of the public DNS record to 300 seconds at least 48 hours before cutover, so a DNS switch or revert takes effect quickly. Restore the normal TTL after hypercare.

13.4 Go/No-Go gates

Table 58. Go/No-Go gates

Gate	When	Criteria	Decision owner
Go/No-Go 1	Seven days before	Two PREPROD rehearsals passed; window fits with 25 % contingency; Pega Support answers received or conservative plan accepted; rollback rehearsed	Business owner with platform lead
First-start gate	After C-5	All checks in Figure 26 up to the index build	Cutover manager
Go/No-Go 2	After C-9	Index build complete and counts match; application data sets started at agreed offsets; smoke tests pass	Cutover manager
Go/No-Go 3	30 minutes after C-10	No severity 1 or 2 incidents; queue backlog falling; search response within target	Cutover manager with business owner

13.5 Application Kafka data set cutover

1. At C-2, record the last committed offset of each 8.8 consumer group that reads the customer's Kafka topics (K-8 on the customer's cluster).
2. For data sets that keep the same cluster and topic, configure the 26.1.1 data flow to start from the recorded offset, or reuse the same consumer group if the application team confirms it, so no message is skipped or processed twice.
3. For data sets that move to a new cluster without history, start from new records only, and keep the 8.8 offset list as evidence of what was processed.
4. For data sets mirrored with Cluster Linking, promote the mirror topics at cutover; Cluster Linking keeps offsets [R35].
5. Record the first offset processed by 26.1.1 for each data set and compare with step 1.

13.6 Rollback

Table 59. Rollback steps

Step	Action
RB-1	Declare rollback; record the reason and the time.
RB-2	Hold intake on 26.1.1. If intake was released, export the list of cases created or changed on 26.1.1 for reconciliation.
RB-3	Revert public DNS to 8.8.
RB-4	Start 8.8 on its untouched database. Its embedded Kafka and embedded Elasticsearch were stopped, not changed.
RB-5	Check the 8.8 stream and search landing pages and queue processors; release 8.8 intake.
RB-6	Reset application Kafka data set consumers on 8.8 to the offsets agreed for the rollback.
RB-7	Scale 26.1.1 to zero; keep it and its Kafka topics and indexes for analysis.

The point after which rollback loses work is C-10, when intake is released on 26.1.1. Before C-10, rollback loses nothing. After C-10, cases created or changed on 26.1.1 are not in the 8.8 database, and messages produced to application Kafka topics by 26.1.1 have been consumed downstream. The business owner decides at that point between rollback with reconciliation and fixing forward, using the reconciliation list from RB-2.

14 Test strategy, rehearsals and failure scenarios

14.1 Test stages

Table 60. Test stages

Stage	Entry criteria	Environment	Owner	Evidence
Connectivity	Network, DNS and secrets steps done	Every environment	Platform team	K-1 to K-3, S-1 to S-3, O-1 outputs
Configuration	Run F complete	Every environment	Pega LSA	Landing page screenshots; inventory sign-off
Functional	Run D and index build complete	DEV, SIT, UAT	Test lead	Checks in Section 14.4
Isolation	Two environments in a group running	NP1, NP2	Security architect	IT-01 to IT-10 records
Performance	PERF built at full size	PERF	Performance lead	PT-01 to PT-04 reports
Resilience and failure	Performance baseline recorded	PERF, PREPROD	Platform team	Failure catalogue records (Section 14.6)
Security	Isolation tests passed	SIT, PREPROD	Security architect	No public endpoints; secret scan of values; token claim checks; TLS versions
Operational acceptance	Monitoring live	PREPROD	Operations lead	Alerts reach on-call; runbooks used by operations staff
Full rehearsal	All above passed	PREPROD (twice)	Cutover manager	Timed cutover record (Appendix D)

14.2 Rehearsals and measurements

Rehearse the full clone-and-upgrade path at least twice in PREPROD on production-sized data before production. Record each measurement against data volume, because they set the production window.

Table 61. Rehearsal measurements

Measurement	Steps	Used for	Rehearsal 1	Rehearsal 2
Drain time on 8.8	C-1 to C-2	Window		
Clone time	C-3	Window		
Upgrade time	C-4	Window		
Clean-up time	C-5	Window		
First start time	ST-3 to ST-6	Window		
Full index build time	ST-7 to ST-8	Window; OpenSearch sizing		
Time to release intake	C-9 to C-10	Window		
Partitions created	K-5	Confluent capacity		
Searchable data volume	S-4	OpenSearch sizing		

14.3 Isolation tests

Each test uses environment A's credentials against environment B in the same group, and must be refused. Run them in NP1 and NP2 after every new environment joins a group.

Table 62. Isolation tests

ID	Test	Method	Pass criteria
IT-01	Read and write B's topics with A's key	kcat produce and consume to <code>pega--<topic></code> with A's credentials	Authorization error
IT-02	Create and delete topics under B's prefix	<code>confluent kafka topic create</code> and <code>delete</code> as A	Refused
IT-03	Join or reset B's consumer group	kcat consumer with group <code>pega-- . . .</code>	Authorization error
IT-04	Use A's key from B's namespace	Check B's Key Vault and secrets contain no A key	Not present
IT-05	Index names after the build	S-4 for <code>pega26-<A>*</code> ; list all indexes	A's indexes all carry A's prefix; none outside a known prefix
IT-06	A's SRS user on B's indexes; SRS works with the scoped role	Search, write and delete on <code>pega26-*</code> as A's user; full build as A	Refused on B; A's build succeeds
IT-07	A's token at B's SRS	Send a request to B's SRS with A's token	Refused (guid mismatch)
IT-08	A's External Secrets identity on B's vault	Create a test ExternalSecret in A pointing at B's vault	Access denied
IT-09	Non-production pod to production endpoints	From a pod in NP1 and NP2: DNS and TCP to cc-prd, os-prd and production integration hosts	No private resolution; connection refused by firewall
IT-10	Cloned environment to production integrations	Firewall log review during run F and the first day of run D	No connection attempts to production systems

14.4 Functional and integration checks

- Queue a test item to a test queue processor and confirm it is processed; repeat after every deployment.
- Run a job scheduler with a test activity and confirm it runs on the batch tier.
- Create a case, wait for indexing, and find it with case search; repeat for each class with custom search properties (CD-14).
- Run each report that depends on search and compare with the 8.8 baseline on the same clone data.
- For each application Kafka data set in SIT, run its data flow against the test topic and compare counts.
- Check the produce path with a message near 5,000,000 bytes, if the application produces large messages (FS-06).

14.5 Performance tests

These are the acceptance tests for Kafka and search. Section 15 gives the workload model, the full set of test types, the harness and the entry and exit criteria they run under.

Table 63. Performance tests

ID	Test	Target	Measured
PT-01	Peak user load plus peak queue processor load in PERF	Agreed response times; consumer lag returns to baseline after peak	
PT-02	Full index build throughput	Fits the production window after scaling (Section 7.8)	
PT-03	Quota enforcement: PERF load while PREPROD runs a light test	PREPROD response unchanged; PERF throttled at its quota	
PT-04	Consumer lag under peak per queue processor	Lag within the alert threshold	

14.6 Failure scenario catalogue

Table 64 lists the failures to test. Run Kafka, search and platform scenarios in PERF or PREPROD, and clone-specific scenarios in DEV during the first build and again in PREPROD. Where Pega does not document the exact behaviour, record what happens in the first run; that record becomes the expected result for later runs.

Table 64. Failure scenario catalogue

ID	Scenario	How to cause it safely	Expected behaviour	Detection	Recovery	Pass criteria
FS-01	Kafka bootstrap unreachable	Block 9092 to the Confluent endpoints with a network policy	Queue processing and cluster messaging stop; Stream landing page not NORMAL	Stream landing page; Pega log Kafka connection errors; alert	Remove the block; pods reconnect or restart	Processing resumes; no item lost
FS-02	Zonal private DNS record missing	Remove one zonal record in DEV	Some broker connections fail after bootstrap	K-3 shows a broker resolving publicly or not at all	Restore the record	All broker names resolve privately
FS-03	API key revoked	Delete the environment's API key in DEV	Authentication failures; stream down	Pega log SASL errors; Confluent audit log	Create a key; update Key Vault; restart tiers	Stream NORMAL within the rotation time
FS-04	ACL missing for a new topic	Remove the TOPIC prefixed ACL briefly in DEV	Topic creation or access refused	Authorization errors in Pega log	Restore the ACL; add literal ACLs for unprefix topics	All topics accessible
FS-05	Partition limit reached	Lower the cluster headroom in a sandbox or simulate with a test topic set	New topic creation fails	Confluent error; Pega log	Delete unused topics or add capacity	New queue processor starts
FS-06	Message larger than max.message.bytes	Produce a 6 MB test message through a test queue processor	Producer error for that item	RecordTooLargeException in Pega log	Set topic to 5,000,000; keep messages within it [R11]	Item within limit processed
FS-07	Broker rolling restart during Confluent maintenance	Observe a scheduled maintenance in PERF	Short retries; no loss	Confluent notice; brief lag rise	None	No broken items caused
FS-08	Consumer lag builds under load	PT-04 at 150 % of peak	Lag grows then drains	Lag alert	Add batch pods within partition count	Lag back to baseline
FS-09	One SRS pod lost	Delete one SRS pod in PERF	Searches continue on other pods	Pod restart event	Kubernetes recreates it	No search errors seen by users
FS-10	Whole SRS deployment lost	Scale SRS to zero in PERF	Search and indexing fail; other functions continue	Search errors; SRS alert	Scale back; check indexing backlog clears	Backlog cleared; counts match
FS-11	Okta token cannot be obtained	Block <code><okta-domain></code> at the firewall for the environment	Search fails once the current token expires	Pega log token errors	Restore the rule	Search resumes without restart, or with restart recorded
FS-12	Token from the wrong issuer	Configure a test client on another authorization server	SRS refuses if it checks issuer; record result	SRS log	Use the right server	Behaviour recorded in Section 20.4
FS-13	<code>guid</code> does not match <code>customerDeploymentId</code>	Change the claim value for the DEV client	SRS refuses requests	401 or 403 in SRS log	Restore the claim	Search resumes
FS-14	Client key rotated in Okta but not in Key Vault	Remove the current public key from the DEV client	Token requests fail	Okta system log; Pega log	Register the key or update Key Vault and restart	Search resumes
FS-15	OpenSearch status yellow	Stop one data node (provider tooling) in PERF	Searches continue	S-2 yellow; alert	Provider restores the node	Green; no Pega errors
FS-16	OpenSearch status red	Simulate in a sandbox service by closing an index copy	Searches on the affected index fail	S-2 red; alert	Restore or rebuild the index	Green; counts match
FS-17	Flood-stage disk watermark	Fill test data on a sandbox service	Indexes read-only; indexing fails	Disk alert; SRS errors	Add storage; clear the read-only block as the provider documents	Indexing resumes

ID	Scenario	How to cause it safely	Expected behaviour	Detection	Recovery	Pass criteria
FS-18	OpenSearch credentials rotated without SRS update	Change the SRS user password only	SRS calls refused	SRS log 401	Update Key Vault; restart SRS	Search resumes
FS-19	Certificate expires	Deploy an expired SRS server certificate in DEV	Pega to SRS TLS fails	TLS errors in Pega log; expiry alert should have fired earlier	Renew; restart	Expiry alert fires 30 days before
FS-20	Key Vault secret rotated without pod restart	Rotate the API key, delete the old key, skip the restart	Stream fails when the old key is deleted	SASL errors	Rolling restart	Runbook includes the restart
FS-21	AKS node drained or zone lost	Cordon and drain all nodes in one zone	Pods reschedule within PDB limits	Pod events	Kubernetes reschedules	Processing and search continue
FS-22	Non-production load test saturates the shared cluster	PERF load above its quota while PREPROD tests	PERF throttled; PREPROD unaffected	Metrics API by principal	Lower quota; reschedule	PREPROD response unchanged
FS-23	Index rebuild during business hours on a shared service	Full build in SIT while UAT tests search	UAT search slower	Latency metrics	Schedule builds out of hours	UAT latency within target
FS-24	Stale 8.8 stream or search DSS overrides Helm	In DEV, record cloned DSS; start with them in place	Landing pages show Helm values (expected) or DSS values (defect)	Stream and search landing pages	Follow Pega Support's answer to GQ-05	Helm values in effect
FS-25	Wrong customerDeploymentId deployed	Deploy SIT with DEV's ID in a sandbox	SRS refuses tokens; if accepted, indexes mix	Pipeline check should block it; SRS log	Redeploy with the right ID; delete wrong indexes	Pipeline check blocks the deploy
FS-26	Installer upgrade fails halfway	Stop the installer job during a DEV rehearsal	Database left partly upgraded	Job status; installer log	Follow Pega's rerun guidance, or restore the clone and restart	Second run completes
FS-27	Clone starts with application Kafka data sets pointing to production	Leave one data set unchanged in DEV, with the firewall rule in place	Connection denied by the firewall	Firewall log; data set errors	Repoint or disable the data set	No production connection; IT-10 passes
FS-28	Delayed or broken items from 8.8 processed unexpectedly	Keep a sample of items in a DEV clone	Items run on 26.1.1 when due or queued	Queue processor statistics; application log	Apply the decision per queue processor (CD-05, CD-06)	Behaviour matches the decision
FS-29	Index build interrupted	Restart the batch tier during the build in DEV	Build stops or continues	Search landing page	Resume or restart the build as Pega documents [R17]	Complete build; counts match
FS-30	All batch pods lost	Scale the batch tier to zero in PERF under load	Queue processing stops; items accumulate in topics	Queue backlog alert	Scale back	Backlog drains; no loss
FS-31	Kafka connection storm	Rolling restart of the web and batch tiers of PERF and PREPROD at the same time	Connection attempts rise; Confluent may throttle [R31]	Metrics API throttle metric by principal [R56]	Stagger restarts across environments; keep the chart's <code>maxSurge: 1</code> [R23]	No throttling with the staggered restart procedure
FS-32	Partition creation paced by the cluster	Refresh PERF (delete and recreate all topics) while PREPROD restarts	Topic creation slows for both environments [R31]	Pega log topic creation retries; time to Stream NORMAL	Never refresh two environments in a group at once	Time recorded; refresh procedure updated
FS-33	Okta signing key rotation	Rotate the DEV authorization server's keys manually in Okta [R53]	SRS picks up the new key from the key set URL	SRS 401 errors if it caches the old key	Restart SRS; set rotation to manual if needed	Search continues without an SRS restart
FS-34	Client key rotation with two keys	Register a second public key on the DEV client; switch Key Vault to the new private key; restart	Okta accepts the assertion signed with either key	Okta System Log <code>invalid_client</code> errors	Keep the old key until the new one works	Token issued with the new key; old key removed
FS-35	Okta rate limit reached	In a sandbox org or with Okta's agreement, drive token requests above the token endpoint limit	HTTP 429 to Pega; search fails once the token expires [R52]	Okta System Log rate-limit warning and violation events	Find the source; lengthen token lifetime; raise the limit with Okta	Warning alert fires before any violation

14.7 Exact checks

Table 65. Exact checks

Check	Command or screen	Expected result
Stream service	Dev Studio: Configure > Decisioning > Infrastructure > Services > Stream	Provider ExternalKafka; status NORMAL; bootstrap and prefix from Helm [R13]
Queue processors	Admin Studio > Queue processors	Ready to process falls to zero; broken items known
Search	Search landing page	SRS connected; all classes indexed; no CONFLICTS FOUND [R19]
Kafka DNS, TLS and metadata	K-1, K-2, K-3	Private IPs; trusted chain; all brokers listed
Topics, partitions, configuration, ACLs	K-4 to K-7	Topics under <code>pega-<code><code></code></code> - only; max.message.bytes 5,000,000; ACLs as Table 21
OpenSearch health, settings, indexes	S-2, S-3, S-4	Green; both settings; indexes under <code>pega26-<code><code></code></code> only
SRS	S-5	Token required; healthy response
Okta token	O-1, O-2	<code>scp</code> includes <code>pega.search:full;guid</code> equals <code>pega26-<code><code></code></code>
Installer job	P-5	Completed without errors
Pods and secrets	P-1, P-4	All Ready; all ExternalSecrets SecretSynced

14.8 Acceptance criteria

Table 66. Acceptance criteria

ID	Criterion
AC-1	All exact checks pass in every environment, with evidence filed.
AC-2	All isolation tests pass in NP1 and NP2.
AC-3	All failure scenarios have been run at least once with results recorded; any defect is fixed or accepted by the business owner.
AC-4	Performance results meet the targets agreed by the business owner.
AC-5	Two PREPROD rehearsals completed, with the measured window within the agreed outage.
AC-6	Monitoring and alerts in Section 18.5 are live and tested.

15 Performance engineering and sizing

15.1 Why performance testing is required

Load figures from Pega 8.8 cannot be carried over to 26.1.1 on AKS. Too much changes at once, and each change moves load to a service that 8.8 never used:

- Every queue item, cluster message and index update now crosses the network to Confluent Cloud or to SRS and OpenSearch, through private endpoints. On 8.8 these stayed inside the Pega nodes.
- Queue processing in '26 is partitioned and multi-threaded [R3], so throughput depends on partitions, threads and batch pods together (Section 6.7).
- From '25, cluster messaging that Hazelcast carried uses Kafka [R7]. Kafka load rises with the number of Pega pods, not only with business volume.
- Confluent enforces connection and request limits on Enterprise clusters [R31]. A design that works at average load can be throttled at peak or during a rolling restart.
- Non-production environments share clusters and search services. Contention between them only shows up under load.
- The full index build sets part of the production outage [R15], and its time depends on resources that only a full-size test can show.
- Pega asks clients to start from its default search sizing and adjust it after measuring [R14].

Performance testing is therefore a gate, not a formality. PROD sizes are signed off only from PERF results (milestone M3), and PREPROD rehearsals confirm them.

15.2 Workload model

The workload model describes production load in terms the tests can reproduce. Build it from 8.8 production data (AS-10), not from estimates, and get the business owner to approve it before the first test.

Table 67. Workload model inputs

Input	Source on 8.8	Used for
Peak concurrent users and sessions per hour, by user journey	Web server or gateway logs; Pega Diagnostic Center; Pega usage reports	Web tier load scripts
API calls per hour by service	Gateway logs; Pega service rule statistics	API load scripts
Cases created and resolved per hour, by case type	Pega reports on the work tables	Data growth; indexing rate
Queue items per hour per queue processor, and the busiest queue processors	Admin Studio queue processor statistics	Kafka produce and consume rates; partitions
Job scheduler runs and their duration	Admin Studio	Batch tier load windows
Search queries and search-based reports per hour	Pega logs; report statistics	SRS and OpenSearch query rate
Calendar peaks: month end, campaigns, seasonal	Business owner	Peak-hour definition
Data volume and yearly growth	Database sizes; DEV index sizes (S-4)	Storage sizing

Define one design peak hour that combines the busiest user load with the busiest background load seen together on 8.8, plus the agreed growth. All targets in this section refer to that hour.

15.3 Test types and measurement points

Table 68. Performance test types

Test	Purpose	Profile	Pass criteria
Baseline	Reference figures with fixed replica counts	20 % of design peak, HPA off, 1 hour	Recorded; used to compare every later run
Load	Prove the design peak hour	Ramp to 100 % over 30 minutes; hold 1 hour	Response times and lag within targets; no throttling
Stress	Find the first limit and how the system fails	Step up by 25 % each 20 minutes until a target fails	Breaking point at least 150 % of peak; the first limit is known
Soak	Find leaks and slow drift	80 % of peak for at least 8 hours, across a token expiry and a key rotation if planned	No heap growth trend; stable lag; no rising error rate
Spike	Prove HPA and Kafka absorb a sudden rise	From 30 % to 120 % within 2 minutes	Recovery to target within 10 minutes
Backlog drain	Prove the batch tier clears a backlog	Queue one peak hour of items with the batch tier at zero, then start it	Drained within the agreed time; no broken items
Index build	Measure the outage component	Full build on the full-size clone (PT-02)	Fits the window after scaling (Section 7.8)
Resilience under load	Prove zone and component loss at peak	FS-09, FS-15, FS-21 and FS-31 run during a load test	Targets met within the recovery time
Noisy neighbour	Prove quotas protect the shared group	PT-03	PREPROD response unchanged

Figure 31 shows the test set-up and the six measurement points. Every test records all six, so a slow response at M1 can be traced to the layer that caused it.

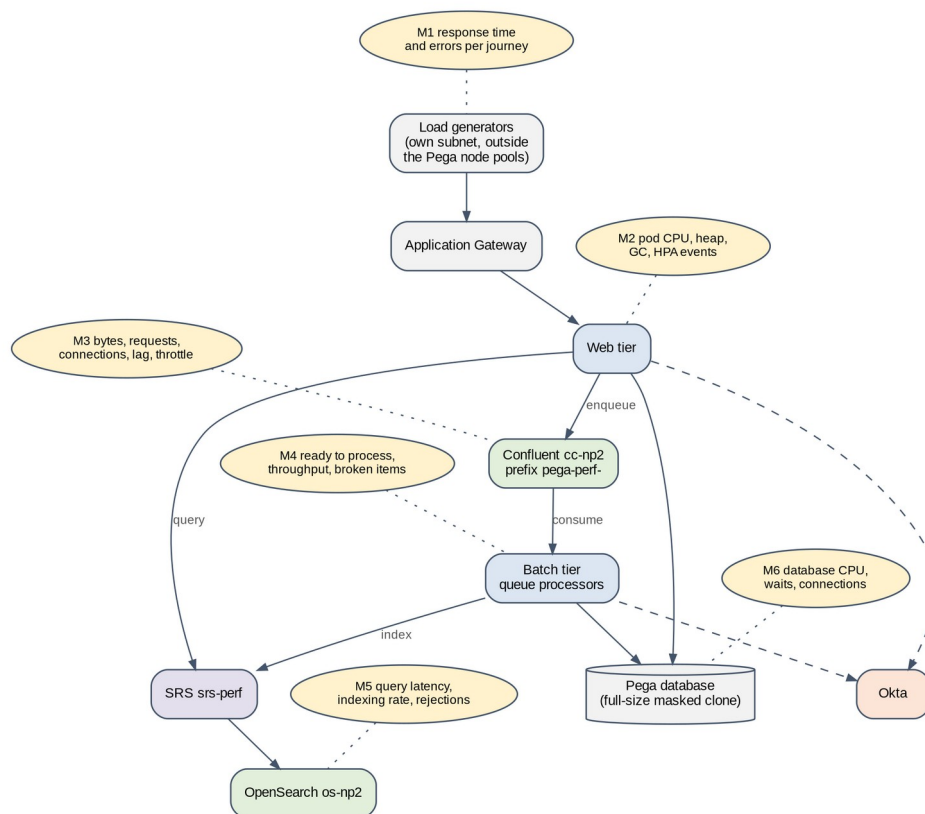


Figure 31. Performance test harness and measurement points in PERF

Prepared for this implementation. Not a Pega-published diagram.

Table 69. Measurement points

Point	What is measured	Source
M1	Response time (median, 95th and 99th percentile) and errors per journey	Load tool
M2	Pod CPU, memory, heap after GC, GC pause time, HPA events, pod restarts	Managed Prometheus; GC log (Section 9.2)
M3	Bytes in and out, requests, connections, consumer lag and throttling per service account	Confluent Metrics API [R56]
M4	Ready to process, throughput and broken items per queue processor	Admin Studio
M5	SRS response time and errors; OpenSearch query latency, indexing rate, rejected requests	SRS logs; provider metrics
M6	Database CPU, waits, connections and slow statements	Database monitoring

15.4 Tools and harness

- Use the customer's standard load tool, for example Apache JMeter or Gatling. Run the generators on their own VMs or node pool in a separate subnet, never on the Pega node pools, and enter through Application Gateway so the path matches production.
- Record scripts against 26.1.1, not 8.8. Pega UI requests carry session-specific values that must be correlated in the script; a script recorded on 8.8 will fail or produce false errors.
- Drive Kafka and search through Pega. Direct load on Confluent or OpenSearch does not show how Pega uses them. A short `kafka-producer-perf-test` run from a Pega namespace pod is useful only as a baseline for the Private Link path.
- Use the full-size masked clone. Take a database snapshot before the first test and restore it between test cycles, then refresh topics and indexes as in Section 19.3, so each cycle starts from the same state.
- Synchronize all clocks to UTC and label each run with a run ID in the load tool, in Grafana annotations and in the test log, so measurements from every source can be lined up.
- Agree each test with the identity team. All environments share one Okta org, and load tests add token requests to the org's rate limit [\[R52\]](#).

15.5 Entry and exit criteria

Table 70. Performance test entry, exit and suspension criteria

Criteria	Detail
Entry	PERF built at full size with the PROD cluster type and OpenSearch size; full index build complete. Workload model and targets approved by the business owner. Dashboards for M1 to M6 live (Section 18.4); synthetic checks green. PREPROD activity scheduled away from the test window (Section 5.4).
Exit	Load, soak and spike tests pass at the design peak; stress test shows at least 150 % headroom. No Confluent throttling at peak; consumer lag returns to baseline within the agreed time. Index build time fits the window with the PROD sizes. Sizing calculator updated with measured values; PROD sizes and cluster type signed off (M3).
Suspend	A shared-service incident, another environment's activity, or a defect that invalidates results. Record and rerun.

15.6 Pitfalls seen in Pega performance tests

- **Empty or partial search indexes.** Queries on a small index are fast and say nothing about production. Test only after the full build.

- **HPA hiding the limit.** Run the baseline and the stress test with fixed replicas, then repeat the load test with HPA on.
- **Warm-up included in results.** Exclude the first 15 minutes after a start, while caches fill and the JIT compiles.
- **Fewer partitions than production.** If PERF topics have fewer partitions than PROD will, batch results are pessimistic; if more, they are optimistic. Keep the partition DSS the same in PERF and PROD (CD-15).
- **Web-only scripts.** User journeys that create cases also create queue items and index updates. Measure M4 and M5 in every run, not only M1.
- **Repeated search terms.** OpenSearch caches results. Use a large, varied set of search values from the masked data.
- **Comparing with 8.8 figures measured differently.** Compare like with like, or use 8.8 figures only as a guide.
- **Connections not measured.** Record connections and connection attempts per pod (M3) at rest, at peak and during a restart, because Confluent limits them per cluster [R31].

15.7 Sizing method

Sizing runs as a loop, shown in Figure 32. Production facts from 8.8 and measurements from DEV go into the sizing calculator (Appendix H). PERF tests the result at production volume. If targets are not met with headroom, the inputs or sizes are adjusted and PERF runs again. After go-live, production monitoring feeds actual usage back into the calculator at each quarterly capacity review.

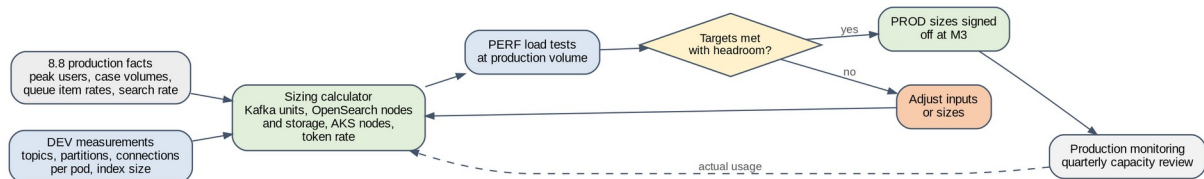


Figure 32. Sizing loop from production facts to signed-off PROD sizes

Prepared for this implementation. Not a Pega-published diagram.

15.8 Sizing rules per layer

Table 71. Sizing rules per layer

Layer	Rule	Source
Pega web tier	Pods at peak = peak sessions / sessions per pod measured in PERF at target response time. HPA maximum = pods at peak x 1.25	Measured; [R23]
Pega batch tier	Effective consumers per queue processor = min(partitions, batch pods x threads). Size pods for the busiest queue processors and for the index build, whichever needs more	[R49, R50]
Pega pod memory	Container 12Gi with heap about two thirds of it; metaspace capped at 768m	[R23, R63]
Confluent capacity units	Units = the largest of: ingress / 60 MBps, egress / 180 MBps, partitions / 3,000 (4,500 on Dedicated), connections / 18,000, requests per second / 7,500 (15,000), and 2 for the SLA	[R31]
Confluent partitions	Topics x partitions per topic (default 6), plus 30 % headroom, summed over the group	[R49]
OpenSearch storage and nodes	Formulas in Table 28	[R54, R55]
SRS	3 pods in PERF, PREPROD and PROD; add pods if SRS CPU stays above 70 % at peak	[R14, R24]
Okta	Token requests per minute summed over all environments, compared with the org's token endpoint limit	[R52]

Layer	Rule	Source
AKS nodes	Pega pods per node = the smaller of allocatable CPU / CPU request and allocatable memory / memory request. Nodes = (pods at HPA maximum / pods per node) x 1.5 for zone loss, plus 1 for upgrade surge, rounded up to a multiple of 3	[R59]

15.9 Worked example

Caution. The figures below are illustrative inputs chosen to show the method. They are not measurements of the customer's system. Replace every input with a measured value before sizing anything.

Table 72. Worked sizing example (illustrative inputs)

Layer	Inputs (illustrative)	Result
Confluent cc-prd	8 MBps in, 24 MBps out, 900 partitions measured, 1,800 connections, 3,000 requests per second	Units needed by ingress 1, egress 1, partitions 1, connections 1, requests 1, SLA minimum 2. Result: 2 eCKU, set by the SLA minimum
OpenSearch os-prd	40 GiB primary data, 1 replica, 20 % growth a year for 3 years, 60 indexes with 1 primary shard each, 100 GiB usable disk and 8 GiB heap per node	Storage 200.4 GiB; nodes by storage 3, by shards 1, zone minimum 3. Result: 3 data nodes
AKS Pega node pool	12 Pega pods at HPA maximum, 3 CPU and 12 GiB each; nodes with 15 CPU and 58 GiB allocatable	4 pods per node; 3 nodes for the load; x 1.5 and + 1 gives 6 nodes

In this example the Confluent cluster is sized by the SLA minimum, not by load, and the OpenSearch service by the three-zone minimum. That is common for Pega workloads of this kind. The shared clusters change the picture: in NP1, three environments' partitions add up, and partitions or connections can set the unit count instead. Run the calculator for each group, not only for PROD.

16 Known issues and challenges

These are the problems that come up in this kind of implementation. Each one names the symptom, the cause, how to prevent it and how to fix it.

16.1 Cloned database

Table 73. Known issues: cloned database

Symptom	Cause	Prevention	Fix
Landing pages show 8.8 values instead of the Helm values	Stream or search DSS from 8.8 in the cloned database (CD-01, CD-03)	Record cloned DSS before the first start; FS-24 in DEV; GQ-05	Apply Pega Support's answer; restart the tiers
A production Kafka consumer group's offsets move while 8.8 is still live	A non-production clone started with its application Kafka data sets still pointing at production (CD-09)	Batch tier at zero in run F; data sets repointed before run D; firewall denies production clusters	Reset the production consumer group with the Kafka owner; repoint the data set
Stream landing page lists unavailable nodes	8.8 embedded stream node records in the clone (CD-02)	Check at ST-5	Remove stale entries on the landing page
Queue items from 8.8 fail on 26.1.1	Rules referenced by the item changed in the upgrade (CD-05, CD-06)	Drain fully on 8.8; resolve broken items before the clone; decide per queue processor	Fix and requeue, or discard with business approval
Index build longer than planned	Production-sized data; too few batch pods or data nodes	Measure in PERF and PREPROD; scale for the build (Section 7.8)	Add batch pods and data nodes; extend the window through the Go/No-Go
Emails or extracts sent from a test environment	Job schedulers and connectors in the clone (CD-07, CD-10)	Batch tier at zero in run F; checklist; firewall	Disable the job; inform recipients

16.2 Kafka and Confluent

Table 74. Known issues: Kafka and Confluent

Symptom	Cause	Prevention	Fix
Client reaches bootstrap, then broker timeouts	Broker names resolve to public IPs or not at all; zonal Private Link records missing [R32]	N-4 to N-6 for every new cluster; synthetic DNS check	Add the zonal records
Too many partitions on a shared cluster	Many queue processors per environment, multiplied by environments	Measured budget (Section 6.7); alert at 70 %	Add capacity or move an environment
RecordTooLargeException	Confluent topic default max.message.bytes below Pega's 5,000,000 [R33]	K-6 after every deployment	Raise the topic setting
SASL authentication fails after a secret change	JAAS value stored with wrong quoting or line breaks	Store as one line; test with K-3 before restarting Pega	Correct the Key Vault value; restart
Old credential still in use after rotation	Pods not restarted after the secret changed (FS-20)	Restart is a rotation step	Rolling restart
Topic creation denied	Security policy removed CREATE from the ACL	Decide OD-11 early; pre-create topics if needed	Restore CREATE or pre-create the topic
Kafka outage stops more than queue processing	From '25, Pega uses Kafka for cluster messaging that Hazelcast carried [R7]	Treat Kafka as tier-1; FS-01	Restore Kafka; check nodes reconnect
Producer and consumer latency rises during restarts; brief stream errors	Connection or request limits enforced on Enterprise clusters [R31]	Measure connections per pod; stagger restarts in shared groups; FS-31	Wait for throttling to end; restart in smaller steps

Symptom	Cause	Prevention	Fix
First start or refresh takes far longer than planned	Partition creation paced at 500 per five minutes per cluster [R31]	Include topic creation time in rehearsals; one refresh per group at a time; FS-32	Let creation finish; do not restart pods mid-way
Partition count far above the budget after first start	Cloned 8.8 partition DSS (CD-15) [R49]	Record the DSS at time point A	Reset the DSS; resize with <code>pxAlterStreamPartitions</code> if Pega Support agrees
Adding batch pods does not increase throughput	Consumers already equal partitions [R50]	Effective consumer formula (Section 6.7)	Raise partitions for the bottleneck queue processor
Installer job stalls or logs Kafka connection errors	Upgrade tooling may need Kafka (GQ-08) [R8]	Allow installer egress to Confluent until GQ-08 is answered	Supply the stream settings as Pega Support advises; rerun

16.3 OpenSearch and SRS

Table 75. Known issues: OpenSearch and SRS

Symptom	Cause	Prevention	Fix
Indexes created with wrong names or settings, or index creation blocked	Auto-creation not disabled, or SRS user lacks rights [R14]	Apply cluster settings before SRS starts; IT-06	Apply settings; widen the role by the missing permission only; rebuild
SRS returns 401 or 403; Pega search fails	Okta token missing <code>pega.search:full</code> , missing or wrong <code>guid</code> , or issued by the org authorization server [R42, R44]	Custom authorization server; O-2 decode in every environment	Fix the scope, claim or server; restart Pega if the client changed
SRS will not start against OpenSearch	SRS image not on the matrix for the OpenSearch version [R14]	Pin versions per Section 7.2	Use a matrix pair
Old indexes remain after a refresh	Refresh skipped the index clean-up	Refresh checklist step (Section 19.3)	Delete <code>pega26-<code><code></code>*</code> as the environment's SRS user, then rebuild
Index deletion fails	<code>destructive_requires_name true</code> [R14]	OS-2	Set to false
<code>helm install</code> of SRS fails: "Only one authentication can be enabled"	<code>srsStorage.tls.enabled</code> and basic authentication both true; <code>tls</code> means certificate authentication [R24]	<code>tls.enabled: false</code> ; <code>helm template</code> in the pipeline	Set <code>tls.enabled: false</code>
Pega search fails at first start; SRS pods healthy	<code>externalURL</code> without <code>:8443</code> ; SRS serves HTTPS on 8443 only [R62]	URL with port; S-5 from the Pega namespace	Correct the URL; restart the Pega tiers
SRS pods never Ready; timeouts to OpenSearch and Okta	Chart network policy allows egress only to an in-cluster Elasticsearch [R62]	Chart policy off; own policy (Section 7.7)	Disable the chart policy; apply the replacement
Search fails for all environments at once	Okta org rate limit reached on the token endpoint [R52]	Token rate measured; System Log warnings alerted; FS-35	Find the caller; lengthen token lifetime
Search fails after an Okta key change	SRS keeps an old signing key, or Pega's client key mismatches [R53]	FS-33 and FS-34 in DEV	Restart SRS; restore the client key

16.4 Platform

Table 76. Known issues: platform

Symptom	Cause	Prevention	Fix
Most pods of a tier in one zone after a zone incident	Spread is a scheduling preference; running pods are not moved back [R61]	Check the spread after FS-21	Rolling restart of the tier
Pods killed with OOMKilled and no Java error	Heap too close to the container memory limit	Heap at about two thirds of the limit (Section 9.2)	Lower the heap or raise the limit
Delayed items and job schedulers run at the wrong hour	JVM time zone differs from the cloned database [R63]	<code>-Duser.timezone</code> in <code>javaOpts</code> and installer <code>customJVMArgs</code>	Set both; restart
GC logs and heap dumps lost after a crash	Written to ephemeral pod storage [R23]	Log collector tails the GC file (Section 18.2)	Collect from the replacement pod; enable collection

16.5 Process

Every refresh of a non-production environment from a new production clone repeats the risky parts of the first build: masking, topic and index clean-up, repointed integrations and a full reindex. Treat each refresh as a change with the checklist in Section 19.3, not as a database task alone.

17 Troubleshooting

17.1 Method

Most failures between Pega and Kafka or search are connectivity failures. Work through the chain in Figure 33 in order: a failure at one step makes every later step fail, so the first failing step is the one to fix. For cloned environments, first rule out a stale 8.8 setting (CD-01, CD-03).

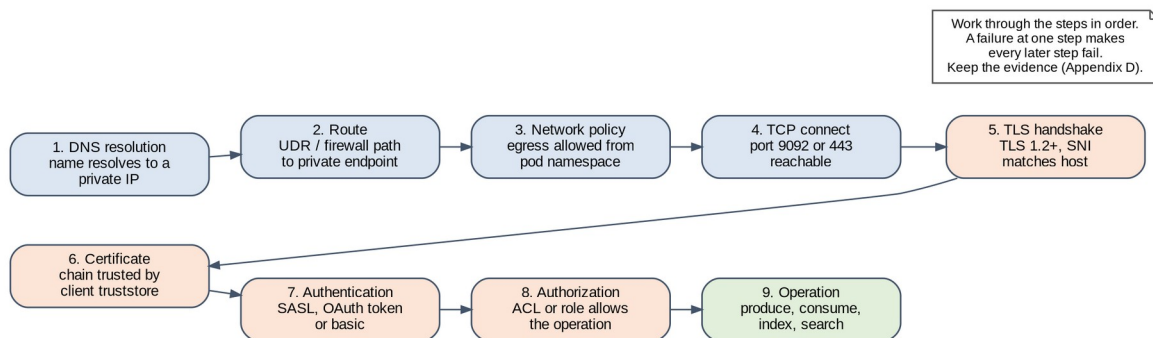


Figure 33. Connectivity chain for Kafka and search
Prepared for this implementation. Not a Pega-published diagram.

17.2 Kafka

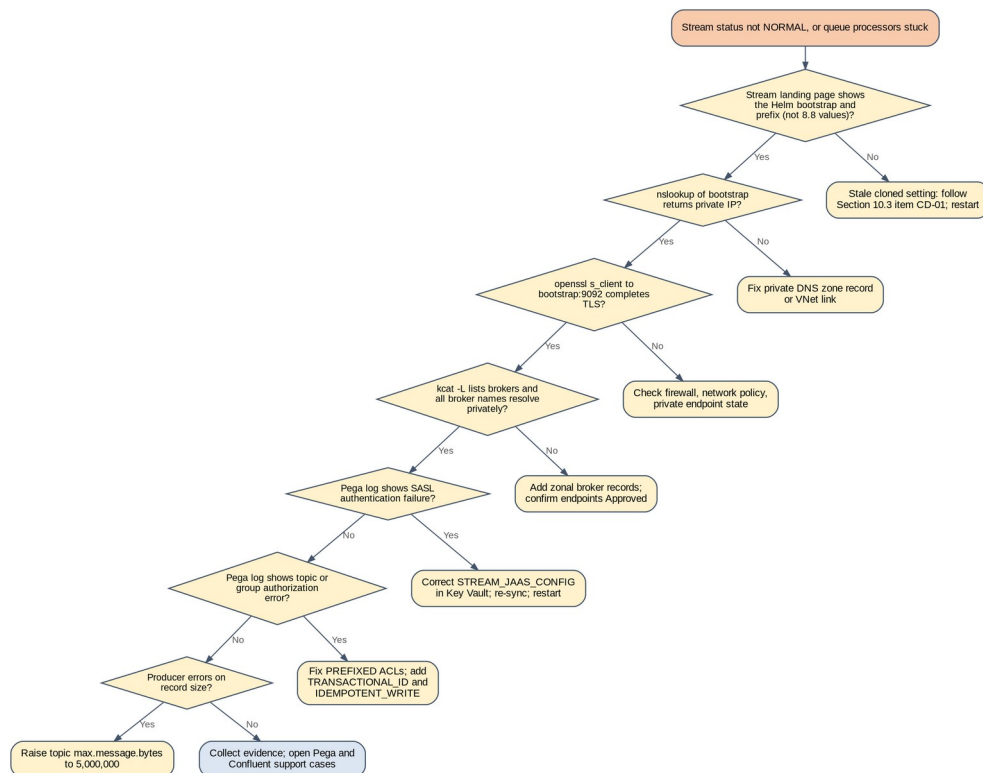


Figure 34. Troubleshooting tree: stream service
Prepared for this implementation. Not a Pega-published diagram.

Table 77. Kafka symptoms

Symptom	Likely cause	Check	Fix
Stream status not NORMAL after start	Stale cloned setting, DNS, network or authentication	Landing page values; K-1 to K-3; Pega log	Follow the tree
Bootstrap works, brokers time out	Zonal DNS records	K-3	Add records [R32]
Topic or group authorization errors	ACL prefix does not match <code>streamNamePattern</code>	K-4, K-7	Correct the prefix or ACL
Transactional or idempotent producer errors	Missing TRANSACTIONAL_ID or IDEMPOTENT_WRITE ACL	K-7	Add the ACL [R11]
RecordTooLargeException	Topic limit	K-6	Raise to 5,000,000
Ready to process keeps growing	Batch tier down or a failing processor	Admin Studio; batch pod logs (P-2)	Restore batch pods; fix the processor
Throughput drops in one shared environment	Quota reached, or another environment's load	Metrics API by principal	Adjust quota or schedule

17.3 Search

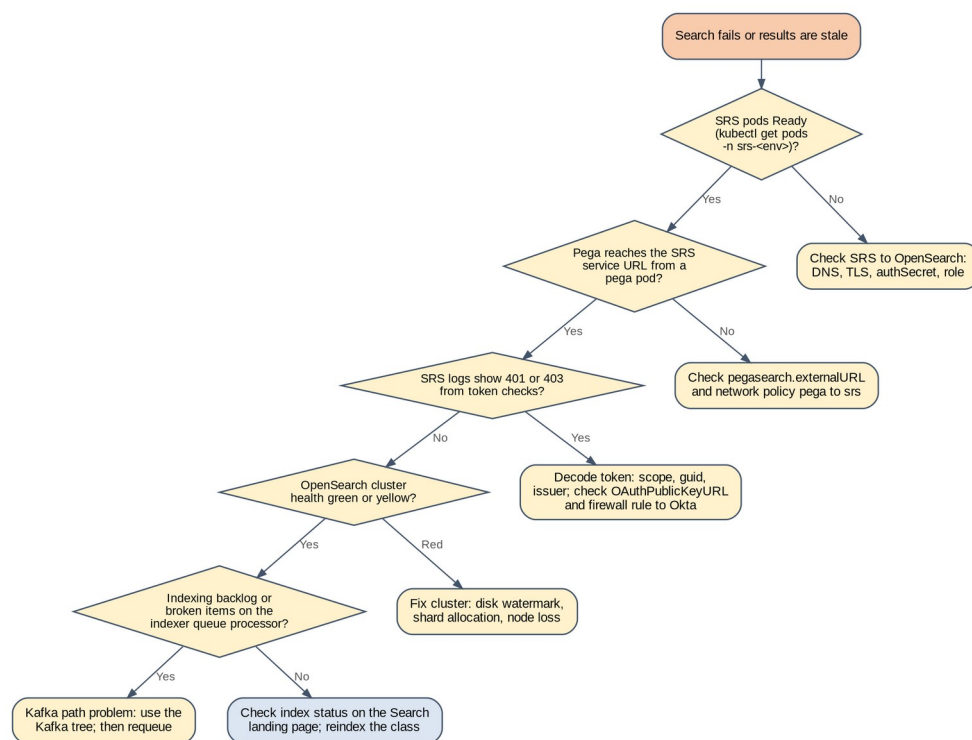


Figure 35. Troubleshooting tree: search

Prepared for this implementation. Not a Pega-published diagram.

Table 78. Search symptoms

Symptom	Likely cause	Check	Fix
SRS pods not Ready	OpenSearch unreachable or wrong credentials	SRS logs; S-1 from an SRS pod	Fix network or <code>authSecret</code> ; restart SRS
401 or 403 in SRS log	Token scope, <code>guid</code> or issuer	O-2	Fix Okta configuration
Token request fails	Firewall to Okta, client key mismatch	O-1; Okta system log	Fix rule or key

Symptom	Likely cause	Check	Fix
Results missing for recent changes	Indexing backlog	Queue processors; Kafka tree	Clear backlog
Class shows CONFLICTS FOUND	Mapping conflict in indexed properties [R19]	Search landing page	Fix the properties; reindex the class
Cluster red or read-only	Node loss or disk watermark	S-2; provider console	Restore capacity

17.4 Cloned database and upgrade

Table 79. Cloned database and upgrade symptoms

Symptom	Likely cause	Fix
Installer job fails	Database connectivity, permissions, schema names, or a missing prerequisite	Read the log (P-5); fix; rerun or restore the clone (FS-26)
primaryKeyUtility reports insufficient space	The utility copies each table missing a primary key	Add database storage; rerun [R10]
ClassNotFoundException for javax classes	Custom JAR not rebuilt for Jakarta EE	Import the rebuilt JAR into a higher CodeSet version [R9]
PEGA0179 alerts	Old queue processor DSS still present (CD-11)	Delete the DSS [R3]

17.5 Support cases

Pega Global Client Support handles externalized services as it handles databases: it determines the cause, then hands over to the third-party provider if the issue is not in Pega software [\[R22\]](#). Open cases with Pega logs from all tiers, Stream and search landing page screenshots, queue processor status, SRS logs, the Appendix C outputs, the decoded token claims, and the values file with secrets removed.

18 Observability, monitoring and logging

Kafka and search now run outside Pega, on three vendors' services. A fault can start in any of them and show up as a slow screen in Pega. Observability here means collecting enough telemetry from every layer, with the same environment label and the same clock, to find the layer at fault within minutes. Figure 36 shows the design.

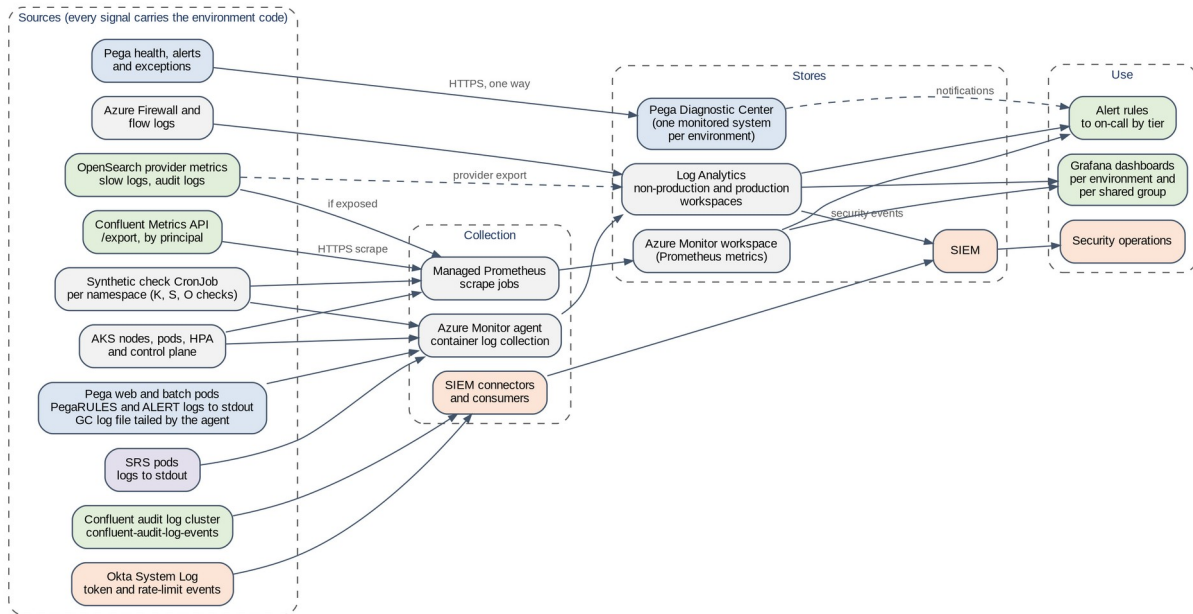


Figure 36. Telemetry sources, collection, stores and use
Prepared for this implementation. Not a Pega-published diagram.

18.1 Telemetry sources

Table 80. Telemetry sources

Source	Telemetry	Collection	Store	Notes
Pega web and batch pods	PegaRULES and ALERT logs on stdout; GC log file	Azure Monitor agent container log collection [R60]	Log Analytics	GC file tailed by the agent [R23]
Pega Platform	Health, alerts, exceptions, guardrail data	Pega Diagnostic Center [R51]	PDC	Section 18.3
AKS	Node, pod, HPA and control-plane metrics	Managed Prometheus [R60]	Azure Monitor workspace	Grafana dashboards
Confluent Cloud	Throughput, requests, connections, lag, throttling, by principal	Metrics API /export endpoint in Prometheus format, scraped every minute [R56]	Azure Monitor workspace	Needs a Cloud API key for a service account with the MetricsViewer role. The Metrics API addresses are not static, so allow it by FQDN
Confluent audit log	Authentication, ACL and management events	Consumer on topic <code>confluent-audit-log-events</code> in the audit log cluster [R57]	SIEM	Kept 7 days by default; export for longer retention
OpenSearch	Cluster health, nodes, disk, latency; slow logs; audit logs	Provider export	Log Analytics	Section 18.2
SRS pods	Application logs on stdout	Azure Monitor agent	Log Analytics	Request errors and latency
Okta	System Log: token issue, failures, rate-limit events	Okta log streaming or System Log API to the SIEM	SIEM	Rate-limit events [R52]

Source	Telemetry	Collection	Store	Notes
Azure Firewall	Allowed and denied flows	Diagnostic settings	Log Analytics	IT-09, IT-10 evidence
Synthetic checks	DNS, TLS, Kafka metadata, SRS and token checks	CronJob per namespace	Log Analytics and metrics	Section 18.6

18.2 Logging design

- **One label everywhere.** The namespace carries the environment code (`pega-<code>`, `srs-<code>`). Add the code as a label to Confluent and OpenSearch metrics at scrape time, so a dashboard can filter one environment on a shared group.
- **Workspaces.** Use one Log Analytics workspace for non-production and one for production. Production logs never go to a non-production workspace. Set retention per the customer's policy, for example 30 days interactive in non-production and 90 days in production with archive after that.
- **Log levels.** Keep Pega and SRS at their default levels in PROD. Raise a level for one logger and one pod for a set time when Pega Support asks, then return it. A custom `prlog4j2` file can be supplied through the chart if the default layout does not suit the log platform [R23].
- **Sensitive data.** Non-production logs come from masked clones only. Never log the JAAS value, tokens or private keys; the pipeline secret scan covers values files, and log queries for `password=` and `Bearer` run weekly.
- **OpenSearch slow logs.** Use the cluster-level search request slow log, available from OpenSearch 2.12, rather than per-index shard slow logs [R58]. It needs no change to index settings, which SRS owns (Section 7.4).
- **Confluent audit log.** The default 7-day retention is too short for incident review [R57]. Stream it to the SIEM.

18.3 Pega Diagnostic Center

Pega Diagnostic Center (PDC) collects health and alert data from Pega and shows it per system. For a client-managed deployment, the endpoint URL that PDC provides is entered in Pega at Configure > System > Settings. Data goes one way, from Pega to PDC, over HTTPS, and the JVM truststore must trust the PDC certificate chain [R51].

- Register each environment as its own system in PDC, so production and non-production alerts are never mixed.
- A clone carries production's PDC setting (CD-16). Change it during run F, before any alert is sent.
- Allow the PDC endpoint at the hub firewall from the Pega subnets (NW-4).
- Use PDC for Pega-level analysis (alerts such as PEGA0179, exceptions, guardrails) and the Azure stack for platform and service telemetry. Both are needed; neither replaces the other.

18.4 Dashboards

Table 81. Dashboards

Dashboard	Scope	Panels	Users
Environment overview	One per environment	M1 response times; pod health; stream status; queue backlog; search errors; token failures	Operations, test leads
Kafka group	One per cluster (NP1, NP2, PROD)	Throughput, requests, connections, lag and throttling by service account; partitions against limit	Kafka team
Search group	One per OpenSearch service	Health, disk against watermarks, latency, indexing rate, shards per node; SRS errors by environment	Search team
Identity	Okta org	Token requests per client, failures, rate-limit events	Identity team

Dashboard	Scope	Panels	Users
Capacity	All groups	Units, partitions, shards, disk, node usage against plan; quarterly review	Platform lead
Cutover	PROD during cutover and hypercare	Steps C-1 to C-10 timings; index build progress; backlog; error rates	Cutover manager

18.5 Alerts and thresholds

Thresholds marked as baselines come from the PERF load test. Route PROD P1 alerts to on-call at any hour, NP2 alerts to the test lead during test windows, and NP1 alerts during business hours. An alert on a shared cluster or search service goes to the owner of that service, with the environment label in the message.

Table 82. Alerts, thresholds and responses

Signal	Source	Threshold	Severity	Response
Stream service status	Pega (Stream landing page, alerts)	Not NORMAL	P1 in PROD	Section 17.2
Queue backlog	Pega Admin Studio	Ready to process grows for longer than the agreed period	P2	Check batch tier and consumer errors
Broken queue items	Pega Admin Studio	Above zero for business-critical processors	P2	Investigate and requeue
Consumer lag and request errors per principal	Confluent Metrics API [R56]	Lag above baseline; authentication or authorization errors	P2	Section 17.2
Client throttling	Metrics API <code>client_limit_millisecnds</code> by principal [R56]	Above zero for 5 minutes	P2	Find the limit hit (Table 18)
Connections per cluster	Confluent Metrics API [R56]	Above 70 % of the cluster's connection limit	P3	Capacity review
Partition count per cluster	Confluent Metrics API	Above 70 % of the cluster limit	P3	Capacity review (Section 6.7)
Throughput per service account	Confluent Metrics API [R36]	At quota for longer than 15 minutes	P3	Review quota or schedule
SRS errors, latency and restarts	SRS logs; pod metrics	Above baseline; any restart loop	P2	Section 17.3
OpenSearch health	Provider metrics	Yellow for longer than 30 minutes; red at once	P2 / P1	Section 17.3
OpenSearch disk	Provider metrics	Above the low watermark less 10 points	P2	Add storage
Okta token failures	Pega log; Okta System Log	Any repeated failure	P2	Section 17.3
Okta rate limit	Okta System Log warning and violation events [R52]	Any warning; any violation	P2 / P1	Find the caller; Section 7.6
Pod health	Managed Prometheus	OOMKilled, restart loop, HPA at maximum for 15 minutes	P2	Section 9.2
Zone spread	Managed Prometheus	More than half of a tier's pods in one zone	P3	Section 4.6
Long GC pauses	GC log	Pauses above 2 seconds, or GC time above 10 % of an interval	P3	Heap review
External secret sync	External Secrets Operator status	Any ExternalSecret not SecretSynced	P2	Section 8
Certificate expiry	Synthetic checks	Less than 30 days	P3	Renew (Section 8.4)
Private DNS resolution	Synthetic checks	A public IP or no answer	P2	Section 4.4

18.6 Synthetic checks

A CronJob in each `pega-<code>` namespace runs a subset of the Appendix C checks and publishes each result as a metric, so a broken path is found before users find it.

Table 83. Synthetic checks

Check	Commands	Interval	Alert when
Confluent DNS and TLS	K-1, K-2	5 minutes	A public IP, no answer, or a TLS failure
Kafka metadata	K-3 with the environment's own key	5 minutes	Any broker missing or unreachable
SRS	S-5 health call	5 minutes	Unhealthy or no answer
Okta token	O-1	15 minutes, to limit token requests	Token not issued
Certificate expiry	Read the chains from K-2 and S-5	Daily	Less than 30 days

18.7 Following an incident across services

Each system names the same environment differently. Table 84 lists the identifiers to search for, so an engineer can follow one incident from a Pega log line to the Confluent, OpenSearch and Okta records.

Table 84. Identifiers used to correlate one environment across systems

System	Identifier for the environment	Identifier for the request or client
Pega	Namespace <code>pega-<code><code></code></code> ; node ID in the log	Thread and requestor in PegaRULES log
Confluent	Service account <code>sa-pega-<code><code></code></code> (<code>principal_id</code> in metrics and audit log)	Topic and consumer group under <code>pega-<code><code></code></code> -
SRS	Namespace <code>srs-<code><code></code></code>	Index prefix <code>pega26-<code><code></code></code>
OpenSearch	User <code>pega26-<code><code></code>-srs</code>	Index name in slow and audit logs
Okta	Client <code>pega-srs-<code><code></code></code>	Client ID in System Log events
Azure	Subnet and pod IP	Firewall flow log

1. Start from the time and environment of the first symptom. All sources use UTC.
2. Check the environment overview dashboard to find the first layer that changed: M2 to M6 in that order.
3. Search that layer's logs with the identifiers above for the five minutes before the symptom.
4. If the layer is a shared service, check the group dashboard for other environments affected at the same time. One environment affected points to its own configuration or load; all environments point to the service.
5. Record the timeline in the incident, with links to the queries used.

19 Operations, onboarding, refresh and retirement

Monitoring, logging and alerting are in Section 18. This section covers the routine work that keeps the services healthy and the procedures for adding, refreshing and removing an environment.

19.1 Routine tasks

Table 85. Routine operations

Task	Frequency	Owner	Procedure
Confluent API key rotation	Per security policy	Kafka engineer	Section 8.3
OpenSearch SRS user password rotation	Quarterly	Search engineer	Section 8.3
Okta client key rotation	Yearly or per policy	Identity team	Section 8.3
Certificate renewal	Before expiry	Platform team	Section 8.4
Partition and shard capacity review	Monthly and after each release	Kafka and search engineers	K-5, S-4
Check new topics for max.message.bytes	After each deployment	Kafka engineer	K-4, K-6
SRS image update	With Pega updates or security fixes	Search engineer	Update <code>srsImage</code> per environment, NP1 first
OpenSearch version update	Per provider notice, within the SRS matrix	Search engineer	NP1, then NP2, then PROD; rebuild if advised
Cost review	Quarterly	Platform lead	Cost model in Section 5.5 with actual usage

19.2 Onboarding a new non-production environment

1. Approve the environment and its group with the decision in Section 5.1. Assign a code that is not the start of another code.
2. Add a row to Table 8 and to Appendix A.
3. Create the namespace, Key Vault, External Secrets identity and network policies (SI-1, SI-2, NW-5).
4. Create the Okta client and, for option 2, add the client to the `guid` expression (Section 7.6).
5. Create the service account, ACLs, quota and API key on the group's cluster (CC-2 to CC-4).
6. Create the OpenSearch user and index-scoped role (OS-3, OS-4).
7. Deploy SRS (SR-1 to SR-3).
8. Clone, mask and upgrade the database (DB-1 to DB-5), then follow the first-start control (Section 10.4).
9. Run the isolation tests against every other environment in the group.
10. Add the environment to monitoring and the cost review.

19.3 Refreshing an environment from a new clone

1. Agree the refresh window with the environment's users and the other environments in the group.
2. Scale the environment's Pega tiers to zero.
3. Delete its topics and consumer groups under `pega-<code>` (K-9), and its indexes `pega26-<code>*` (S-6). Keep the prefix, customerDeploymentId, ACLs, quota, Okta client and SRS user.
4. Take, mask and upgrade the new clone (DB-1 to DB-5).

5. Follow the first-start control (Section 10.4), including repointing application Kafka data sets and other endpoints.
6. Run the full index build and the functional checks.
7. Record the refresh in the environment log with the evidence.

19.4 Retiring an environment

1. Scale the Pega tiers and SRS to zero; uninstall the Helm releases.
2. Delete the topics and consumer groups under the prefix (K-9).
3. Delete the ACLs, API keys and service account (Section 6.5).
4. Delete the indexes `pega26-<code>*` (S-6), then the OpenSearch user and role.
5. Delete the Okta client and remove it from the `guid` expression.
6. Delete the Key Vault secrets, External Secrets objects and the namespace; purge the vault per retention policy.
7. Remove firewall rules and DNS records specific to the environment.
8. Record evidence of each deletion in the retirement record (Appendix D).

19.5 Roles and responsibilities

R = responsible, A = accountable, C = consulted, I = informed.

Table 86. RACI (Okta objects are owned by the customer identity team: A/R)

Activity	Platform team	DBA team	Kafka team	Search team	Pega team	Pega Support	Confluent	OpenSearch provider
Network, DNS, firewall	A/R	I	C	C	I	I	C	C
Confluent clusters, ACLs, quotas	C	I	A/R	I	C	I	C	I
OpenSearch services, roles	C	I	I	A/R	C	I	I	R
SRS deployment	R	I	I	A/R	C	C	I	I
Okta authorization servers and clients	C	I	I	C	C	I	I	I
Clone, masking, upgrade	R	A/R	I	I	C	C	I	I
Cloned item clean-up	C	R	C	C	A/R	C	I	I
Testing and rehearsals	R	R	R	R	A/R	I	C	C
Cutover and rollback	R	R	R	R	A/R	C	I	I
Operations after handover	A/R	R	R	R	R	C	C	C

20 Risks, open decisions, assumptions and source reconciliation

20.1 Risk register

Table 87. Risk register (L likelihood, I impact: H high, M medium, L low)

ID	Risk	L	I	Mitigation	Owner
RK-01	Pega Support does not confirm the direct 8.8 to 26.1.1 clone-and-upgrade path, or sets conditions late	M	H	Raise the case before M2 (OD-01); conservative plan; two rehearsals	Pega LSA
RK-02	A stale 8.8 DSS overrides the Helm stream or search configuration	M	H	FS-24 in DEV; GQ-05	Pega LSA
RK-03	A non-production clone touches production Kafka topics or systems	M	H	Batch tier at zero in run F; repointing; firewall deny; IT-09, IT-10	Platform team
RK-04	Index build longer than the outage window	M	H	Measure in PERF and PREPROD; scale for the build	Search team
RK-05	Okta licence lacks custom authorization servers, or the client-based claim does not work for client credentials	M	H	Check licence before M2; DEV test; fall back to option 1	Identity team
RK-06	SRS needs cluster permissions beyond the index-scoped role	M	M	IT-06 in DEV; widen only by named permissions	Search team
RK-07	TRANSACTIONAL_ID * ACL is shared by all environments on a cluster	L	M	Accepted residual risk; separate clusters for PROD	Kafka team
RK-08	Noisy neighbour in a shared group	M	M	Quotas; schedule; per-principal metrics	Kafka team
RK-09	No OpenSearch provider passes the selection	L	H	Self-managed fallback on AKS	Enterprise architect
RK-10	Delayed or broken items fail after the upgrade	M	M	Drain fully; resolve broken items; FS-28; GQ-06	Pega LSA
RK-11	Masking breaks search or integration keys	M	M	Masking rules agreed with the application team; search tests on masked data	DBA team
RK-12	Vendor documentation changes between rehearsal and cutover	M	M	Recheck Section 3 before each rehearsal	Enterprise architect
RK-13	Confluent connection and request limits throttle Pega during restarts or peaks	M	M	Measure connections and requests per pod; stagger restarts; throttle alert; FS-31	Kafka team
RK-14	One Okta org rate limit shared by all environments and other applications	M	H	Token rate measured; System Log alerts; token lifetime; FS-35	Identity team
RK-15	The installer needs Kafka during the upgrade and the chart does not supply it	L	H	GQ-08; installer egress to Confluent allowed in DEV	Pega LSA
RK-16	Load test unrepresentative, so PROD is undersized	M	H	Workload model from production logs; exit criteria in Section 15.5	Performance lead

20.2 Open decisions

Each open decision has a recommended answer, an owner and the milestone by which it is needed (Section 13.2). If a decision is not taken by its milestone, the recommended answer applies and the risk is recorded.

Table 88. Open decisions

ID	Decision	Options	Recommended answer	Owner	Needed by
OD-01	Pega Support answers (GQ-01 to GQ-08)	Answers received; conservative plan	Raise now; plan conservatively until answered	Pega LSA	M2

ID	Decision	Options	Recommended answer	Owner	Needed by
OD-02	Confluent cluster types	Enterprise; Dedicated	Enterprise for NP1; PROD type for NP2 and PROD after PERF measurement	Kafka team	M2 (NP1), M3 (NP2, PROD)
OD-03	OpenSearch provider	Third-party managed on Azure; self-managed on AKS	Provider that passes Section 7.3	Enterprise architect	M2
OD-04	SRS per environment or shared	Per environment; per group	Per environment	Search team	M2
OD-05	Okta design	Server per environment; shared non-production server	PROD dedicated; shared non-production with client-based claim if proven	Identity team	M2
OD-06	Environment grouping	Options A to D (Section 5.1)	Option B	Enterprise architect	M1
OD-07	Masking approach	Before upgrade; after upgrade before run D	Before upgrade	DBA team	M2
OD-08	Treatment of each application Kafka data set	Keep; new cluster; Cluster Linking	Per data set (Table 45)	Application owner	M4
OD-09	Quota approach	Quotas per service account; no quotas	Quotas in NP1 and NP2 from measured peaks	Kafka team	M3
OD-10	Kafka authentication	SASL/PLAIN; SASL/OAUTHBEARER	SASL/PLAIN	Security architect	M2
OD-11	Topic creation	Pega creates (CREATE ACL); pre-created topics	Pega creates	Security architect	M2
OD-12	Outage and rollback windows	From rehearsal	Rehearsal time plus 25 %	Business owner	M5
OD-13	OpenSearch snapshots	None; provider snapshots	Provider snapshots for PROD and NP2; none for NP1	Search team	M3

20.3 Assumptions

Table 89. Assumptions

ID	Assumption	Check that would prove it wrong
AS-01	The customer's Okta tenant includes API Access Management	Okta admin console shows no custom authorization servers
AS-02	Confluent Cloud offers Enterprise and Dedicated with Private Link in the AKS region	Confluent console region list
AS-03	The application does not need Cassandra (no Customer Decision Hub or Process AI)	Application stack review
AS-04	The database engine and version of the clone are on the 26.1.1 Platform Support Guide [R1]	DBA check against [R1]
AS-05	8.8 queues can be drained within the planned window	Rehearsal drain time
AS-06	Non-production environments have their own AKS namespaces and network policies	Platform design review
AS-07	The firewall supports FQDN rules for <code><okta-domain></code>	Firewall policy test
AS-08	The AKS clusters were created with a network policy engine, so NetworkPolicy objects are enforced	A test pod reaches a target that a policy denies
AS-09	The Azure region offers three availability zones for the node sizes chosen	Node pool creation with zones 1, 2 and 3 fails

ID	Assumption	Check that would prove it wrong
AS-10	Production request logs or Pega PDC data are available to build the workload model	No usable 8.8 usage data

20.4 Source reconciliation

These are the places where sources disagree or leave a gap, with the action taken.

Table 90. Source reconciliation register

ID	Topic	What the sources say	Action
RC-01	SRS version	The '26 page lists SRS 1.44.3 or later and OpenSearch best practice 2.15 [R14]; the chart README lists a newer SRS and OpenSearch 2.19 [R24]	Choose OpenSearch 2.15 or 2.19; use the SRS tag the README lists for chart 4.13.0
RC-02	Kafka SASL mechanisms	Pega lists JAAS, OAUTHBEARER, PLAIN and SCRAM [R11]; the chart's saslMechanism lists PLAIN and SCRAM [R23]	Use PLAIN (OD-10)
RC-03	Broker settings on Confluent Cloud	Pega lists replica.fetch.* and unclean.leader.election.enable [R11]; not editable on Confluent Cloud [R34]	Written confirmation from Confluent; topic max.message.bytes set
RC-04	Kafka version	The chart's Kafka requirements page names an older Kafka [R26]; the '26 page names client 4.0.0 [R3]	Follow the '26 page
RC-05	Default customerDeploymentId	Pega page: generated automatically by the chart [R14]; README: defaults to the namespace name [R23]	Always set it explicitly
RC-06	Hazelcast removal for a cloned database	Removal required before updating to '25 [R5]; steps assume a running system [R6]	GQ-03 to Pega Support
RC-07	DSS and Helm precedence	Not settled by the pages read	FS-24; GQ-05
RC-08	SRS checks on issuer and audience	Pega documents the signature and guid checks only [R23]	FS-12 in DEV
RC-09	Okta app attributes in client credentials tokens	Expression language lists app attributes [R46]; client credentials page does not mention them [R43]	DEV test before choosing option 2
RC-10	SRS replicas in test environments	Pega sizing: 1 SRS for testing and development [R14]; SRS chart README: minimum 2, best practice 3 [R24]	2 in DEV, SIT and UAT; 3 elsewhere
RC-11	SRS pod resources	Pega sizing: 2 CPU, 2 GB [R14]; chart defaults: 0.65 to 1.3 CPU, 4Gi [R24]	Request 1 CPU, limit 2 CPU, 4Gi; adjust from PERF
RC-12	Topology spread for Pega tiers	Read by the chart 4.13.0 template [R61]; not in the README [R23]	Use it; check in <code>helm template</code> output after each chart upgrade
RC-13	OpenSearch storage and shard sizing	Pega gives node sizes only [R14]; general OpenSearch guidance gives formulas [R54, R55]	Use the formulas with measured index sizes (Section 7.4)

Appendix A. Configuration inventory

This appendix holds every environment-specific value. The names in the first table follow the convention in Section 2.6 and are fixed once the environment is built. The values in the later tables are recorded during the build and kept under change control. Secrets are never recorded here; only the Key Vault secret names are.

A.1 Names per environment

Table 91. Names and sizes per environment

Item	DEV	SIT	UAT	PERF	PREPROD	PROD
Code	dev	sit	uat	perf	ppd	prd
Service group	NP1	NP1	NP1	NP2	NP2	PRD
Pega namespace	pega-dev	pega-sit	pega-uat	pega-perf	pega-ppd	pega-prd
SRS namespace and release	srs-dev	srs-sit	srs-uat	srs-perf	srs-ppd	srs-prd
Key Vault	kv-pega-dev	kv-pega-sit	kv-pega-uat	kv-pega-perf	kv-pega-ppd	kv-pega-prd
Confluent cluster	cc-np1	cc-np1	cc-np1	cc-np2	cc-np2	cc-prd
Service account	sa-pega-dev	sa-pega-sit	sa-pega-uat	sa-pega-perf	sa-pega-ppd	sa-pega-prd
Topic and group prefix	pega-dev-	pega-sit-	pega-uat-	pega-perf-	pega-ppd-	pega-prd-
customerDeploymentId and index prefix	pega26-dev	pega26-sit	pega26-uat	pega26-perf	pega26-ppd	pega26-prd
OpenSearch service	os-np1	os-np1	os-np1	os-np2	os-np2	os-prd
OpenSearch user	srs-dev	srs-sit	srs-uat	srs-perf	srs-ppd	srs-prd
OpenSearch role	pega26-dev-srs	pega26-sit-srs	pega26-uat-srs	pega26-perf-srs	pega26-ppd-srs	pega26-prd-srs
Okta client	pega-srs-dev	pega-srs-sit	pega-srs-uat	pega-srs-perf	pega-srs-ppd	pega-srs-prd
Web replicas (min to max)	1 to 2	1 to 3	2 to 3	3 to 6	3 to 6	3 to 6
Batch replicas (min to max)	1 to 2	1 to 3	1 to 3	3 to 6	3 to 6	3 to 6
SRS replicas	2	2	2	3	3	3

The replica counts are starting values. PERF measurements (M3) set the PROD and PREPROD values.

A.2 Values recorded per service group

Table 92. Values recorded per service group

Value	Where used	NP1	NP2	PRD
Confluent environment and cluster ID (lkc-...)	Commands; Confluent console			
Cluster type and capacity	OD-02; Section 6.3			
Bootstrap host (<bootstrap-host>)	stream.bootstrapServer			
Confluent network ID and private endpoint	Section 6.4			
Private DNS zone for Confluent	Section 4.4			
Partition limit and measured count	Section 6.7			
OpenSearch provider and version	OD-03; Section 7.2			
OpenSearch endpoint (<search-host>) and port	srs.srsStorage.domain, port			

Value	Where used	NP1	NP2	PRD
OpenSearch data nodes, storage and shards	Section 7.4			
Disk watermarks (low, high, flood stage)	Section 7.4; Section 18.5			
Okta authorization server ID (<auth-server-id>)	Token URL			
Token endpoint and JWKS URL (<okta-jwks-url>)	pegasearch.srsAuth.url; 0AuthPublicKeyURL			
Access token lifetime	Okta access policy rule			

A.3 Values recorded per environment

Table 93. Values recorded per environment (one copy per environment)

Value	Where used	Recorded value and date
Confluent API key ID (not the secret)	confluent-jaas in Key Vault	
Client quota (produce and consume)	Section 5.4	
Okta client ID (<okta-client-id>)	pegasearch.srsAuth.clientId	
Okta client key ID (<kid>) and expiry	Section 8.3	
JDBC URL, driver class and database type	global.jdbc	
Rules and data schema names	global.jdbc.rulesSchema, dataSchema	
Pega host name (<pega-host>)	Web tier ingress	
Application Gateway request timeout	Ingress annotation	
Pega, installer and SRS image digests	Pipeline record	
Helm chart version	Pipeline	4.13.0
SRS image tag (<tag>)	srs.srsRuntime.srsImage	
Index shard and replica counts after the first build	Section 7.4	
Full index build time and document counts	Section 7.8	
Cloned DSS for stream and search (CD-01, CD-03)	Section 10.3	

Appendix B. Complete Helm values files

These files are generated from the same source as the tables in Section 9, for chart version 4.13.0 [R23, R24]. SIT stands for the NP1 environments and PROD for PROD; the other environments differ only in the code, the names in Table 91 and the replica counts. Values in angle brackets come from Appendix A. No secret is written in these files: each `external_secret_name` and `authSecret` names a Kubernetes secret created by the External Secrets Operator (Section 8.2). Render every file with `helm template` and review the output before use. Every file below was rendered with `helm template` against charts 4.13.0 without errors before this version was issued.

B.1 SIT: pega chart, upgrade run

`pega-sit-upgrade.yaml`. Upgrade run: installer job only, no Pega pods.

```
global:
  provider: "aks"
  deployment:
    name: "pega"
  actions:
    execute: "upgrade"
  customerDeploymentId: "pega26-sit"
  jdbc:
    url: "<jdbc-url>"
    driverClass: "<jdbc-driver-class>"
    dbType: "<db-type>"
    driverUri: "<driver-uri>"
    external_secret_name: "pega-db-secret"
    rulesSchema: "<rules-schema>"
    dataSchema: "<data-schema>"
  docker:
    registry:
      url: "<acr-name>.azurecr.io"
    imagePullSecretNames: []
    pega:
      image: "<acr-name>.azurecr.io/platform/pega:26.1.1"
  tier:
    - name: "web"
      nodeType: "WebUser"
      replicas: 1
      service:
        port: 80
        targetPort: 8080
        tls:
          enabled: true
          external_secret_names: ["pega-tier-tls"]
          port: 443
          targetPort: 8443
      ingress:
        enabled: true
        domain: "<pega-host>"
        annotations:
          appgw.ingress.kubernetes.io/request-timeout: "<seconds>"
        tls:
          enabled: true
      resources:
        requests:
          memory: "12Gi"
          cpu: 3
        limits:
          memory: "12Gi"
          cpu: 4
      javaOpts: >-
        -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
        -XX:MaxMetaspaceSize=768m
        -XX:+UseStringDeduplication
        -XX:+HeapDumpOnOutOfMemoryError
      hpa:
        enabled: true
        minReplicas: 1
```

```

    maxReplicas: 3
  pdb:
    enabled: true
    minAvailable: 1
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: ScheduleAnyway
      labelSelector:
        matchLabels:
          app: pega-web
- name: "batch"
  nodeType: "BackgroundProcessing,Search,Batch,RealTime,\
    Custom1,Custom2,Custom3,Custom4,Custom5,BIX"
  replicas: 1
  hpa:
    enabled: true
    minReplicas: 1
    maxReplicas: 3
  resources:
    requests:
      memory: "12Gi"
      cpu: 3
    limits:
      memory: "12Gi"
      cpu: 4
  javaOpts: >-
    -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
    -XX:MaxMetaspaceSize=768m
    -XX:+UseStringDeduplication
    -XX:+HeapDumpOnOutOfMemoryError
  pdb:
    enabled: true
    minAvailable: 1
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: ScheduleAnyway
      labelSelector:
        matchLabels:
          app: pega-batch
cassandra:
  enabled: false
hazelcast:
  enabled: false
  clusteringServiceEnabled: false
stream:
  enabled: true
  bootstrapServer: "<bootstrap-host>:9092"
  securityProtocol: SASL_SSL
  saslMechanism: PLAIN
  trustStore: ""
  keyStore: ""
  jaasConfig: ""
  streamNamePattern: "pega-sit-{stream.name}"
  replicationFactor: "3"
  external_secret_name: "pega-stream-secret"
pegasearch:
  externalSearchService: true
  externalURL: "https://srs-sit.srs-sit.svc.cluster.local:8443"
  srsAuth:
    enabled: true
    url: "https://<okta-domain>/oauth2/<auth-server-id>/v1/token"
    clientId: "<okta-client-id>"
    scopes: "pega.search:full"
    authType: "private_key_jwt"
    privateKeyAlgorithm: "RS256"
    external_secret_name: "pega-srs-oauth"
  srsMTLS:
    enabled: false
installer:
  image: "<acr-name>.azurecr.io/platform/installer:26.1.1"
  upgrade:
    upgradeType: "in-place"
    targetRulesSchema: ""

```

```
targetDataSchema: ""
```

B.2 SIT: pega chart, first-start run

pega-sit-first.yaml. First-start run: web tier only, batch tier held at zero.

```
global:
  provider: "aks"
  deployment:
    name: "pega"
  actions:
    execute: "deploy"
  customerDeploymentId: "pega26-sit"
  jdbc:
    url: "<jdbc-url>"
    driverClass: "<jdbc-driver-class>"
    dbType: "<db-type>"
    driverUri: "<driver-uri>"
    external_secret_name: "pega-db-secret"
    rulesSchema: "<rules-schema>"
    dataSchema: "<data-schema>"
  docker:
    registry:
      url: "<acr-name>.azurecr.io"
      imagePullSecretNames: []
    pega:
      image: "<acr-name>.azurecr.io/platform/pega:26.1.1"
  tier:
    - name: "web"
      nodeType: "WebUser"
      replicas: 1
      service:
        port: 80
        targetPort: 8080
        tls:
          enabled: true
          external_secret_names: ["pega-tier-tls"]
          port: 443
          targetPort: 8443
      ingress:
        enabled: true
        domain: "<pega-host>"
        annotations:
          appgw.ingress.kubernetes.io/request-timeout: "<seconds>"
        tls:
          enabled: true
      resources:
        requests:
          memory: "12Gi"
          cpu: 3
        limits:
          memory: "12Gi"
          cpu: 4
      javaOpts: >-
        -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
        -XX:MaxMetaspaceSize=768m
        -XX:+UseStringDeduplication
        -XX:+HeapDumpOnOutOfMemoryError
      hpa:
        enabled: true
        minReplicas: 1
        maxReplicas: 3
      pdb:
        enabled: true
        minAvailable: 1
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: pega-web
    - name: "batch"
```

```

nodeType: "BackgroundProcessing,Search,Batch,RealTime,\
Custom1,Custom2,Custom3,Custom4,Custom5,BIX"
replicas: 0
hpa:
  enabled: false
resources:
  requests:
    memory: "12Gi"
    cpu: 3
  limits:
    memory: "12Gi"
    cpu: 4
javaOpts: >-
  -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
  -XX:MaxMetaspaceSize=768m
  -XX:+UseStringDeduplication
  -XX:+HeapDumpOnOutOfMemoryError
pdb:
  enabled: true
  minAvailable: 1
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: ScheduleAnyway
    labelSelector:
      matchLabels:
        app: pega-batch
cassandra:
  enabled: false
hazelcast:
  enabled: false
  clusteringServiceEnabled: false
stream:
  enabled: true
  bootstrapServer: "<bootstrap-host>:9092"
  securityProtocol: SASL_SSL
  saslMechanism: PLAIN
  trustStore: ""
  keyStore: ""
  jaasConfig: ""
  streamNamePattern: "pega-sit-{stream.name}"
  replicationFactor: "3"
  external_secret_name: "pega-stream-secret"
pegasearch:
  externalSearchService: true
  externalURL: "https://srs-sit.srs-sit.svc.cluster.local:8443"
  srsAuth:
    enabled: true
    url: "https://<okta-domain>/oauth2/<auth-server-id>/v1/token"
    clientId: "<okta-client-id>"
    scopes: "pega.search:full"
    authType: "private_key_jwt"
    privateKeyAlgorithm: "RS256"
    external_secret_name: "pega-srs-oauth"
  srsMTLS:
    enabled: false
installer:
  image: "<acr-name>.azurecr.io/platform/installer:26.1.1"
  upgrade:
    upgradeType: "in-place"
    targetRulesSchema: ""
    targetDataSchema: ""

```

B.3 SIT: pega chart, steady-state deploy run

pega-sit-deploy.yaml. Steady-state deploy run.

```

global:
  provider: "aks"
  deployment:
    name: "pega"
  actions:
    execute: "deploy"

```

```

customerDeploymentId: "pega26-sit"
jdbc:
  url: "<jdbc-url>"
  driverClass: "<jdbc-driver-class>"
  dbType: "<db-type>"
  driverUri: "<driver-uri>"
  external_secret_name: "pega-db-secret"
  rulesSchema: "<rules-schema>"
  dataSchema: "<data-schema>"
docker:
  registry:
    url: "<acr-name>.azurecr.io"
  imagePullSecretNames: []
  pega:
    image: "<acr-name>.azurecr.io/platform/pega:26.1.1"
tier:
- name: "web"
  nodeType: "WebUser"
  replicas: 1
  service:
    port: 80
    targetPort: 8080
    tls:
      enabled: true
      external_secret_names: ["pega-tier-tls"]
      port: 443
      targetPort: 8443
  ingress:
    enabled: true
    domain: "<pega-host>"
    annotations:
      appgw.ingress.kubernetes.io/request-timeout: "<seconds>"
    tls:
      enabled: true
  resources:
    requests:
      memory: "12Gi"
      cpu: 3
    limits:
      memory: "12Gi"
      cpu: 4
  javaOpts: >-
    -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
    -XX:MaxMetaspaceSize=768m
    -XX:+UseStringDeduplication
    -XX:+HeapDumpOnOutOfMemoryError
  hpa:
    enabled: true
    minReplicas: 1
    maxReplicas: 3
  pdb:
    enabled: true
    minAvailable: 1
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: ScheduleAnyway
      labelSelector:
        matchLabels:
          app: pega-web
- name: "batch"
  nodeType: "BackgroundProcessing,Search,Batch,RealTime,\
    Custom1,Custom2,Custom3,Custom4,Custom5,BIX"
  replicas: 1
  hpa:
    enabled: true
    minReplicas: 1
    maxReplicas: 3
  resources:
    requests:
      memory: "12Gi"
      cpu: 3
    limits:
      memory: "12Gi"
      cpu: 4

```

```

javaOpts: >-
  -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
  -XX:MaxMetaspaceSize=768m
  -XX:+UseStringDeduplication
  -XX:+HeapDumpOnOutOfMemoryError
pdb:
  enabled: true
  minAvailable: 1
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: ScheduleAnyway
    labelSelector:
      matchLabels:
        app: pega-batch
cassandra:
  enabled: false
hazelcast:
  enabled: false
  clusteringServiceEnabled: false
stream:
  enabled: true
  bootstrapServer: "<bootstrap-host>:9092"
  securityProtocol: SASL_SSL
  saslMechanism: PLAIN
  trustStore: ""
  keyStore: ""
  jaasConfig: ""
  streamNamePattern: "pega-sit-{stream.name}"
  replicationFactor: "3"
  external_secret_name: "pega-stream-secret"
pegasearch:
  externalSearchService: true
  externalURL: "https://srs-sit.srs-sit.svc.cluster.local:8443"
  srsAuth:
    enabled: true
    url: "https://<okta-domain>/oauth2/<auth-server-id>/v1/token"
    clientId: "<okta-client-id>"
    scopes: "pega.search:full"
    authType: "private_key_jwt"
    privateKeyAlgorithm: "RS256"
    external_secret_name: "pega-srs-oauth"
  srsMTLS:
    enabled: false
installer:
  image: "<acr-name>.azurecr.io/platform/installer:26.1.1"
  upgrade:
    upgradeType: "in-place"
    targetRulesSchema: ""
    targetDataSchema: ""

```

B.4 SIT: backingservices chart (SRS)

backingservices-sit.yaml

```

global:
  k8sProvider: "aks"
  imageCredentials:
    registry: "<acr-name>.azurecr.io"
srs:
  enabled: true
  deploymentName: "srs-sit"
  srsRuntime:
    replicaCount: 2
    srsImage: "<acr-name>.azurecr.io/platform-services/search-n-reporting-service-os:<tag>"
  resources:
    requests:
      cpu: 1
      memory: "4Gi"
    limits:
      cpu: 2
      memory: "4Gi"
  affinity:

```

```

    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            topologyKey: topology.kubernetes.io/zone
            labelSelector:
              matchLabels:
                app.kubernetes.io/name: srs-service
    env:
      AuthEnabled: true
      OAuthPublicKeyURL: "<okta-jwks-url>"
    ssl:
      enabled: true
      clientAuthentication: "want"
      keystore:
        file: "srs-keystore.p12"
        type: "PKCS12"
      truststore:
        file: "srs-truststore.jks"
        type: "JKS"
      certsSecret: "srs-runtime-certs"
    srsStorage:
      provisionInternalESCluster: false
      domain: "<search-host>"
      port: 443
      protocol: https
      tls:
        enabled: false
      basicAuthentication:
        enabled: true
      authSecret: "srs-search-credentials"
      requireInternetAccess: false
      networkPolicy:
        enabled: false

```

B.5 SIT: network policies for SRS

netpol-srs-sit.yaml (applied with `kubectl apply`, Section 7.7)

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: srs-sit-default-deny
  namespace: srs-sit
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: srs-sit-allow
  namespace: srs-sit
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: srs-service
  policyTypes: ["Ingress", "Egress"]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: pega-sit
      ports:
        - protocol: TCP
          port: 8443
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
      podSelector:
        matchLabels:
          k8s-app: kube-dns

```

```

ports:
  - protocol: UDP
    port: 53
  - protocol: TCP
    port: 53
- to:
  - ipBlock:
    cidr: "<opensearch-private-endpoint-cidr>"
ports:
  - protocol: TCP
    port: 443
- to:
  - ipBlock:
    cidr: "0.0.0.0/0"
    except: ["10.0.0.0/8", "172.16.0.0/12", "192.168.0.0/16"]
ports:
  - protocol: TCP
    port: 443

```

B.6 PROD: pega chart, upgrade run

pega-prd-upgrade.yaml. Upgrade run: installer job only, no Pega pods.

```

global:
  provider: "aks"
  deployment:
    name: "pega"
  actions:
    execute: "upgrade"
  customerDeploymentId: "pega26-prd"
  jdbc:
    url: "<jdbc-url>"
    driverClass: "<jdbc-driver-class>"
    dbType: "<db-type>"
    driverUri: "<driver-uri>"
    external_secret_name: "pega-db-secret"
    rulesSchema: "<rules-schema>"
    dataSchema: "<data-schema>"
  docker:
    registry:
      url: "<acr-name>.azurecr.io"
    imagePullSecretNames: []
    pega:
      image: "<acr-name>.azurecr.io/platform/pega:26.1.1"
  tier:
    - name: "web"
      nodeType: "WebUser"
      replicas: 3
      service:
        port: 80
        targetPort: 8080
        tls:
          enabled: true
          external_secret_names: ["pega-tier-tls"]
          port: 443
          targetPort: 8443
      ingress:
        enabled: true
        domain: "<pega-host>"
        annotations:
          appgw.ingress.kubernetes.io/request-timeout: "<seconds>"
        tls:
          enabled: true
      resources:
        requests:
          memory: "12Gi"
          cpu: 3
        limits:
          memory: "12Gi"
          cpu: 4
      javaOpts: >-
        -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
-XX:MaxMetaspaceSize=768m

```

```

-XX:+UseStringDeduplication
-XX:+HeapDumpOnOutOfMemoryError
hpa:
  enabled: true
  minReplicas: 3
  maxReplicas: 6
pdb:
  enabled: true
  minAvailable: 1
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: ScheduleAnyway
    labelSelector:
      matchLabels:
        app: pega-web
- name: "batch"
  nodeType: "BackgroundProcessing,Search,Batch,RealTime,\
    Custom1,Custom2,Custom3,Custom4,Custom5,BIX"
  replicas: 3
  hpa:
    enabled: true
    minReplicas: 3
    maxReplicas: 6
  resources:
    requests:
      memory: "12Gi"
      cpu: 3
    limits:
      memory: "12Gi"
      cpu: 4
  javaOpts: >-
    -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
-XX:MaxMetaspaceSize=768m
-XX:+UseStringDeduplication
-XX:+HeapDumpOnOutOfMemoryError
pdb:
  enabled: true
  minAvailable: 1
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: ScheduleAnyway
    labelSelector:
      matchLabels:
        app: pega-batch
cassandra:
  enabled: false
hazelcast:
  enabled: false
  clusteringServiceEnabled: false
stream:
  enabled: true
  bootstrapServer: "<bootstrap-host>:9092"
  securityProtocol: SASL_SSL
  saslMechanism: PLAIN
  trustStore: ""
  keyStore: ""
  jaasConfig: ""
  streamNamePattern: "pega-prd-{stream.name}"
  replicationFactor: "3"
  external_secret_name: "pega-stream-secret"
pegasearch:
  externalSearchService: true
  externalURL: "https://srs-prd.srs-prd.svc.cluster.local:8443"
  srsAuth:
    enabled: true
    url: "https://<okta-domain>/oauth2/<auth-server-id>/v1/token"
    clientId: "<okta-client-id>"
    scopes: "pega.search:full"
    authType: "private_key_jwt"
    privateKeyAlgorithm: "RS256"
    external_secret_name: "pega-srs-oauth"
  srsMTLS:
    enabled: false

```

```

installer:
  image: "<acr-name>.azurecr.io/platform/installer:26.1.1"
  upgrade:
    upgradeType: "in-place"
    targetRulesSchema: ""
    targetDataSchema: ""

```

B.7 PROD: pega chart, first-start run

pega-prd-first.yaml. First-start run: web tier only, batch tier held at zero.

```

global:
  provider: "aks"
  deployment:
    name: "pega"
  actions:
    execute: "deploy"
  customerDeploymentId: "pega26-prd"
  jdbc:
    url: "<jdbc-url>"
    driverClass: "<jdbc-driver-class>"
    dbType: "<db-type>"
    driverUri: "<driver-uri>"
    external_secret_name: "pega-db-secret"
    rulesSchema: "<rules-schema>"
    dataSchema: "<data-schema>"
  docker:
    registry:
      url: "<acr-name>.azurecr.io"
      imagePullSecretNames: []
    pega:
      image: "<acr-name>.azurecr.io/platform/pega:26.1.1"
  tier:
    - name: "web"
      nodeType: "WebUser"
      replicas: 3
      service:
        port: 80
        targetPort: 8080
        tls:
          enabled: true
          external_secret_names: ["pega-tier-tls"]
          port: 443
          targetPort: 8443
      ingress:
        enabled: true
        domain: "<pega-host>"
        annotations:
          appgw.ingress.kubernetes.io/request-timeout: "<seconds>"
        tls:
          enabled: true
      resources:
        requests:
          memory: "12Gi"
          cpu: 3
        limits:
          memory: "12Gi"
          cpu: 4
      javaOpts: >-
        -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
        -XX:MaxMetaspaceSize=768m
        -XX:+UseStringDeduplication
        -XX:+HeapDumpOnOutOfMemoryError
      hpa:
        enabled: true
        minReplicas: 3
        maxReplicas: 6
      pdb:
        enabled: true
        minAvailable: 1
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone

```

```

        whenUnsatisfiable: ScheduleAnyway
        labelSelector:
          matchLabels:
            app: pega-web
- name: "batch"
  nodeType: "BackgroundProcessing,Search,Batch,RealTime,\
  Custom1,Custom2,Custom3,Custom4,Custom5,BIX"
  replicas: 0
  hpa:
    enabled: false
  resources:
    requests:
      memory: "12Gi"
      cpu: 3
    limits:
      memory: "12Gi"
      cpu: 4
  javaOpts: >-
    -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
    -XX:MaxMetaspaceSize=768m
    -XX:+UseStringDeduplication
    -XX:+HeapDumpOnOutOfMemoryError
  pdb:
    enabled: true
    minAvailable: 1
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: ScheduleAnyway
      labelSelector:
        matchLabels:
          app: pega-batch
cassandra:
  enabled: false
hazelcast:
  enabled: false
  clusteringServiceEnabled: false
stream:
  enabled: true
  bootstrapServer: "<bootstrap-host>:9092"
  securityProtocol: SASL_SSL
  saslMechanism: PLAIN
  trustStore: ""
  keyStore: ""
  jaasConfig: ""
  streamNamePattern: "pega-prd-{stream.name}"
  replicationFactor: "3"
  external_secret_name: "pega-stream-secret"
pegasearch:
  externalSearchService: true
  externalURL: "https://srs-prd.srs-prd.svc.cluster.local:8443"
  srsAuth:
    enabled: true
    url: "https://<okta-domain>/oauth2/<auth-server-id>/v1/token"
    clientId: "<okta-client-id>"
    scopes: "pega.search:full"
    authType: "private_key_jwt"
    privateKeyAlgorithm: "RS256"
    external_secret_name: "pega-srs-oauth"
  srsMTLS:
    enabled: false
installer:
  image: "<acr-name>.azurecr.io/platform/installer:26.1.1"
  upgrade:
    upgradeType: "in-place"
    targetRulesSchema: ""
    targetDataSchema: ""

```

B.8 PROD: pega chart, steady-state deploy run

pega-prd-deploy.yaml. Steady-state deploy run.

```
global:
```

```

provider: "aks"
deployment:
  name: "pega"
actions:
  execute: "deploy"
customerDeploymentId: "pega26-prd"
jdbc:
  url: "<jdbc-url>"
  driverClass: "<jdbc-driver-class>"
  dbType: "<db-type>"
  driverUri: "<driver-uri>"
  external_secret_name: "pega-db-secret"
  rulesSchema: "<rules-schema>"
  dataSchema: "<data-schema>"
docker:
  registry:
    url: "<acr-name>.azurecr.io"
  imagePullSecretNames: []
  pega:
    image: "<acr-name>.azurecr.io/platform/pega:26.1.1"
tier:
- name: "web"
  nodeType: "WebUser"
  replicas: 3
  service:
    port: 80
    targetPort: 8080
    tls:
      enabled: true
      external_secret_names: ["pega-tier-tls"]
      port: 443
      targetPort: 8443
  ingress:
    enabled: true
    domain: "<pega-host>"
    annotations:
      appgw.ingress.kubernetes.io/request-timeout: "<seconds>"
    tls:
      enabled: true
  resources:
    requests:
      memory: "12Gi"
      cpu: 3
    limits:
      memory: "12Gi"
      cpu: 4
  javaOpts: >-
    -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
    -XX:MaxMetaspaceSize=768m
    -XX:+UseStringDeduplication
    -XX:+HeapDumpOnOutOfMemoryError
  hpa:
    enabled: true
    minReplicas: 3
    maxReplicas: 6
  pdb:
    enabled: true
    minAvailable: 1
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: ScheduleAnyway
      labelSelector:
        matchLabels:
          app: pega-web
- name: "batch"
  nodeType: "BackgroundProcessing,Search,Batch,RealTime,\
Custom1,Custom2,Custom3,Custom4,Custom5,BIX"
  replicas: 3
  hpa:
    enabled: true
    minReplicas: 3
    maxReplicas: 6
  resources:
    requests:

```

```

        memory: "12Gi"
        cpu: 3
      limits:
        memory: "12Gi"
        cpu: 4
      javaOpts: >-
        -Xlog:gc*,gc+heap=debug,gc+humongous=debug:file=/usr/local/tomcat/logs/
gc.log:uptime,pid,level,time,tags:filecount=3,filesize=2M
        -XX:MaxMetaspaceSize=768m
        -XX:+UseStringDeduplication
        -XX:+HeapDumpOnOutOfMemoryError
      pdb:
        enabled: true
        minAvailable: 1
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: pega-batch
cassandra:
  enabled: false
hazelcast:
  enabled: false
  clusteringServiceEnabled: false
stream:
  enabled: true
  bootstrapServer: "<bootstrap-host>:9092"
  securityProtocol: SASL_SSL
  saslMechanism: PLAIN
  trustStore: ""
  keyStore: ""
  jaasConfig: ""
  streamNamePattern: "pega-prd-{stream.name}"
  replicationFactor: "3"
  external_secret_name: "pega-stream-secret"
pegasearch:
  externalSearchService: true
  externalURL: "https://srs-prd.srs-prd.svc.cluster.local:8443"
  srsAuth:
    enabled: true
    url: "https://<okta-domain>/oauth2/<auth-server-id>/v1/token"
    clientId: "<okta-client-id>"
    scopes: "pega.search:full"
    authType: "private_key_jwt"
    privateKeyAlgorithm: "RS256"
    external_secret_name: "pega-srs-oauth"
  srsMTLS:
    enabled: false
installer:
  image: "<acr-name>.azurecr.io/platform/installer:26.1.1"
  upgrade:
    upgradeType: "in-place"
    targetRulesSchema: ""
    targetDataSchema: ""

```

B.9 PROD: backingservices chart (SRS)

backingservices-prd.yaml

```

global:
  k8sProvider: "aks"
  imageCredentials:
    registry: "<acr-name>.azurecr.io"
srs:
  enabled: true
  deploymentName: "srs-prd"
  srsRuntime:
    replicaCount: 3
    srsImage: "<acr-name>.azurecr.io/platform-services/search-n-reporting-service-os:<tag>"
  resources:
    requests:
      cpu: 1

```

```

    memory: "4Gi"
  limits:
    cpu: 2
    memory: "4Gi"
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            topologyKey: topology.kubernetes.io/zone
            labelSelector:
              matchLabels:
                app.kubernetes.io/name: srs-service
  env:
    AuthEnabled: true
    OAuthPublicKeyURL: "<okta-jwks-url>"
  ssl:
    enabled: true
    clientAuthentication: "want"
    keystore:
      file: "srs-keystore.p12"
      type: "PKCS12"
    truststore:
      file: "srs-truststore.jks"
      type: "JKS"
    certsSecret: "srs-runtime-certs"
  srsStorage:
    provisionInternalESCluster: false
    domain: "<search-host>"
    port: 443
    protocol: https
    tls:
      enabled: false
    basicAuthentication:
      enabled: true
    authSecret: "srs-search-credentials"
    requireInternetAccess: false
    networkPolicy:
      enabled: false

```

B.10 PROD: network policies for SRS

netpol-srs-prd.yaml (applied with `kubectl apply`, Section 7.7)

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: srs-prd-default-deny
  namespace: srs-prd
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: srs-prd-allow
  namespace: srs-prd
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: srs-service
  policyTypes: ["Ingress", "Egress"]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: pega-prd
      ports:
        - protocol: TCP
          port: 8443
  egress:
    - to:
        - namespaceSelector:

```

```
      matchLabels:
        kubernetes.io/metadata.name: kube-system
    podSelector:
      matchLabels:
        k8s-app: kube-dns
  ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
- to:
  - ipBlock:
      cidr: "<opensearch-private-endpoint-cidr>"
  ports:
    - protocol: TCP
      port: 443
- to:
  - ipBlock:
      cidr: "0.0.0.0/0"
      except: ["10.0.0.0/8", "172.16.0.0/12", "192.168.0.0/16"]
  ports:
    - protocol: TCP
      port: 443
```

Appendix C. Command reference

Run these commands from a tooling pod in `pega-<code>`, so they use the same network path, DNS and egress rules as Pega, unless the command says otherwise. Use a tooling image mirrored into the customer registry. Read secrets from Key Vault at run time and keep them out of shell history. Check option names against the installed tool versions before use.

C.1 Kafka and Confluent Cloud

Kafka commands K-1 to K-7

```
# K-1 DNS: bootstrap and broker names must resolve to private IPs
nslookup <bootstrap-host>
nslookup <broker-host>

# K-2 TLS: certificate chain and protocol
openssl s_client -connect <bootstrap-host>:9092 -servername <bootstrap-host> </dev/null

# K-3 Authentication and metadata: every broker listed and reachable
kcat -b <bootstrap-host>:9092 -X security.protocol=SASL_SSL -X sasl.mechanisms=PLAIN \
-X sasl.username="$API_KEY" -X sasl.password="$API_SECRET" -L

# K-4 Topics under the environment prefix
confluent kafka topic list --cluster <lkc-id> | grep "pega-<code>-"

# K-5 Partition count under the prefix
kcat -b <bootstrap-host>:9092 -X security.protocol=SASL_SSL -X sasl.mechanisms=PLAIN \
-X sasl.username="$API_KEY" -X sasl.password="$API_SECRET" -L -J \
| jq --arg p "pega-<code>-" \
' [.topics[] | select(.topic | startswith($p)) | .partitions | length] | add'

# K-6 Topic configuration and the message size limit
confluent kafka topic describe <topic> --cluster <lkc-id>
confluent kafka topic update <topic> --config max.message.bytes=5000000 \
--cluster <lkc-id>

# K-7 ACLs for the environment's service account
confluent kafka acl list --service-account <sa-id> --cluster <lkc-id>
```

K-8 runs against the customer's own Kafka cluster that 8.8 application data sets read, with that cluster's client properties. K-9 runs only during a refresh or retirement, after the environment's Pega tiers are at zero.

Kafka commands K-8 and K-9

```
# K-8 Committed offsets of an application consumer group (customer cluster)
kafka-consumer-groups.sh --bootstrap-server <customer-bootstrap> \
--command-config customer-client.properties --describe --group <group>

# K-9 Delete the environment's topics and consumer groups (refresh or retirement)
confluent kafka topic list --cluster <lkc-id> -o json \
| jq -r '.[].name | select(startswith("pega-<code>"))' > topics.txt
xargs -n1 confluent kafka topic delete --force --cluster <lkc-id> < topics.txt
kafka-consumer-groups.sh --bootstrap-server <bootstrap-host>:9092 \
--command-config sa-pega-<code>.properties --list | grep "^pega-<code>-" > groups.txt
xargs -n1 kafka-consumer-groups.sh --bootstrap-server <bootstrap-host>:9092 \
--command-config sa-pega-<code>.properties --delete --group < groups.txt
```

Caution. Review `topics.txt` and `groups.txt` before running the delete lines. Every name must start with this environment's prefix. The prefixed ACLs stop the service account from deleting another environment's topics, but the review is still required.

C.2 OpenSearch and SRS

Search commands S-1 to S-6

```
# S-1 DNS and version: private IP; version 2.15 or 2.19
nslookup <search-host>
curl -s -u "$OS_USER:$OS_PASSWORD" https://<search-host>/

# S-2 Cluster health (green expected)
curl -s -u "$OS_USER:$OS_PASSWORD" "https://<search-host>/_cluster/health?pretty"

# S-3 Settings required by SRS (administrator)
curl -s -u "$OS_ADMIN:$OS_ADMIN_PASSWORD" \
  "https://<search-host>/_cluster/settings?pretty&include_defaults=false"

# S-4 Indexes under the environment's customerDeploymentId
curl -s -u "$OS_USER:$OS_PASSWORD" "https://<search-host>/_cat/indices/pega26-<code>?*v"

# S-5 SRS from the Pega namespace: refused without a token, accepted with one
SRS=https://srs-<code>.srs-<code>.svc.cluster.local:8443
curl -s --cacert srs-ca.crt "$SRS/health"
curl -s --cacert srs-ca.crt -o /dev/null -w "%{http_code}\n" "$SRS/"
curl -s --cacert srs-ca.crt -o /dev/null -w "%{http_code}\n" \
  -H "Authorization: Bearer $TOKEN" "$SRS/"

# S-6 Delete the environment's indexes (refresh or retirement, as the SRS user)
curl -s -u "$OS_USER:$OS_PASSWORD" \
  "https://<search-host>/_cat/indices/pega26-<code>?*h=index"
curl -s -u "$OS_USER:$OS_PASSWORD" -X DELETE "https://<search-host>/pega26-<code>?"
```

For S-5, `srs-ca.crt` is the customer private CA that signed the SRS server certificate. The health call is the same one the SRS readiness probe makes, and must return a healthy status. The call without a token must return 401. The call with a token must not return 401 or 403; the exact code depends on the path, which is not part of the check. S-6 relies on `destructive_requires_name` being false (OS-2) and on the index-scoped role, which limits the delete to this environment's indexes.

C.3 Okta tokens

O-1 builds the signed client assertion that `private_key_jwt` needs and requests a token, as Pega does [R43, R45]. The assertion carries the client ID as issuer and subject, the token endpoint as audience, a unique ID and a short expiry. Run it from the Pega namespace, so the egress path to `<okta-domain>` is tested too. It needs Python with the PyJWT and cryptography packages in the tooling image.

Okta commands O-1 and O-2

```
# O-1 Token request with private_key_jwt
export TOKEN_URL="https://<okta-domain>/oauth2/<auth-server-id>/v1/token"
export CLIENT_ID="<okta-client-id>"
ASSERTION=$(python3 - <<'EOF'
import os, time, uuid, jwt
now = int(time.time())
claims = {"iss": os.environ["CLIENT_ID"], "sub": os.environ["CLIENT_ID"],
          "aud": os.environ["TOKEN_URL"], "iat": now, "exp": now + 300,
          "jti": str(uuid.uuid4())}
key = open("/tmp/okta-srs-private-key.pem").read()
print(jwt.encode(claims, key, algorithm="RS256", headers={"kid": "<kid>"}))
EOF
)
TOKEN=$(curl -s -X POST "$TOKEN_URL" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d grant_type=client_credentials -d scope=pega.search:full \
  -d client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer \
  -d client_assertion="$ASSERTION" | jq -r .access_token)
rm -f /tmp/okta-srs-private-key.pem

# O-2 Decode the token claims without verifying them.
# Check iss, scp (contains pega.search:full), guid (pega26-<code>) and exp.
# Do not paste production tokens into web tools.
python3 - "$TOKEN" <<'EOF'
import base64, json, sys
payload = sys.argv[1].split(".")[1]
payload += "=" * (-len(payload) % 4)
print(json.dumps(json.loads(base64.urlsafe_b64decode(payload)), indent=2))
EOF
```

C.4 Kubernetes and Helm

Kubernetes and Helm commands P-1 to P-6

```
# P-1 Pod status
kubectl get pods -n pega-<code> -o wide
kubectl get pods -n srs-<code> -o wide

# P-2 Pod logs: stream and search errors
kubectl logs -n pega-<code> <pod> --since=30m | grep -Ei "stream|kafka|srs|search"

# P-3 Rolling restart of a tier
kubectl rollout restart deployment/pega-<tier> -n pega-<code>
kubectl rollout status deployment/pega-<tier> -n pega-<code>

# P-4 External secret synchronization (SecretSynced expected)
kubectl get externalsecret -n pega-<code>
kubectl get externalsecret -n srs-<code>

# P-5 Installer job log
kubectl get jobs -n pega-<code>
kubectl logs -n pega-<code> job/<installer-job> -f

# P-6 Render and compare values before any Helm run
helm template pega pega/pega --version 4.13.0 -n pega-<code> \
  -f pega-<code>-<run>.yaml > rendered.yaml
helm diff upgrade pega pega/pega --version 4.13.0 -n pega-<code> \
  -f pega-<code>-<run>.yaml
```

P-6 uses the helm-diff plug-in. If it is not allowed, compare the rendered output with the output of the previous run instead.

Appendix D. Evidence and rehearsal templates

Each environment build, refresh, rehearsal and cutover produces an evidence pack, stored with the change record. The templates fix what each record holds, so rehearsal and production results can be compared line by line.

D.1 Evidence pack contents

Table 94. Evidence pack items

Item	Content	Produced by
E-01 Values diff	P-6 output against the previous run, secrets excluded	Platform team
E-02 Kafka connectivity	K-1 to K-3 output from a pod in <code>pega -<code></code>	Kafka engineer
E-03 Kafka objects	K-4 to K-7 output	Kafka engineer
E-04 Search connectivity	S-1 to S-3 and S-5 output	Search engineer
E-05 Token claims	O-2 output (claims only; never the token)	Identity team
E-06 Clean-up record	One line per CD item (template below)	Pega LSA
E-07 Landing pages	Stream and search landing page screenshots showing the Helm values	Pega LSA
E-08 Index record	S-4 output, build time and count sheet	Pega LSA
E-09 Test records	Isolation, functional, performance and failure scenario results	Test lead
E-10 Timings	Start and end time of each step	Cutover manager

D.2 Clean-up record

Table 95. Clean-up record (one per clone)

CD item	Found in the clone	Action taken	Done by and time	Check
CD-01				
CD-02				
CD-03				
CD-04				
CD-05				
CD-06				
CD-07				
CD-08				
CD-09				
CD-10				
CD-11				
CD-12				
CD-13				
CD-14				
CD-15				

CD item	Found in the clone	Action taken	Done by and time	Check
CD-16				

D.3 Check record

Table 96. Check record

Field	Entry
Check or scenario ID	
Environment and occasion	
Run by and date	
Command or screen	
Expected result	
Actual result	
Result (Pass, Fail, Pass with note)	
Evidence item	
Deviation and action	

D.4 Timed cutover record

Table 97. Timed cutover record (rehearsals and production)

Step	Planned start	Actual start	Actual end	Duration	Notes
C-1					
C-2					
C-3					
C-4					
C-5					
C-6					
C-7					
C-8					
C-9					
C-10					

D.5 Go/No-Go record

Table 98. Go/No-Go record

Gate	Criteria met	Evidence items	Decision owner	Decision and time
Go/No-Go 1			Business owner with platform lead	
First-start gate			Cutover manager	

Gate	Criteria met	Evidence items	Decision owner	Decision and time
Go/No-Go 2			Cutover manager	
Go/No-Go 3			Cutover manager with business owner	

D.6 Retirement record

Table 99. Retirement record

Item	Deleted by and date	Evidence
Helm releases and namespaces		
Topics and consumer groups (K-9)		
ACLs, API keys and service account		
Indexes (S-6), OpenSearch user and role		
Okta client and guid expression entry		
Key Vault secrets and vault		
Firewall rules and DNS records		

Appendix E. Glossary

Table 100. Glossary of terms

Term	Meaning
ACL	Access control list. In Kafka, a rule that allows a principal an operation on a resource such as a topic or consumer group.
AKS	Azure Kubernetes Service.
Authorization server (Okta)	The Okta component that issues access tokens. A custom authorization server allows custom scopes and claims.
Clone-and-upgrade	The path in this document: copy the 8.8 database, upgrade the copy to 26.1.1 with the installer job, and start 26.1.1 on it.
CodeSet	A Pega rule that groups Java libraries imported into Pega Platform.
Confluent Cloud	Managed Apache Kafka service from Confluent.
customerDeploymentId	The ID that SRS uses to separate each Pega environment's data. It becomes the index name prefix and must match the guid claim.
Data set (Kafka)	A Pega rule that reads from or writes to a Kafka topic for an application integration. It is separate from the stream service.
DSS	Dynamic System Setting, a configuration record in the Pega database.
External Secrets Operator	A Kubernetes operator that copies secrets from Azure Key Vault into Kubernetes secrets.
Go/No-Go	A decision point with written criteria, at which the cutover continues or stops.
guid	The token claim that SRS compares with the customerDeploymentId of the environment.
Hazelcast	The in-memory clustering library used by Pega Platform up to '24, removed in '25.
Helm	The package manager for Kubernetes used to deploy the pega and backingservices charts.
Index build	Full reindex of Pega data into SRS from the upgraded database.
Installer job	The Kubernetes job the pega chart runs to install or upgrade the Pega database.
JAAS	Java Authentication and Authorization Service. The Kafka client reads its SASL credentials from a JAAS configuration string.
JWKS	JSON Web Key Set, the public keys that SRS uses to check token signatures.
Masking	Replacing personal or sensitive data in a non-production copy of the database.
OpenSearch	Open-source search engine. SRS stores Pega indexes in it.
Private Link	Azure service that exposes a service through a private IP in the customer VNet.
private_key_jwt	Client authentication in which the client signs an assertion with its private key instead of sending a secret.
Queue processor	A Pega rule that processes queued items asynchronously through the stream service.
SASL_SSL	Kafka security protocol that combines TLS encryption with SASL authentication.
Service group	The set of environments that share one Confluent cluster and one OpenSearch service (NP1, NP2) or the dedicated production set (PRD).
SRS	Search and Reporting Service, the Pega backing service between Pega Platform and OpenSearch.
Stream service	The Pega service that uses Kafka for queue processors, job schedulers, data flows and, from '25, cluster messaging.
streamNamePattern	Helm setting that defines how Pega names its Kafka topics. It carries the environment prefix.

Appendix F. References

Sources were read between 28 September and 1 October 2026. Vendor pages change; check each page again before each rehearsal (RK-12).

- [R1] Pega Documentation. Platform Support Guide, Pega Platform '26.
<https://docs.pega.com/bundle/platform/page/platform/deployment/platform-support-guide/platform-support-guide.html>
- [R2] Pega Documentation. Adopting Pega Platform infrastructure changes.
<https://docs.pega.com/bundle/platform/page/platform/deployment/adopting-infrastructure-changes.html>
- [R3] Pega Documentation. Completing prerequisites to update to Pega Platform '26.
<https://docs.pega.com/bundle/platform/page/platform/deployment/client-managed-cloud/prereq-pegaplatform-26.html>
- [R4] Pega Documentation. Prerequisites for Pega Platform '25.
<https://docs.pega.com/bundle/platform/page/platform/deployment/client-managed-cloud/prereq-pegaplatform-25.html>
- [R5] Pega Documentation. Removing Hazelcast from your deployment.
<https://docs.pega.com/bundle/platform/page/platform/deployment/externalization-of-services/removing-hazelcast.html>
- [R6] Pega Documentation. Performing prerequisite steps (Hazelcast removal). <https://docs.pega.com/bundle/platform-25/page/platform/deployment/externalization-of-services/hz-removal-prereq.html>
- [R7] Pega Documentation. Removing Hazelcast FAQ.
<https://docs.pega.com/bundle/platform-25/page/platform/deployment/externalization-of-services/removing-hazelcast-faq.html>
- [R8] Pega Documentation. Configuring a prpcUtils streaming service.
<https://docs.pega.com/bundle/platform/page/platform/deployment/client-managed-cloud/configuring-prpcutils-streaming.html>
- [R9] Pega Documentation. Updating Pega Platform to Tomcat 10.1.
<https://docs.pega.com/bundle/platform/page/platform/deployment/tomcat-10-prepare-overview.html>
- [R10] Pega Documentation. Create primary key constraints for tables in your database (primaryKeyUtility).
<https://docs.pega.com/bundle/platform/page/platform/deployment/run-primarykeyutility.html>
- [R11] Pega Documentation. External Kafka in your deployment.
<https://docs.pega.com/bundle/platform/page/platform/deployment/externalization-of-services/externalize-kafka-in-your-deployment.html>
- [R12] Pega Documentation. Configuring a containerized deployment to use a Kafka service.
<https://docs.pega.com/bundle/platform/page/platform/deployment/externalization-of-services/deploy-external-kafka.html>
- [R13] Pega Documentation. Switching Kafka providers while preserving Stream data.
<https://docs.pega.com/bundle/platform/page/platform/deployment/externalization-of-services/migrate-external-kafka-providers.html>
- [R14] Pega Documentation. External Search in your deployment.
<https://docs.pega.com/bundle/platform/page/platform/deployment/externalization-of-services/externalize-search-in-your-deployment.html>
- [R15] Pega Documentation. Connecting Pega Platform to SRS.
<https://docs.pega.com/bundle/platform/page/platform/deployment/externalization-of-services/connect-pegaplatform-to-srs.html>
- [R16] Pega Documentation. Connecting a new containerized deployment to SRS.
<https://docs.pega.com/bundle/platform/page/platform/deployment/externalization-of-services/connect-new-k8s-to-srs.html>
- [R17] Pega Documentation. Rebuilding search indexes. <https://docs.pega.com/bundle/platform/page/platform/search-engine/rebuilding-search-indexes.html>
- [R18] Pega Documentation. Indexing of data. <https://docs.pega.com/bundle/platform/page/platform/search-engine/indexing-data.html>
- [R19] Pega Documentation. Checking search index status. <https://docs.pega.com/bundle/platform/page/platform/search-engine/search-index-status-checking.html>
- [R20] Pega Documentation. Meeting requirements and prerequisites (client-managed cloud).
<https://docs.pega.com/bundle/platform/page/platform/deployment/client-managed-cloud/requirements-prerequisites.html>

- [R21] Pega Documentation. Understanding the Pega Platform deployment architecture. <https://docs.pega.com/bundle/platform/page/platform/deployment/client-managed-cloud/pega-kubernetes-architecture.html>
- [R22] Pega Documentation. Third-party externalized services FAQs. <https://docs.pega.com/bundle/platform/page/platform/deployment/client-managed-cloud/third-party-externalized-services-faqs.html>
- [R23] Pegasystems, GitHub. Pega Helm charts: charts/pega/README.md (chart 4.13.0). <https://github.com/pegasystems/pega-helm-charts/blob/master/charts/pega/README.md>
- [R24] Pegasystems, GitHub. Pega Helm charts: backingservices SRS chart README. <https://github.com/pegasystems/pega-helm-charts/blob/master/charts/backingservices/charts/srs/README.md>
- [R25] Pegasystems, GitHub. Pega Helm charts: Switch from embedded Stream to externalized Kafka service. <https://github.com/pegasystems/pega-helm-charts/blob/master/charts/pega/MigrationToExternalStream.md>
- [R26] Pegasystems, GitHub. Pega Helm charts: Kafka cluster requirements. <https://github.com/pegasystems/pega-helm-charts/blob/master/charts/pega/KafkaClusterRequirement.md>
- [R27] Pegasystems, GitHub. Pega Helm charts: Deploying Pega Platform on AKS. <https://github.com/pegasystems/pega-helm-charts/blob/master/docs/Deploying-Pega-on-AKS.md>
- [R28] Pega Academy. Search and Reporting Service. <https://academy.pega.com/topic/search-and-reporting-service/v3>
- [R29] Pega Academy. Deployment architecture with external services. <https://academy.pega.com/topic/deployment-architecture-external-services/v1>
- [R30] Pega Academy. Cloud deployment architecture. <https://academy.pega.com/topic/cloud-deployment-architecture/v1>
- [R31] Confluent Documentation. Kafka cluster types in Confluent Cloud. <https://docs.confluent.io/cloud/current/clusters/cluster-types.html>
- [R32] Confluent Documentation. Use Azure Private Link for Dedicated clusters on Confluent Cloud. <https://docs.confluent.io/cloud/current/networking/private-links/azure-privatelink.html>
- [R33] Confluent Documentation. Configuration reference for topics in Confluent Cloud. <https://docs.confluent.io/cloud/current/client-apps/topics/manage.html>
- [R34] Confluent Documentation. Kafka cluster configuration (editable broker settings). <https://docs.confluent.io/cloud/current/clusters/broker-config.html>
- [R35] Confluent Documentation. Geo-replication with Cluster Linking on Confluent Cloud. <https://docs.confluent.io/cloud/current/multi-cloud/cluster-linking/index.html>
- [R36] Confluent Documentation. Multi-tenancy and client quotas on Confluent Cloud. <https://docs.confluent.io/cloud/current/clusters/client-quotas.html>
- [R37] OpenSearch Documentation. Creating a cluster. <https://docs.opensearch.org/latest/tuning-your-cluster/index/>
- [R38] OpenSearch Documentation. Snapshot and restore for migration. <https://docs.opensearch.org/latest/migrate-or-upgrade/snapshot-restore/>
- [R39] OpenSearch Documentation. Is Migration Assistant right for you? (supported migration paths). <https://docs.opensearch.org/latest/migration-assistant/is-migration-assistant-right-for-you/>
- [R40] OpenSearch Documentation. Defining users and roles. <https://docs.opensearch.org/latest/security/access-control/users-roles/>
- [R41] OpenSearch Documentation. Security permissions. <https://docs.opensearch.org/latest/security/access-control/permissions/>
- [R42] Okta Developer. Authorization servers. <https://developer.okta.com/docs/concepts/auth-servers/>
- [R43] Okta Developer. Implement authorization by grant type: client credentials. <https://developer.okta.com/docs/guides/implement-grant-type/clientcreds/main/>
- [R44] Okta Developer. Customize tokens returned from Okta with custom claims. <https://developer.okta.com/docs/guides/customize-tokens-returned-from-okta/main/>
- [R45] Okta Developer. Client authentication methods. <https://developer.okta.com/docs/api/openapi/okta-oauth/guides/client-auth/>
- [R46] Okta Developer. Okta Expression Language overview guide. <https://developer.okta.com/docs/reference/okta-expression-language/>

- [R47] Microsoft Learn. Azure Private Endpoint DNS configuration. <https://learn.microsoft.com/en-us/azure/private-link/private-endpoint-dns>
- [R48] External Secrets Operator. Azure Key Vault provider. <https://external-secrets.io/latest/provider/azure-key-vault/>
- [R49] Pega Documentation. Changing the default number of partitions per topic. <https://docs.pega.com/bundle/platform-241/page/platform/deployment/embedded-kafka-cassandra/change-default-partitions-count-stream-nodes.html>
- [R50] Pega Documentation. Creating a Queue Processor rule ('25). <https://docs.pega.com/bundle/platform-25/page/platform/background-processing/queue-processor-create-rule.html>
- [R51] Pega Documentation. Configuring Pega Diagnostic Center for on-premises systems. <https://docs.pega.com/bundle/pega-diagnostic-center/page/pega-diagnostic-center/guide/configuring-on-premises-systems.html>
- [R52] Okta Developer. Rate limits overview. <https://developer.okta.com/docs/reference/rate-limits/>
- [R53] Okta Developer. Key rotation. <https://developer.okta.com/docs/concepts/key-rotation/>
- [R54] Amazon OpenSearch Service Developer Guide. Sizing domains: storage requirements. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/bp-storage.html>
- [R55] Amazon OpenSearch Service Developer Guide. Shard strategy. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/bp-sharding.html>
- [R56] Confluent Documentation. Confluent Cloud Metrics API. <https://docs.confluent.io/cloud/current/monitoring/metrics-api.html>
- [R57] Confluent Documentation. Audit log concepts on Confluent Cloud. <https://docs.confluent.io/cloud/current/monitoring/audit-logging/cloud-audit-log-concepts.html>
- [R58] OpenSearch Documentation. Logs (slow logs, task logs, deprecation logs). <https://docs.opensearch.org/latest/install-and-configure/configuring-opensearch/logs/>
- [R59] Microsoft Learn. Availability zones in Azure Kubernetes Service. <https://learn.microsoft.com/en-us/azure/aks/availability-zones>
- [R60] Microsoft Learn. Kubernetes monitoring in Azure Monitor. <https://learn.microsoft.com/en-us/azure/azure-monitor/containers/kubernetes-monitoring-overview>
- [R61] Pegasystems, GitHub. Pega Helm charts: pega deployment template (chart 4.13.0). <https://github.com/pegasystems/pega-helm-charts/blob/master/charts/pega/templates/pega-deployment.tpl>
- [R62] Pegasystems, GitHub. Pega Helm charts: SRS network policy and service templates. https://github.com/pegasystems/pega-helm-charts/blob/master/charts/backingservices/charts/srs/templates/srsservice_networkpolicy.yaml
- [R63] Pegasystems, GitHub. Pega Helm charts: Recommended JVM arguments. <https://github.com/pegasystems/pega-helm-charts/blob/master/charts/pega/RecommendedJVMArgs.md>

Appendix G. Image sources and attribution

Figures marked "Prepared for this implementation" were drawn for this document from the facts cited in the surrounding text. They are not Pega-published diagrams. The figures below are reproduced from public vendor pages, unchanged except for scaling.

Table 101. Third-party images

Figure	Publisher	Source page
Pega Platform Kubernetes architecture for '26	Pega Documentation	[R21]
Earlier Pega cluster layout with embedded Kafka and in-cluster Elasticsearch	Pega Academy	[R30]
Externalized services in a Pega deployment	Pega Academy	[R29]
The two Kafka use cases in Pega Platform	Pega Documentation	[R11]
Azure Private Link between a customer VNet and Confluent Cloud	Confluent Documentation	[R32]
DNS records that Confluent lists for Azure Private Link	Confluent Documentation	[R32]
Client-managed cloud deployment with SRS as a backing service	Pega Academy	[R28]
OpenSearch cluster roles: cluster-manager and data nodes	OpenSearch Documentation	[R37]

Important. Pega, Pega Academy, Confluent and OpenSearch content is subject to each publisher's terms of use. Before this document is shared outside the customer and implementation teams, confirm that reuse of these images is allowed, or replace them with links to the source pages.

Appendix H. Sizing calculator

The sizing calculator is the Excel workbook `Pega_26_Sizing_Calculator.xlsx`, issued with this document. It applies the rules in Table 71 and Table 28 with live formulas. Yellow cells are inputs and green cells are formulas. The example inputs are illustrative, as in Section 15.9, and must be replaced with measured values.

Table 102. Sizing calculator sheets

Sheet	Inputs	Outputs	Sources of constants
Kafka	Per group: cluster type, peak ingress and egress, measured partitions, headroom, connections, requests per second	Units needed by each limit; capacity units; the limit that sets them; warnings above 10 and 32 eCKU	[R31, R49]
OpenSearch	Per service: primary data, replicas, growth, years, rebuild peak, indexes, shards per index, disk and heap per node	Storage; total shards; data nodes by storage, by shards and by zones; average shard size	[R54, R55]
AKS	Pega pods at HPA maximum, pod requests, node allocatable CPU and memory	Pods per node; nodes for the load; nodes with zone headroom and surge	[R23, R59]
Okta	Pods and measured token requests per pod per hour, per environment; org token endpoint limit	Token requests per minute; share of the org limit	[R52]

H.1 Example outputs

Table 103. Calculator example: Confluent capacity units (illustrative inputs)

Group	Type	Planned partitions	Units needed by	Units
NP1	Enterprise	3,510	ingress 1, egress 1, partitions 2, connections 1, requests 1, SLA minimum 2	2
NP2	Enterprise	2,340	ingress 1, egress 1, partitions 1, connections 1, requests 1, SLA minimum 2	2
PROD	Enterprise	1,170	ingress 1, egress 1, partitions 1, connections 1, requests 1, SLA minimum 2	2

Table 104. Calculator example: OpenSearch data nodes (illustrative inputs)

Service	Primary GiB	Storage GiB	Shards	Nodes by storage	Nodes by shards	Data nodes
os-np1	30	150.3	360	2	2	3
os-np2	80	400.9	240	5	2	6
os-prd	40	200.4	120	3	1	3

The workbook formulas were checked by recalculating the workbook in LibreOffice Calc and comparing every output with the Python functions that produce these tables. Recheck the constants on the vendor pages before each use.